WEB APPLICATION

SECURITY ASSESSMENT

REPORT OF

Apollo Tyres

CONFIDENTIAL

Apollo Sampark (B2B Application)

https://jsapps.chrijfcq3f-apollotyr1-s2-public.model-t.cc.commerce.ondemand.com/

20th March 2026

Disclaimer

The information shared in this report is confidential and may be legally privileged. It is intended solely for the addressee only and access to this report by anyone else is unauthorized. If you are not the intended recipient, any disclosure, copying,

CONFIDENTIAL

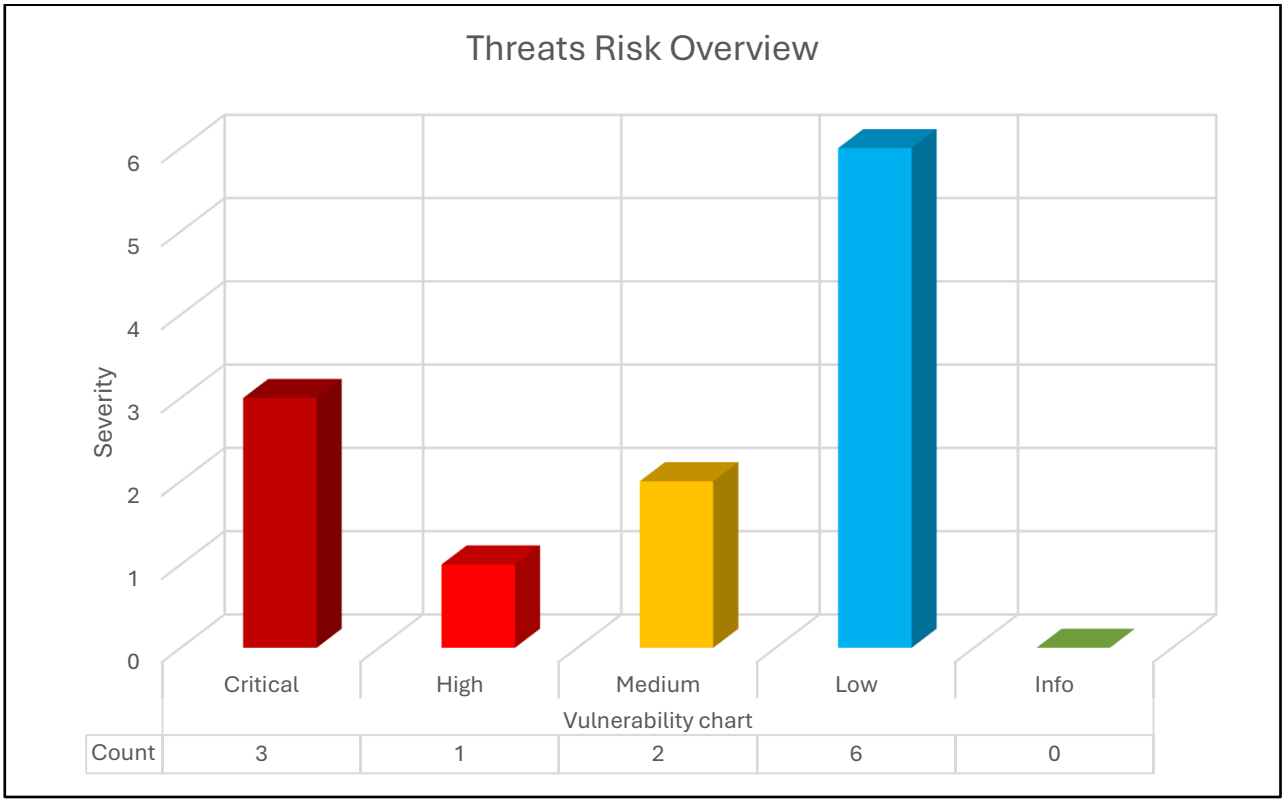
TABLE OF CONTENTS

1.PENTEST DETAILS

Application Name	**Apollo**
Application URL	Abc.com
Tester/Auditor	Urmil Savla
Tester Qualifications	BSc IT & CEH certifited
Pen testing Organization Name	Abc Pvt Ltd
Report Release Date	4th April 2026
Report Submitted To	Sundar Pichai – CEO Google
Report Submitted Org	Alphabet Inc
SPOC Contact	Urmil Savla (+91 7045188096)

The Objective

The following graph represents the total number of vulnerabilities and their severity levels



2.AUDIT OBJECTIVE

- To Identifying if a remote attacker could penetrate Apollo Sampark (B2B Application) website defenses
- Determining the impact of a security breach of confidentiality of the company's private data

Overall, there were total **12 Observations** recorded by security auditors for above Web application, out of which **3 Critical**, **1 High**, **2 Medium**, **6 Low** and **0 Info** observations were found.

2.1 Audit Methodology

Overview

The purpose of Web Application Penetration Testing is to assess and evaluate the security of web application by simulating various attack vectors against the target application and is a **GreyBox assessment**. The web application testing is carried out to find the vulnerabilities in a web based or Client / Server based applications and is aimed to identify vulnerabilities through inadequate coding practices, data input and other handling techniques. The OWASP Top 10 methodology is generally used and is referred to finding the potential vulnerabilities within the web application.

Application Fingerprinting

Information about the target system such as web server, OS and a profile of the application would be gathered in this step. This information would help the assessment in obtaining a snapshot of the target system to work with.

Web Application Scanning

In this stage, the assessment team would scan for threats associated with a service. We mainly concentrate on identifying the application related vulnerabilities at this stage. Some of the vulnerabilities the team would target are the OWASP application layer vulnerabilities such as SQL injections, Cross site scripting (XSS), Logical flaws, application specific threats, unauthorised privilege escalation, unvalidated inputs, weak authentication method and sensitive data in memory.

Proof of concept

The vulnerabilities identified in the previous stage would be exploited in this stage by simulating various kinds of attacks. The application security test would be of both black-box and grey-box in nature.

Remediation Analysis and Reporting

Includes documentation of results and creation of categorised remediation steps.

OWASP Top 10 Security Risks

WEB APPLICATION
SECURITY ASSESSMENT

This is a comparison of the top 10 security risks between the years 2017 and 2021, as per the OWASP.

2017		2021	
A01	Injection	A01	Broken Access Control
A02	Broken Authentication	A02	Cryptographic Failures
A03	Sensitive Data Exposure	A03	Injection
A04	XML External Entities (XML)	A04	Insecure Design [NEW]
A05	Broken Access Control	A05	Security Misconfiguration
A06	Security Misconfiguration	A06	Vulnerable and Outdated Components
A07	Cross-Site Scripting (XSS)	A07	Identification and Authentication Failures
A08	Insecure Deserialization	A08	Software and Data Integrity Failures [NEW]
A09	Using Components with Known Vulnerabilities	A09	Security Logging and Monitoring Failures
A10	Insufficient Logging & Monitoring	A10	Server-Side Request Forgery [NEW]

3.DETAILED FINDINGS

3.1 Payment Logic Bypass

Description:

The application is vulnerable to a critical business logic flaw in its payment processing workflow. It was observed that the payable amount can be manipulated before reaching the payment gateway, allowing users to pay a lower amount than intended. Furthermore, the payment verification endpoint lacks proper validation and can be repeatedly invoked to artificially reduce the outstanding balance. The backend fails to validate the integrity of the transaction amount against the original invoice value. This indicates improper server-side validation and flawed payment reconciliation logic.

Status:

Current Status: New

Overall CVSS Score:

8.8

Risk Rating:

Critical

Impact:

This vulnerability allows attackers to bypass the intended payment logic and settle invoices for significantly lower amounts. By repeatedly invoking the payment verification endpoint, attackers can reduce outstanding balances without making equivalent payments. This can lead to direct financial loss, fraudulent transactions, and abuse of the payment system. The issue severely impacts on the integrity of financial transactions and can be exploited at scale to cause significant business damage.

Affected URL:

https://api.chrijfcq3f-apollotyr1-s2-public.model-t.cc.commerce.ondemand.com/occ/v2/atl-in/B2BUnit/10901_1000_DM_AL/invoice/payments/razorpay/verify-payment?

Affected Parameters:

Application

OWASP Ref No:

Insecure Design

CWE/CVE Reference No:

CWE-602

Recommendation:

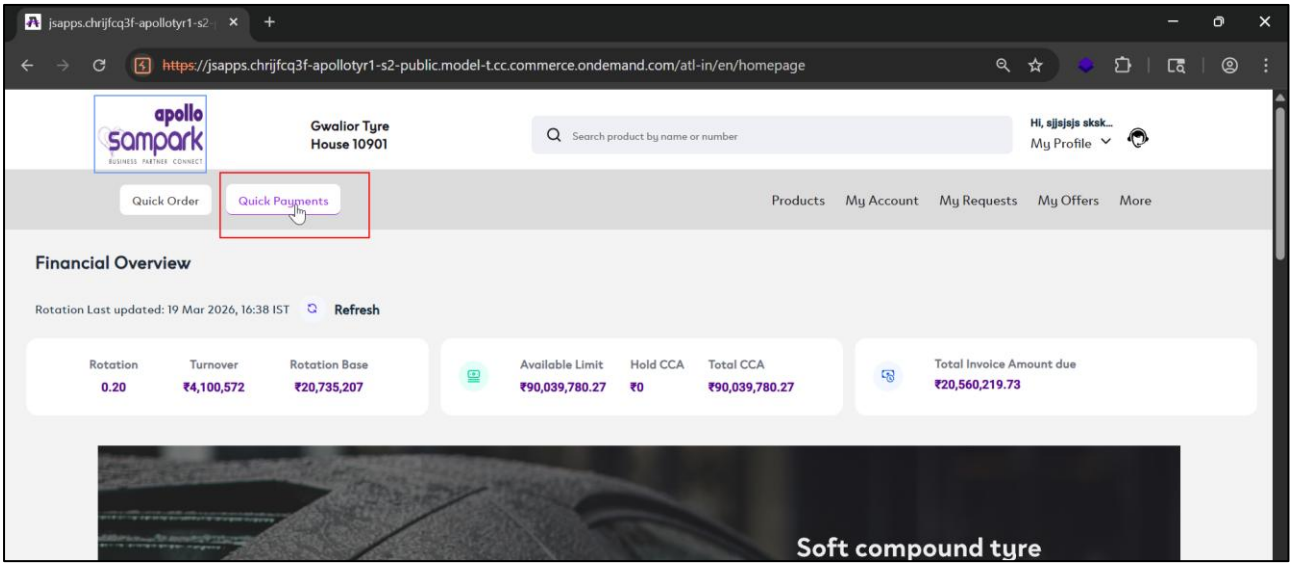
1. Validate payment amounts strictly on the server side against original invoice values before processing.
2. Ensure payment verification endpoints enforce idempotency and cannot be replayed multiple times.
3. Bind each payment transaction to a unique identifier and validate it only once.
4. Cross-verify payment details (amount, invoice ID, transaction ID) with the payment gateway response.
5. Reject any mismatched or tampered payment requests.
6. Implement proper reconciliation logic to ensure outstanding balances are updated only once per valid transaction.
7. Log and monitor abnormal or repeated verification requests for fraud detection.

POC:

Case-Payment Amount Tampering Leading to Underpayment

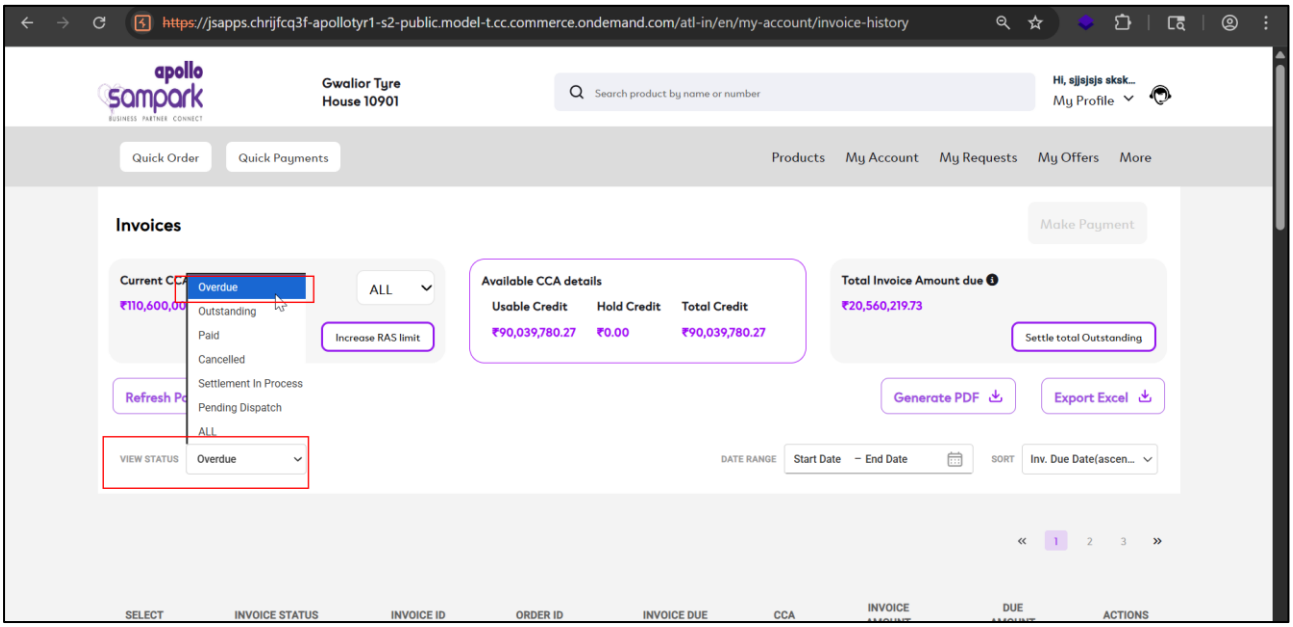
Step 1:

The auditor logs into the application using valid credentials and successfully accesses the dashboard. From the main menu, the auditor navigates to the Quick Payments section to review pending financial transactions.



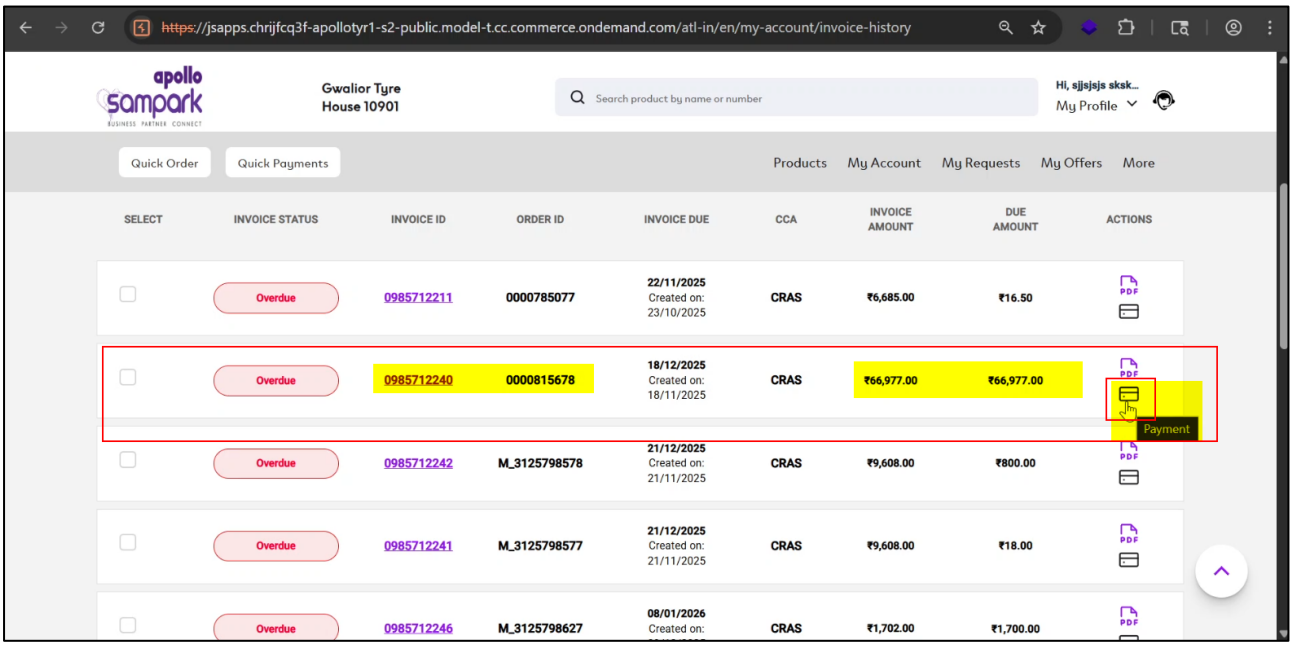
Step 2:

On the Quick Payments page, the auditor interacts with the View Status dropdown and selects the Overdue Invoices option. This action filters and displays only those invoices that are pending payment.



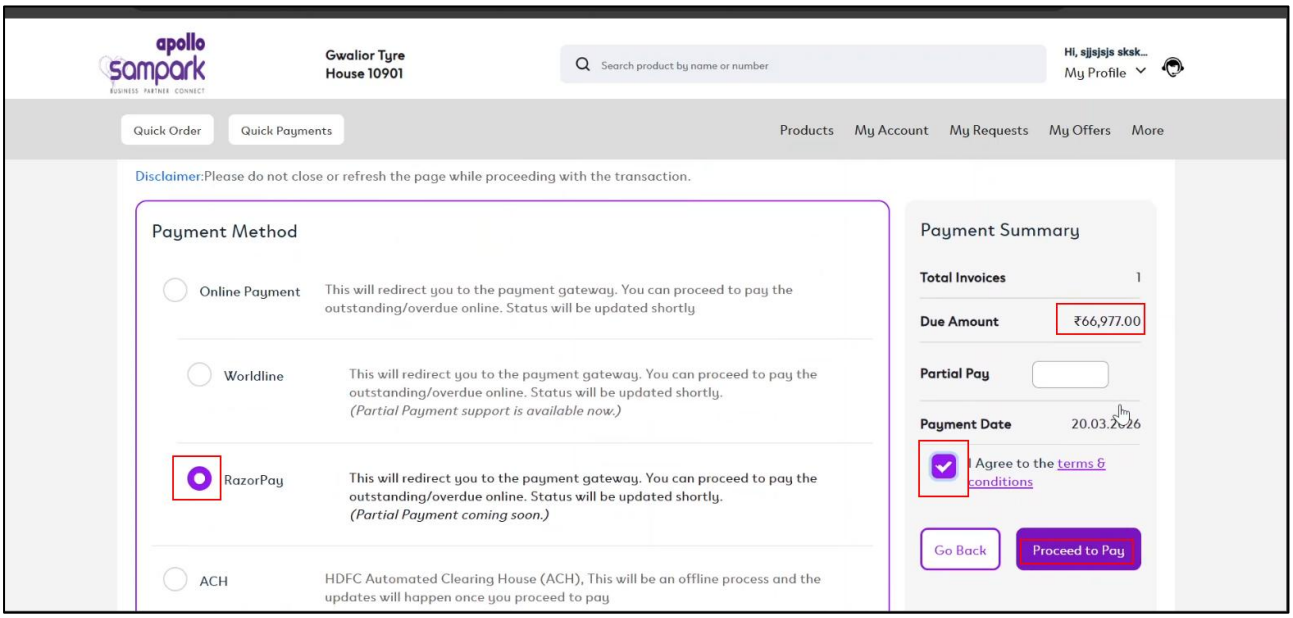
Step 3:

A list of overdue invoices is presented on the screen. The auditor selects a specific invoice (e.g., amount ₹66977) and clicks on the Payment button to initiate the payment process.



Step 4:

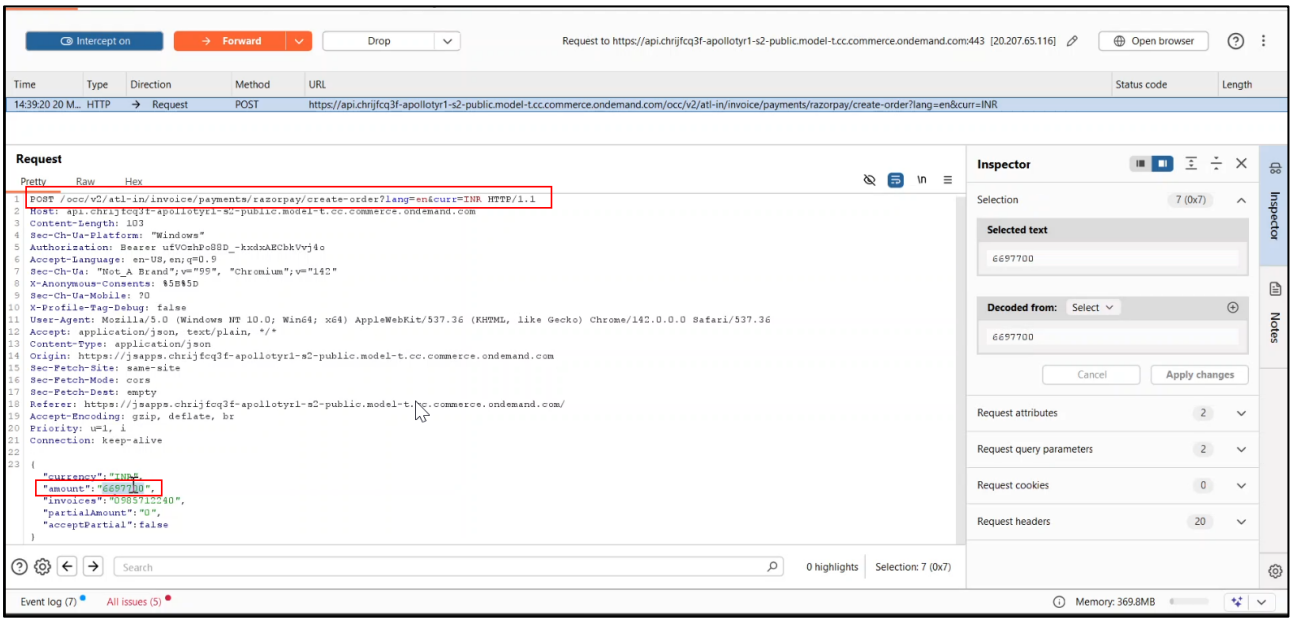
On the make payment page, Auditor selects the razor pay API, Agress to the Terms and Conditions and further clicks on "Proceed to Pay"



Step 5:

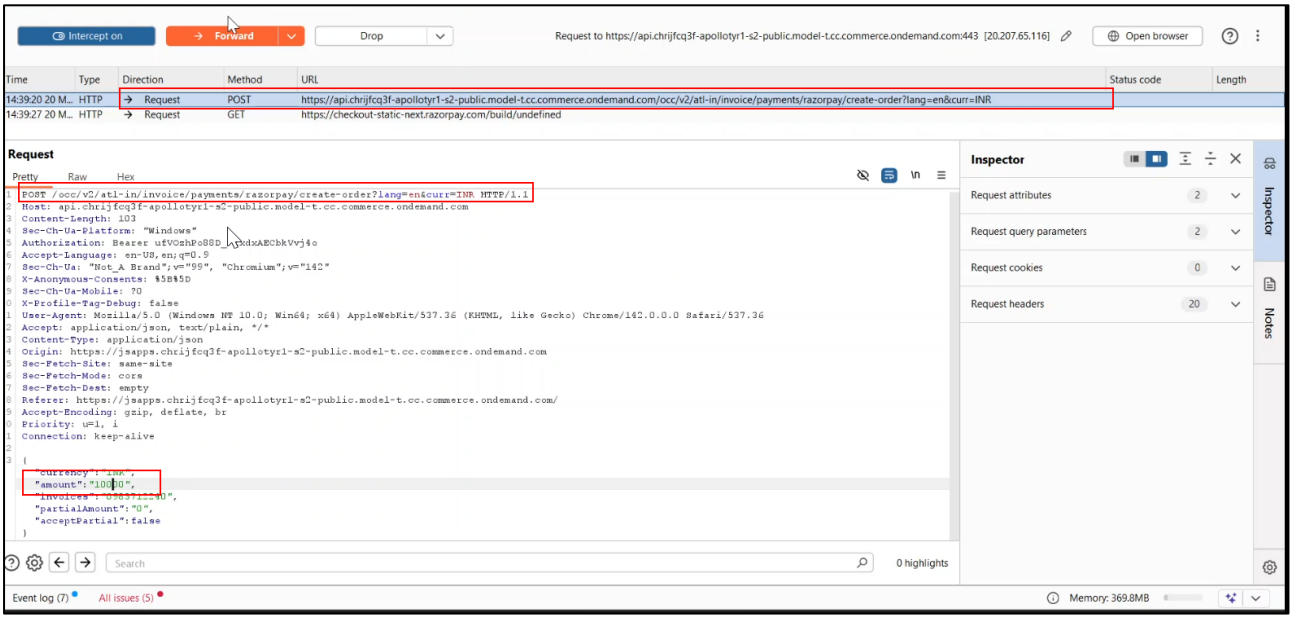
While the payment request is being initiated, the auditor intercepts the outgoing request using a proxy tool (BurpSuite).

Upon analyzing the intercepted request, the auditor identifies the request with /create-order endpoint responsible for the invoice amount. The original value (₹66977.00) is clearly visible in the request payload.



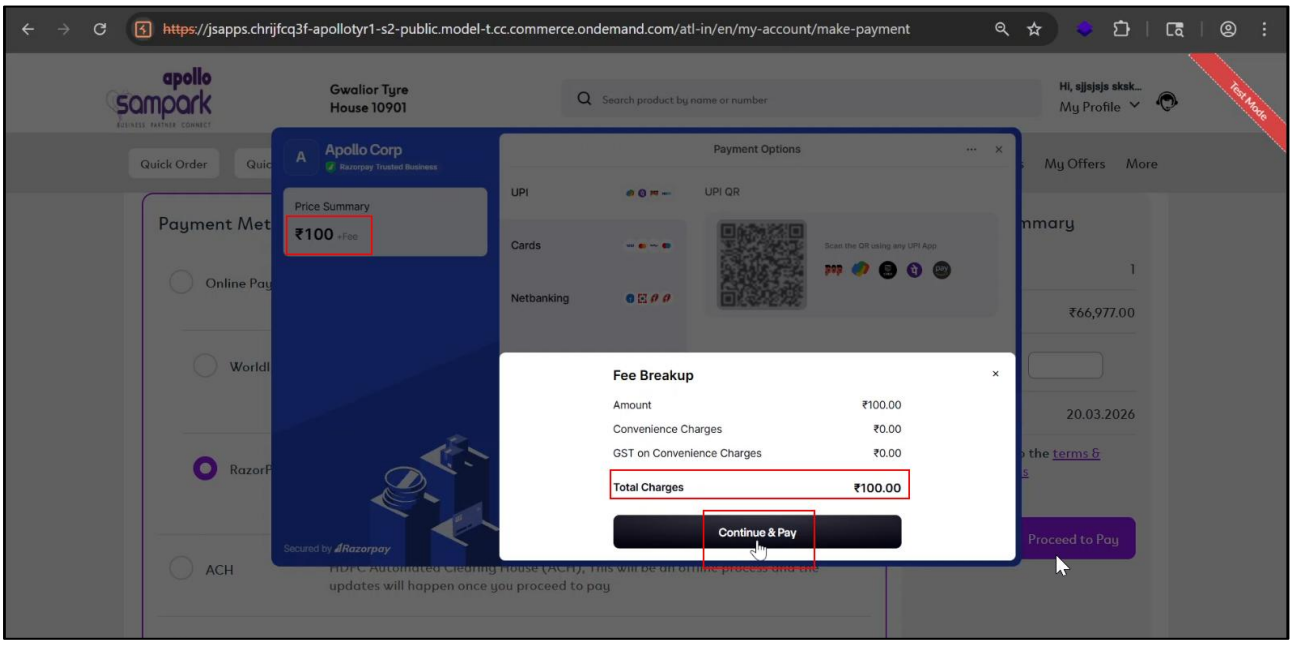
Step 6:

The auditor modifies the payable amount in the request from ₹6697700 to ₹10000 and forwards the manipulated request to the server. No additional validation or restrictions are applied at this stage.



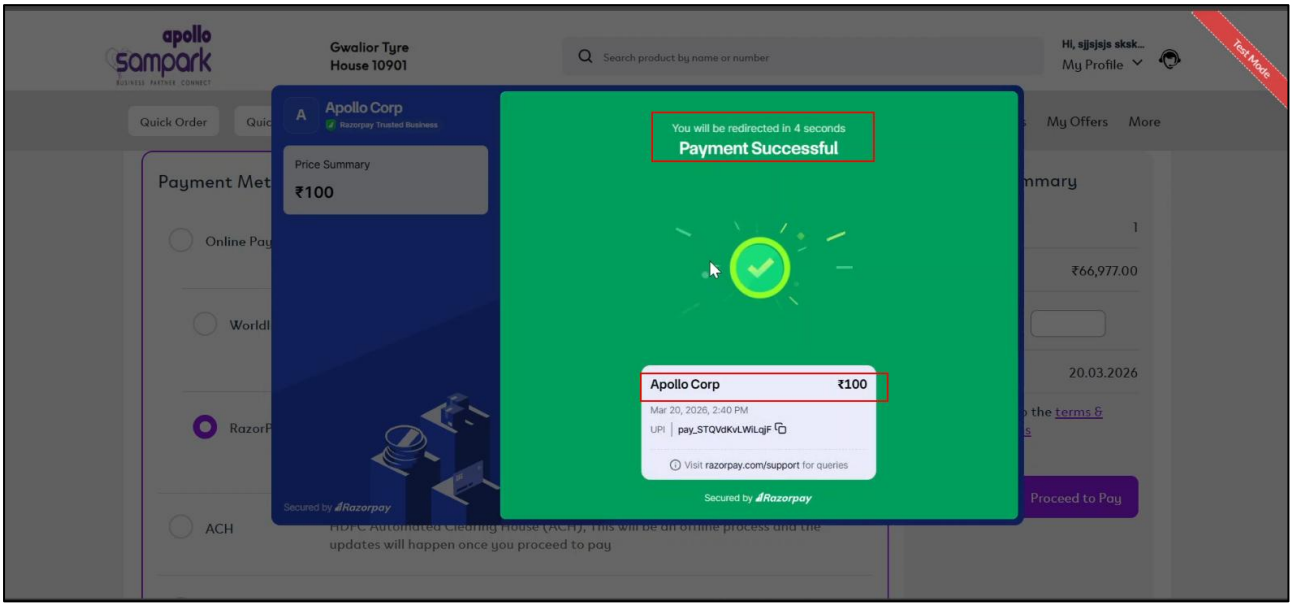
Step 7:

The application redirects the auditor to the payment gateway page. It is observed that the gateway reflects the tampered amount (₹1000) instead of the original invoice value.



Step 8:

The auditor proceeds to complete the payment using the reduced amount. The transaction is processed successfully without any error or discrepancy being flagged.
The application accepts the payment and records it as successful, despite the amount being lower than the actual invoice value. This confirms that no server-side validation is enforced on the payable amount.



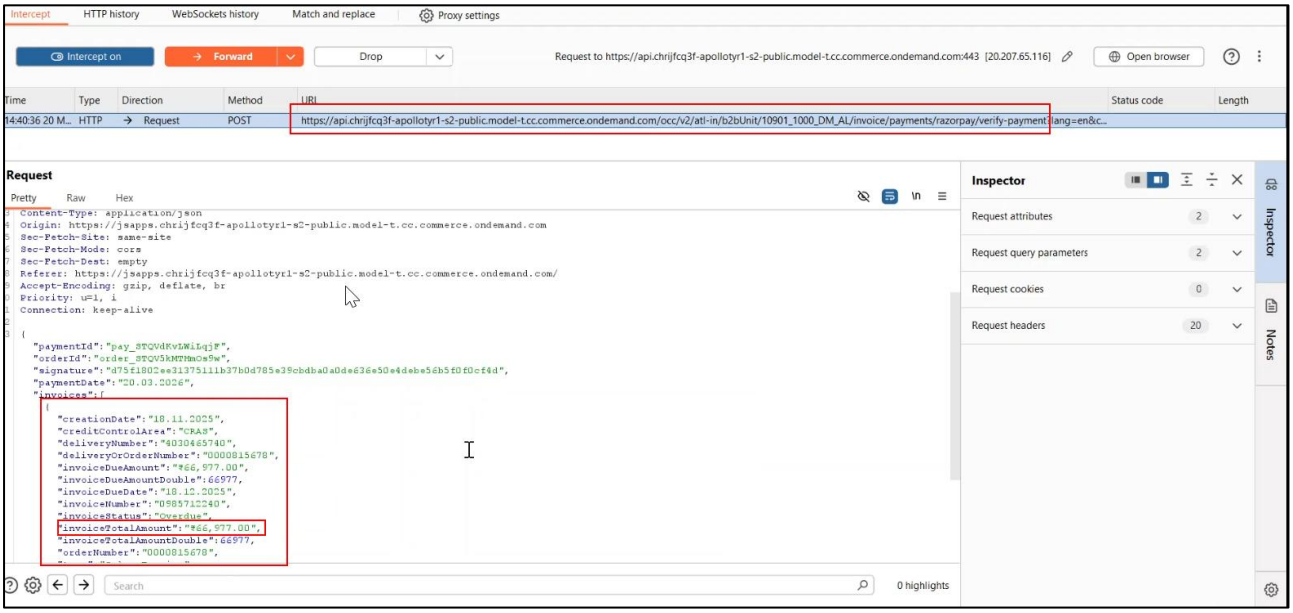
Case- 2 Replay of Payment Verification API Leading to Unauthorized Invoice Reduction

Step 1:

(Refer CASE 1 Before Going with CASE 2 as it's an Chain) The auditor completes a payment of ₹100 for an invoice with a total outstanding amount of ₹66,977 via the payment gateway and is redirected back to the application.

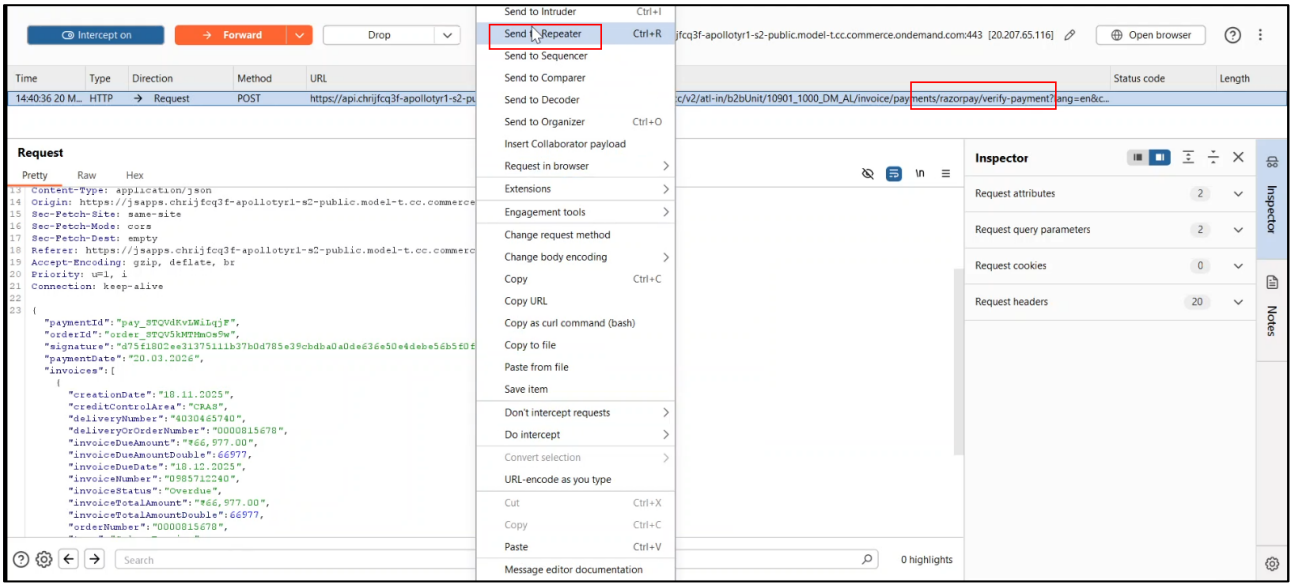
Step 2:

During redirection, the application triggers a request to the /verify-payment endpoint, which is responsible for validating the transaction and updating the outstanding balance.



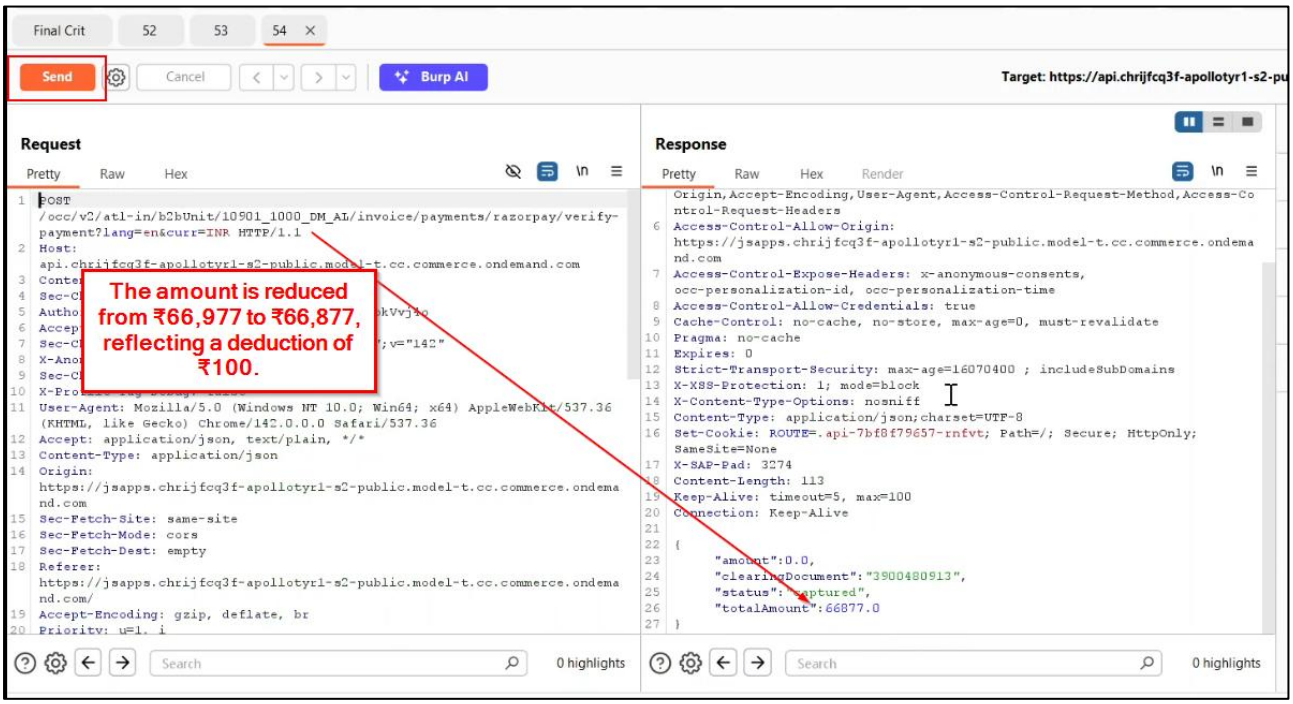
Step 3:

The auditor intercepts the /verify-payment request using a proxy tool and sends the captured request to Repeater for further testing.



Step 4:

Upon sending the request once, the outstanding amount is reduced from ₹66,977 to ₹66,877, reflecting a deduction of ₹100.

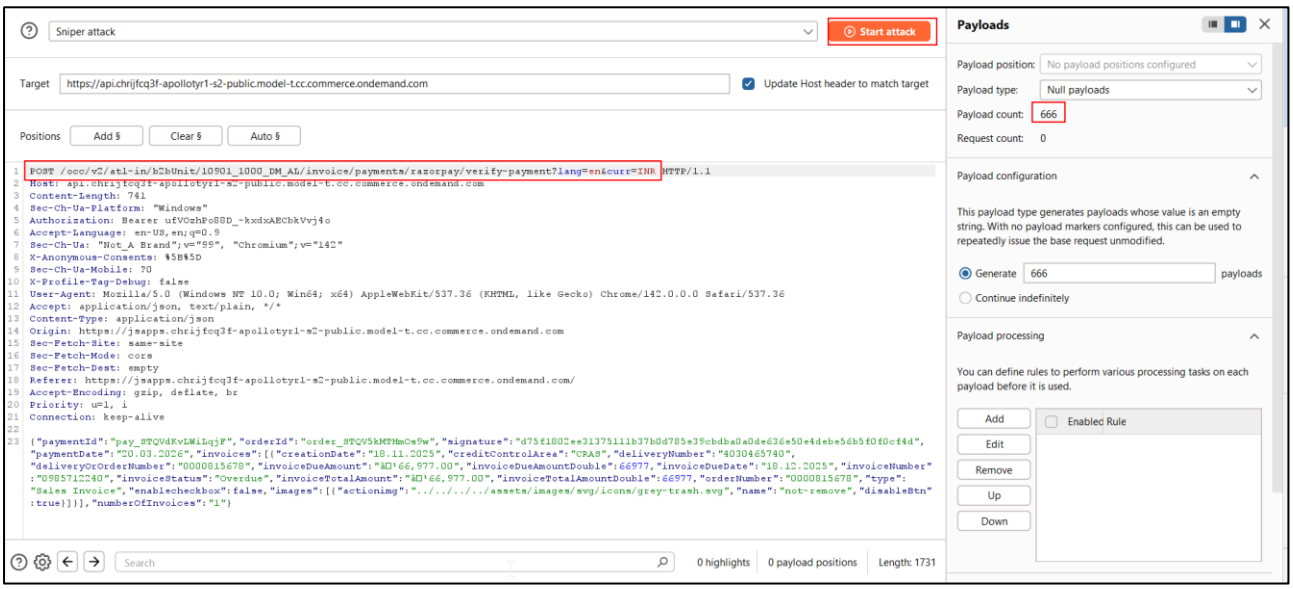


Step 5:

The auditor submits the same request without performing any additional payment, and each request reduces the outstanding amount further (e.g., ₹66,877 → ₹66,777 → ₹66,677).

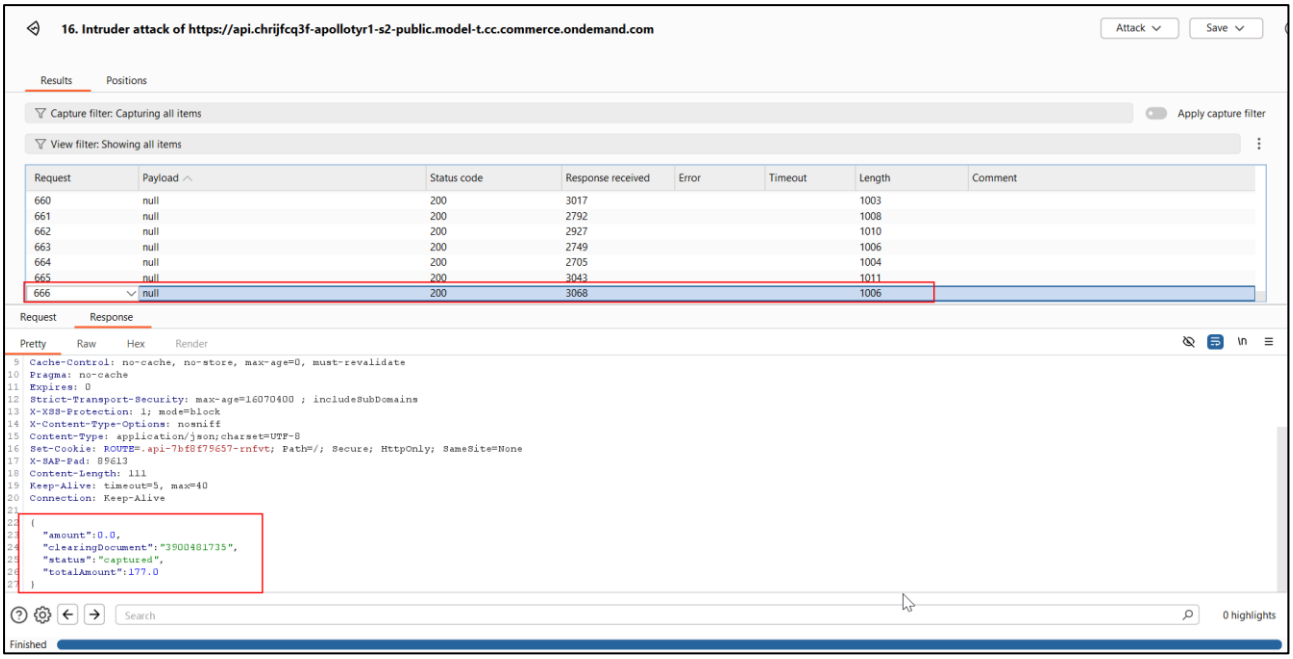
Step 6:

The auditor automates the attack by sending the request to Intruder with null payloads and configuring 666 iterations repeatedly triggering the same verification request.



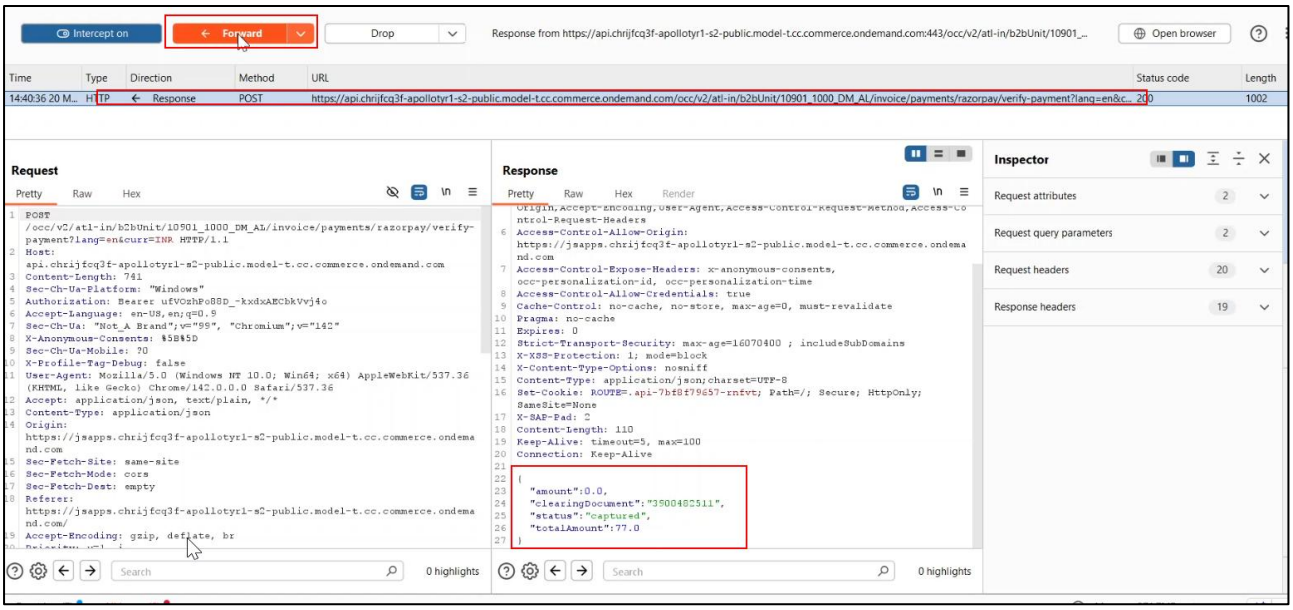
Step 7:

After the attack completes, the outstanding amount is reduced from ₹66,977 to ₹177 despite only a single payment of ₹100 being made.



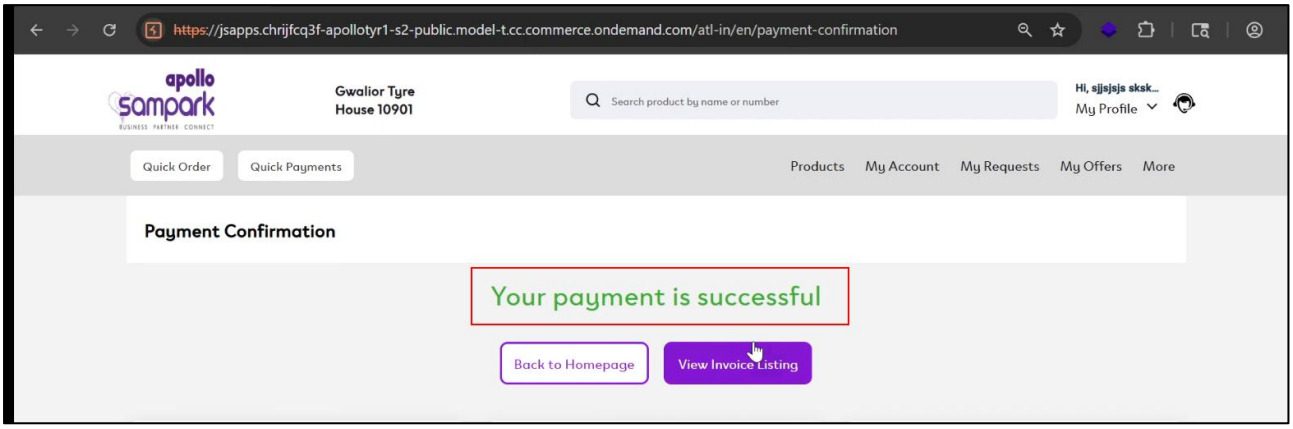
Step 8:

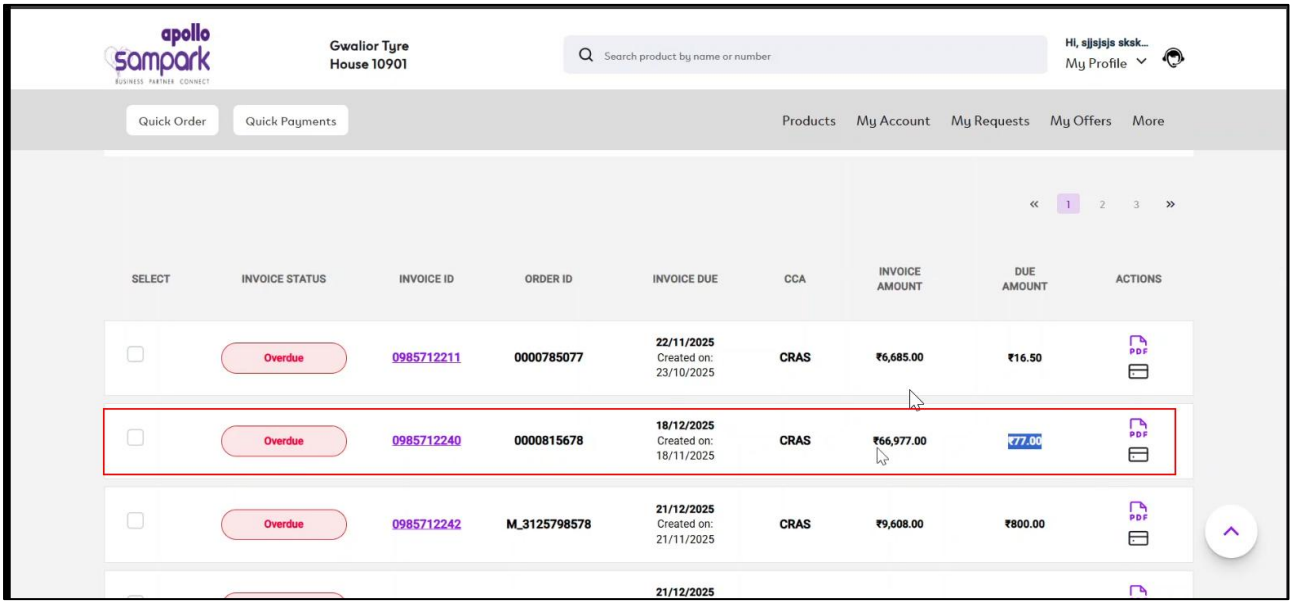
The auditor replays the request one final time, reducing the outstanding amount further from ₹177 to ₹77, confirming that repeated invocation of the verify endpoint results in unauthorized deduction of invoice amounts.



Step 9:

The auditor forwards the final /verify-payment request and receives a successful payment response, and upon revisiting the Overdue Invoices section, observes that the outstanding amount has been reduced to ₹77 from ₹66,677 despite only a payment of ₹100 being made, confirming a critical flaw in the payment verification logic.





3.2 Insecure Direct Object Reference (IDOR)

Description:

The application was found to be vulnerable to Insecure Direct Object Reference (IDOR), allowing unauthorized access to business unit information and functionality. The backend APIs relies on user-supplied identifiers (e.g., business unit IDs) without enforcing proper authorization checks. By manipulating these identifiers, the auditor was able to access data belonging to other business units not assigned to the user. This indicates a lack of server-side access control validation, leading to unauthorized data exposure and access.

Status:

Current Status: New

Overall CVSS Score:

8.8

Risk Rating:

Critical

Impact:

An attacker can enumerate and access sensitive information of other business units by modifying identifiers in requests. This may lead to exposure of confidential company data and unauthorized access to restricted organizational contexts. Additionally, attackers can switch into unauthorized business units and perform actions within them. Overall, this results in a severe breach of confidentiality and access control boundaries.

Affected URL:

https://jsapps.chrijfcq3f-apollotyr1-s2-public.model-t.cc.commerce.ondemand.com/atl-in/en/my-company

Affected Parameters:

Application

OWASP Ref No:

https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

CWE/CVE Reference No:

CWE-639

Recommendation:

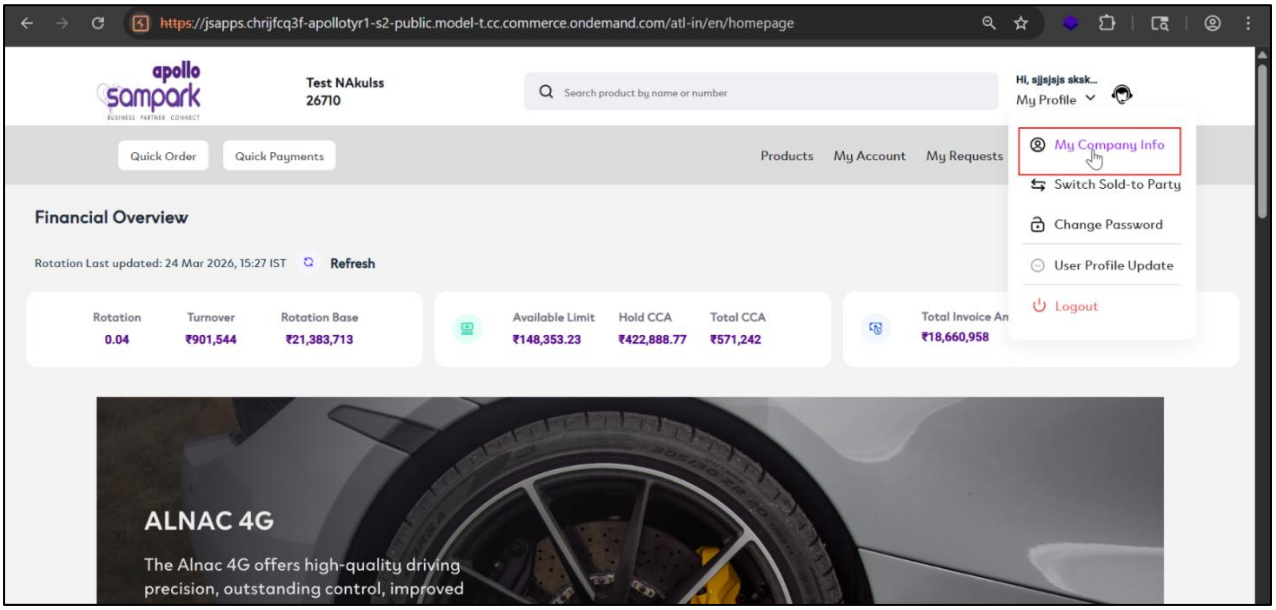
- Enforce strict server-side authorization checks for all object-level access requests.
- Validate that the requested resource (e.g., business unit ID) belongs to the authenticated user.
- Avoid relying on predictable or sequential identifiers; use indirect references or UUIDs where possible.
- Implement logging and monitoring to detect abnormal access patterns or ID enumeration attempts.

POC:

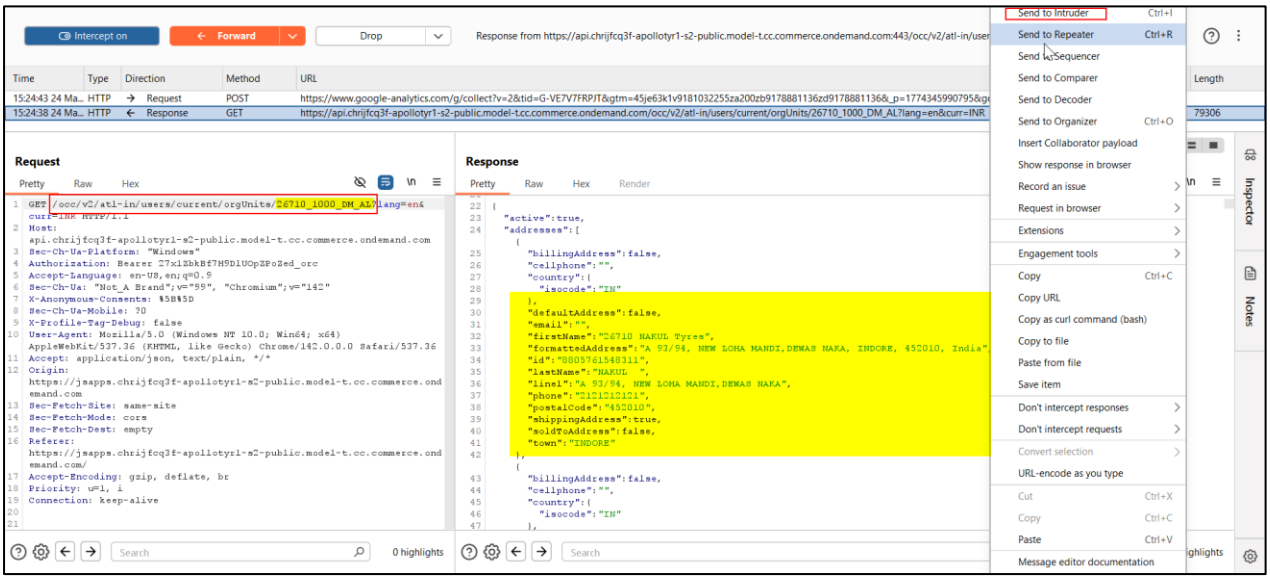
Case-1 IDOR Leading to Information Disclosure

Step 1:

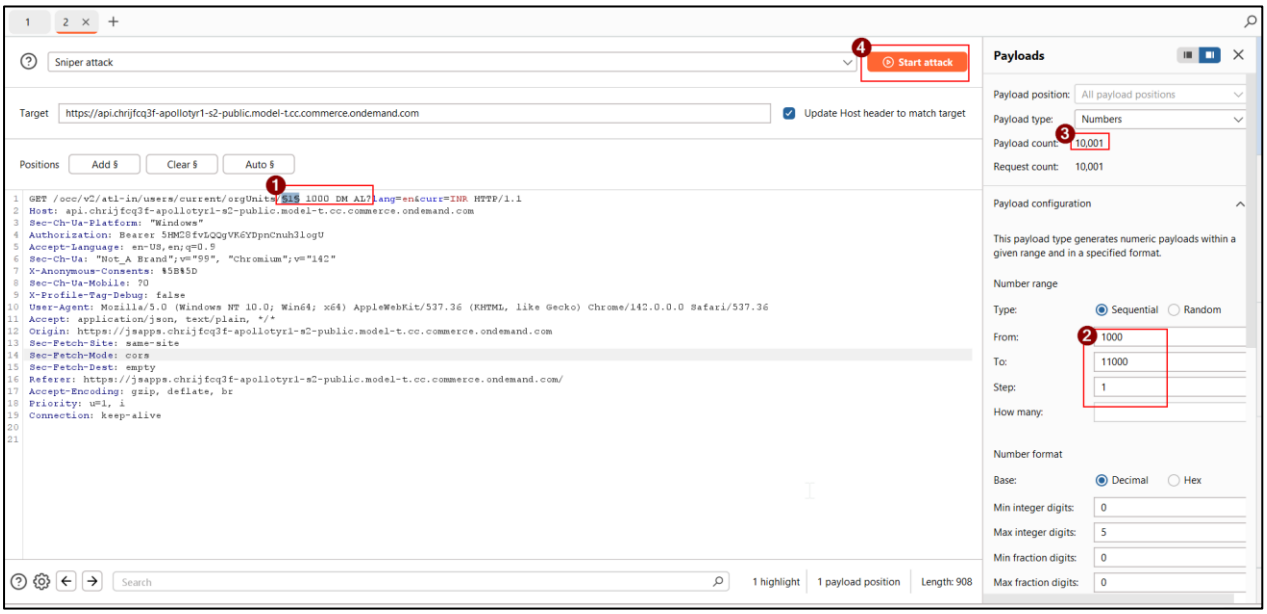
The auditor logged into the application and navigated to the My Company Info page, where static information about the assigned business unit was displayed as shown in the artifact below:



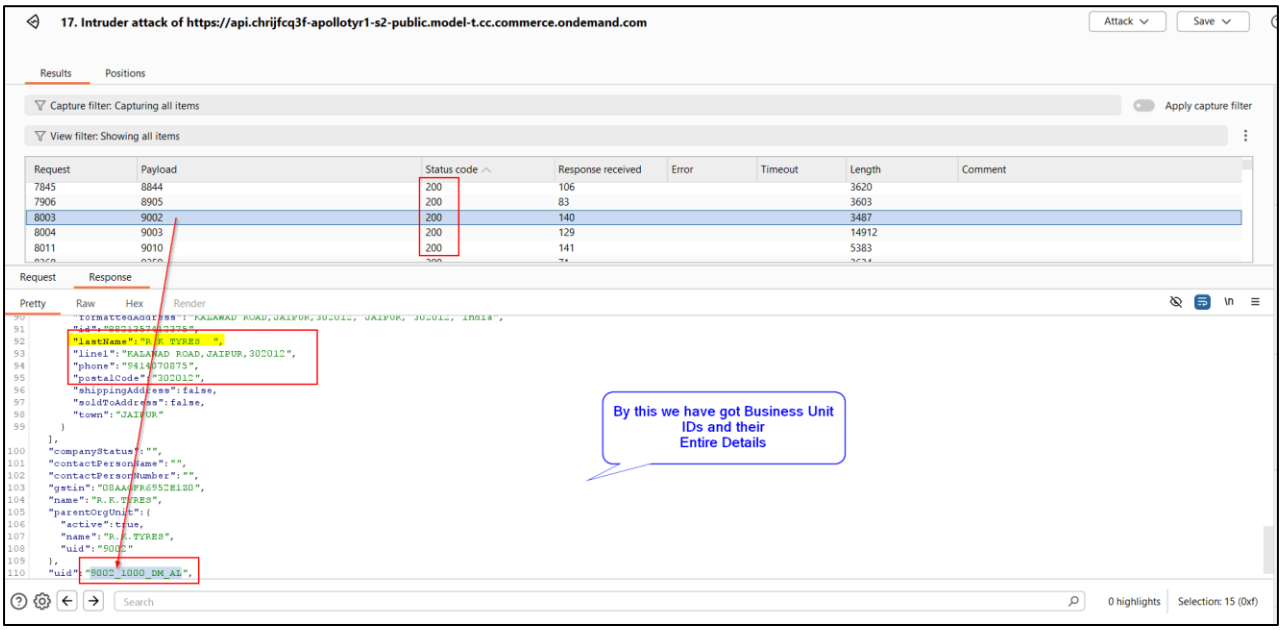
Step 2: The auditor captured the request in Burp Suite proxy, sent it to Repeater, and observed in the response that complete company details were returned for the provided business unit ID as shown in the artifact below:



Step 3: The auditor sent the same request to Intruder and configured a brute-force attack on the business unit ID parameter with payload values ranging from 1000 to 10000 as shown in the artifact below:



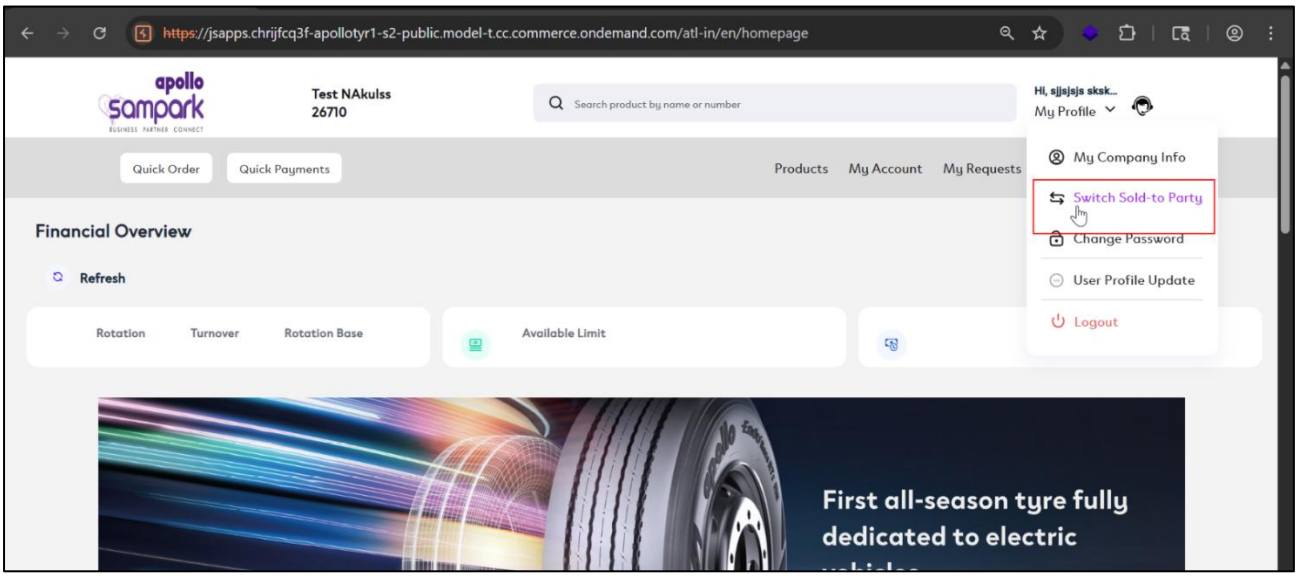
Step 4: The auditor initiated the attack and analyzed the responses, observing multiple HTTP 200 responses for different business unit IDs as shown in the artifact below:



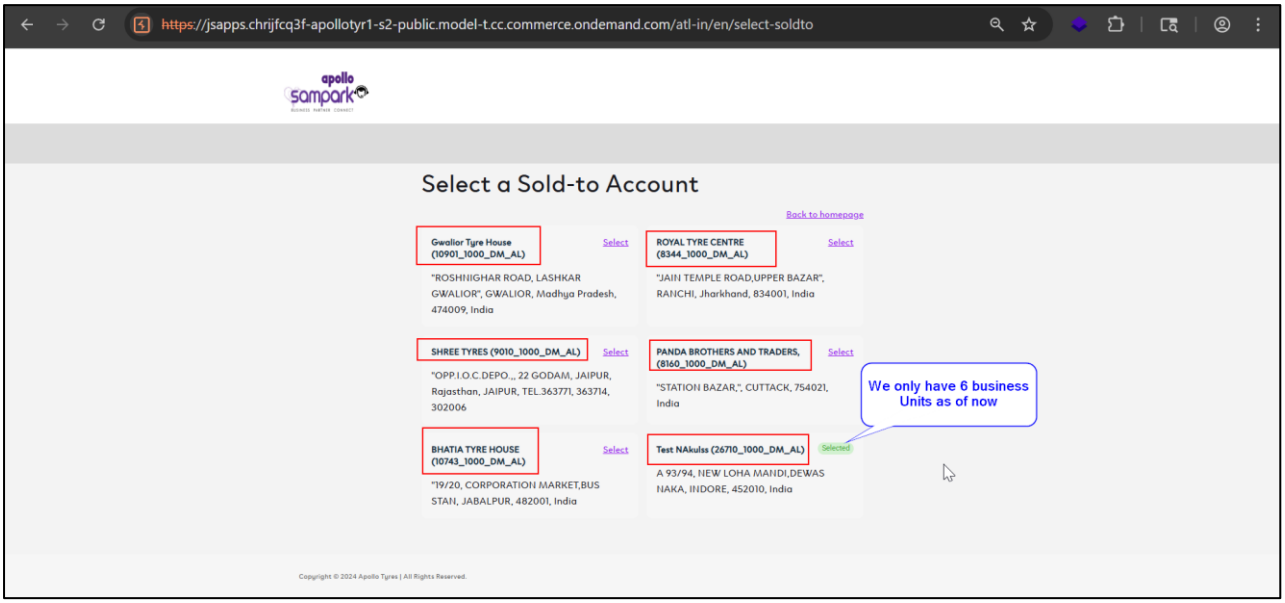
Step 5: The auditor confirmed that details of multiple business units were accessible, indicating information disclosure via IDOR.

Case 2: IDOR Leading to Unauthorized Business Unit Access

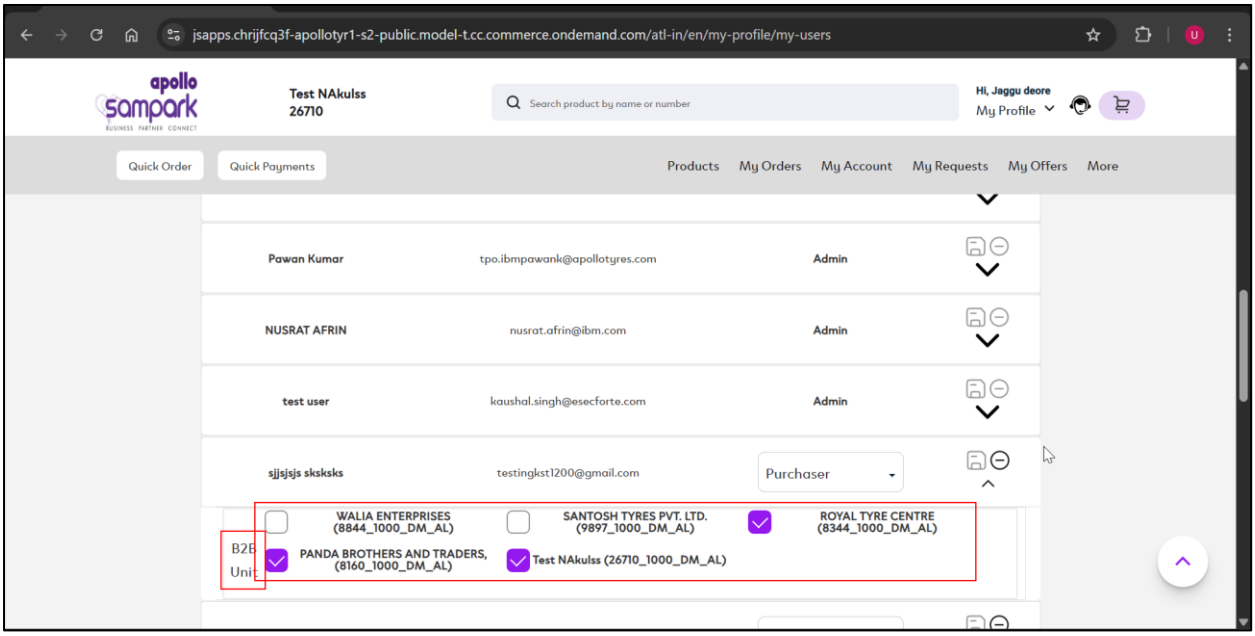
Step 1: The auditor logged into the application and accessed the **Switch Sold-To Party** functionality as shown in the artifact below:



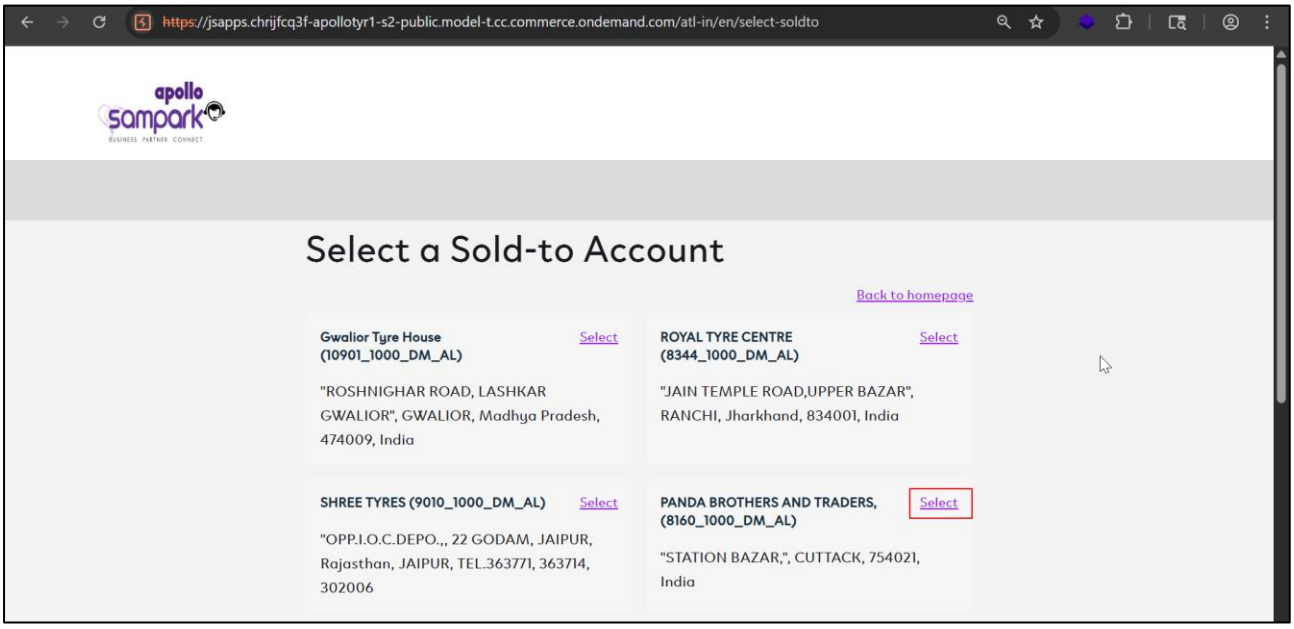
Step 2: The auditor observed that only a limited number of business units were assigned and visible to the user as shown in the artifact below:



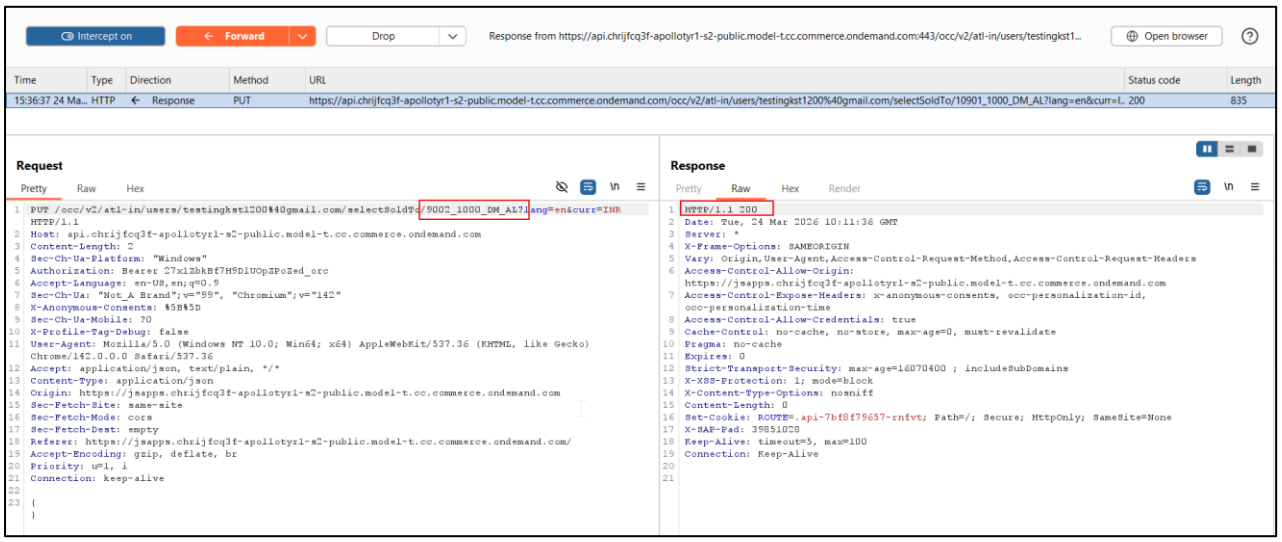
Step 2.2: The auditor additionally verified by logging in through an admin account that only specific business units were intended to be assigned to this user; however, in subsequent steps, it was observed that additional unauthorized business units could still be added as shown in the artifact below:

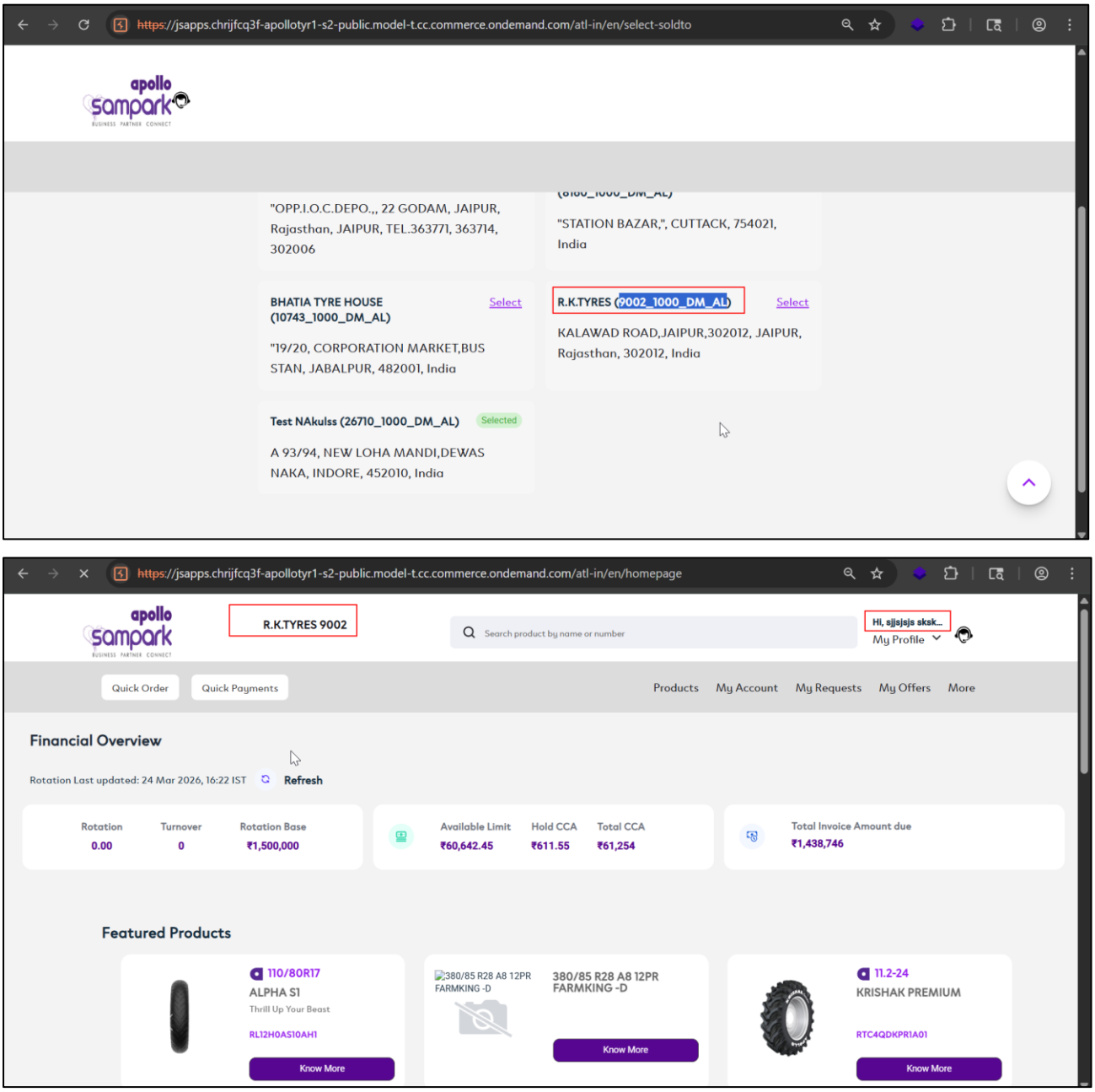


Step 3: The auditor selected any available business unit, captured the request in Burp Suite, and identified the business unit ID parameter in the request as shown in the artifact below:



Step 4: The auditor modified the business unit ID to a value obtained from Case 1 and forwarded the request, observing a successful (200 OK) response as shown in the artifact below:





Step 5: The auditor observed that the unauthorized business unit was added to the switch list and was accessible upon selection, confirming privilege escalation via IDOR.

3.3 Privilege Escalation

Description:

The application includes role-related parameters within the user invitation URL, which are only Base64 encoded and not securely protected. It was observed that these parameters can be decoded, modified, and re-encoded by an attacker to alter the assigned role during the registration process. Since the application trusts client-supplied data without proper server-side validation, it allows manipulation of privilege levels. This indicates a critical flaw in authorization logic and improper handling of sensitive parameters. Such design exposes the system to unauthorized privilege escalation through simple parameter tampering.

Status:

Current Status: New

Overall CVSS Score:

8.8

Risk Rating:

Critical

Impact:

This vulnerability allows an attacker to create accounts with elevated privileges, such as administrative access, without proper authorization. Exploitation can lead to full compromise of the application, including unauthorized access to sensitive data, user management, and critical functionalities. Attackers may perform malicious actions such as modifying or deleting data, creating additional privileged accounts, or disrupting business operations. This significantly impacts the confidentiality, integrity, and availability of the system. The issue poses a high risk to overall application security and trust.

Affected URL:

https://jsapps.chrijfcq3f-apollotyr1-s2-public.model-t.cc.commerce.ondemand.com/atl-in/en/my-profile/invite-users

Affected Parameters:

Application

OWASP Ref No:

OTG-AUTHZ-003

CWE/CVE Reference No:

CWE-269

Recommendation:

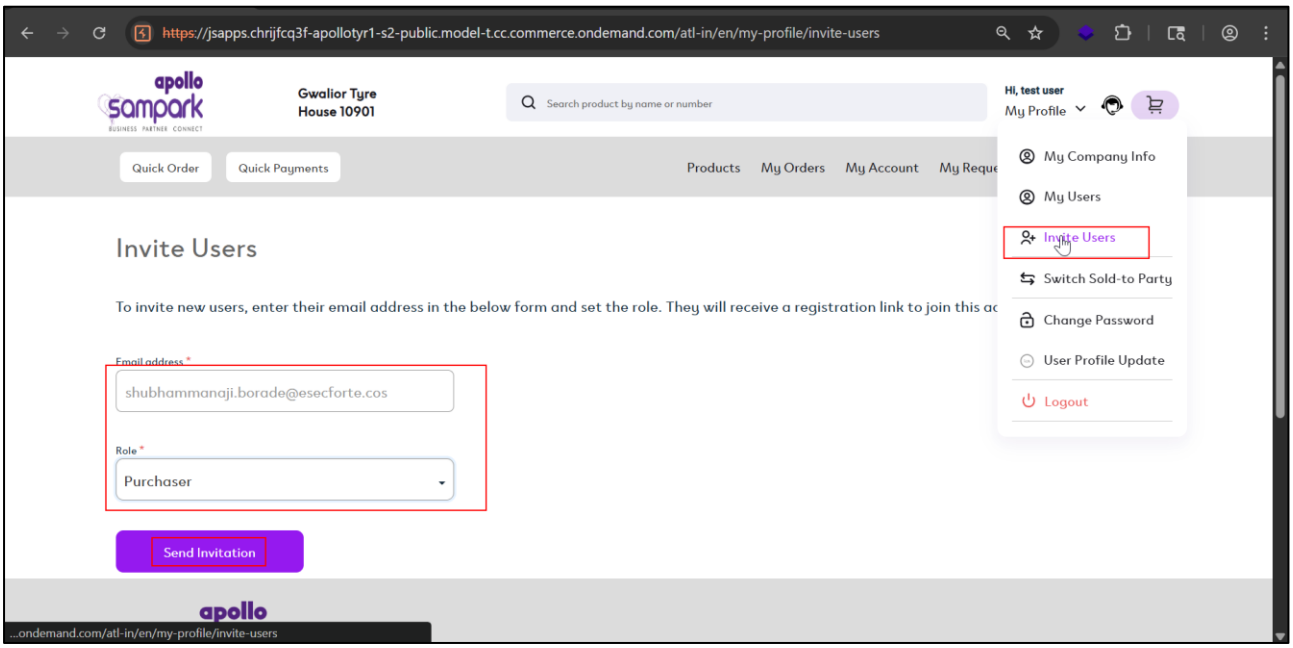
1. Do not include sensitive parameters such as roles in client-controlled URLs; enforce role assignment strictly on the server side
2. Use securely signed tokens (e.g., JWT with signature validation) or encrypted parameters to prevent tampering
3. Bind invitation links to server-side records (e.g., store email and role mapping in backend and validate during registration)
4. Implement strict server-side validation to ensure roles cannot be modified by user input
5. Use one-time, time-bound invitation tokens to prevent reuse and manipulation
6. Log and monitor suspicious invitation and registration activities for early detection

POC:

Case-Privilege Escalation on Invite Users Functionality

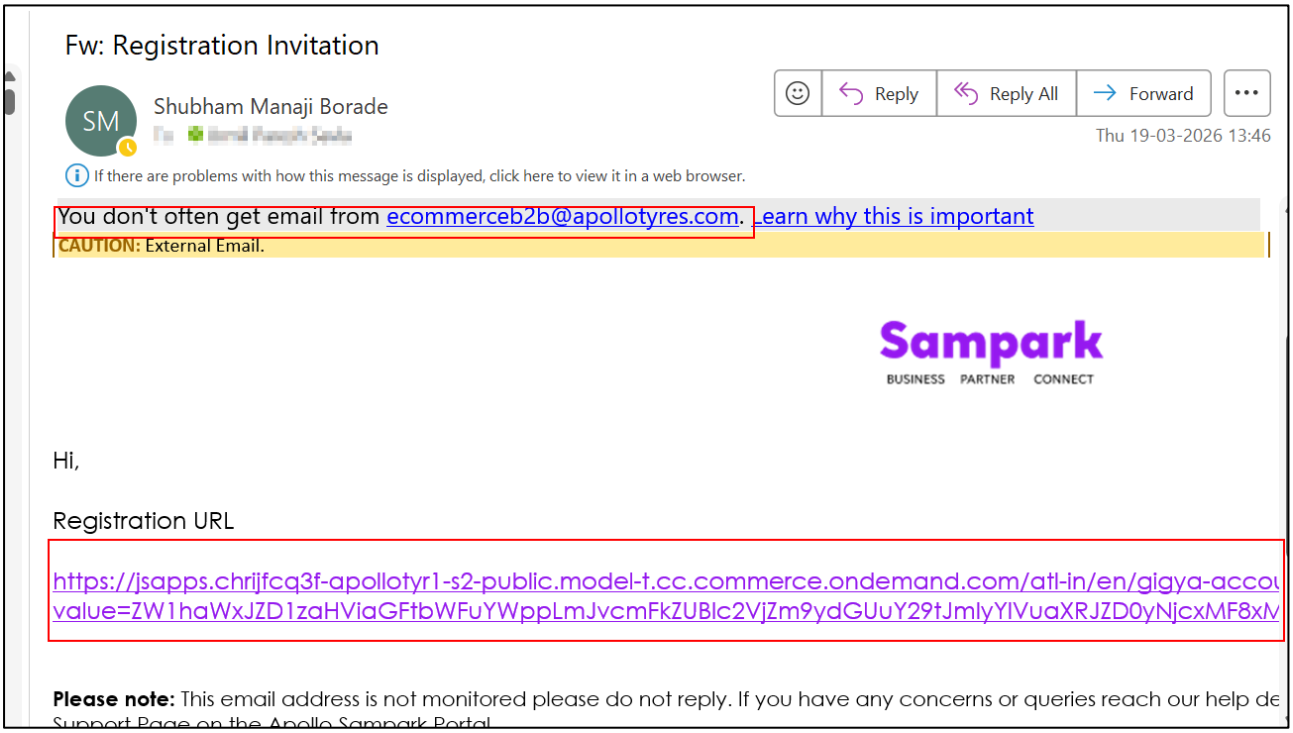
Step 1:

During the assessment, the auditor logs into the application, navigates to *My Profile* → *Invite Users*, enters an email address, selects a lower-privileged role, and sends the invitation as shown in the artifact below:



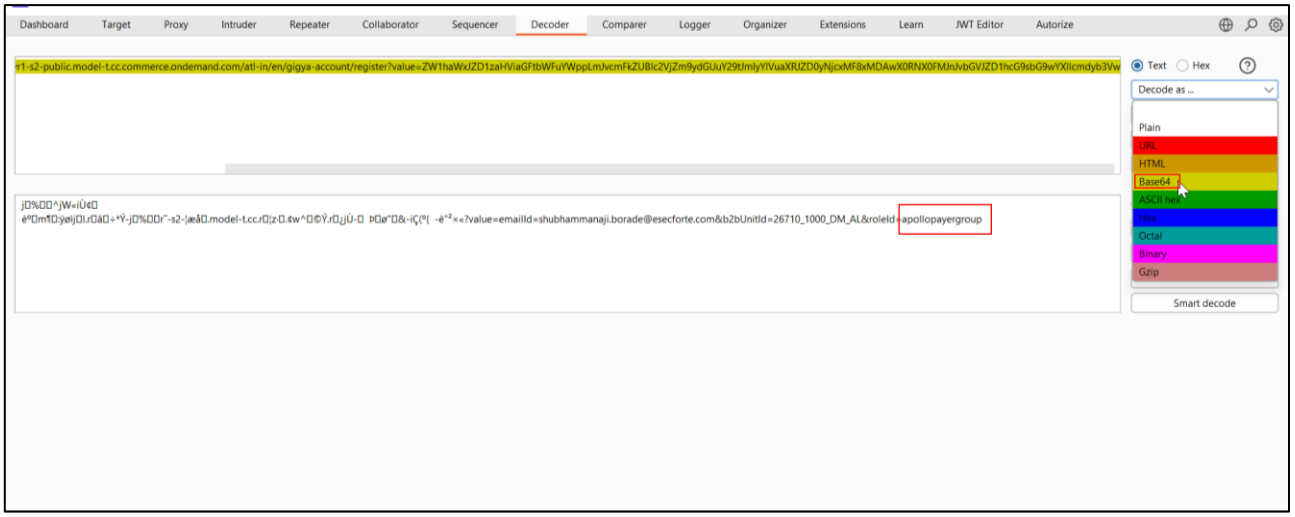
Step 2:

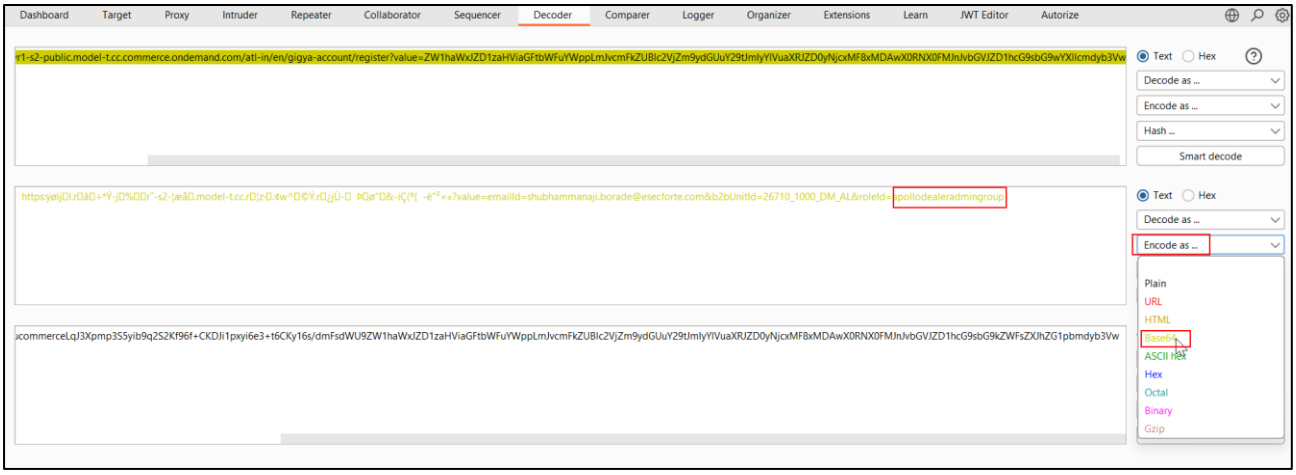
The auditor observes that the invited user receives an email containing the registration URL, as shown in the artifact below:



Step 3:

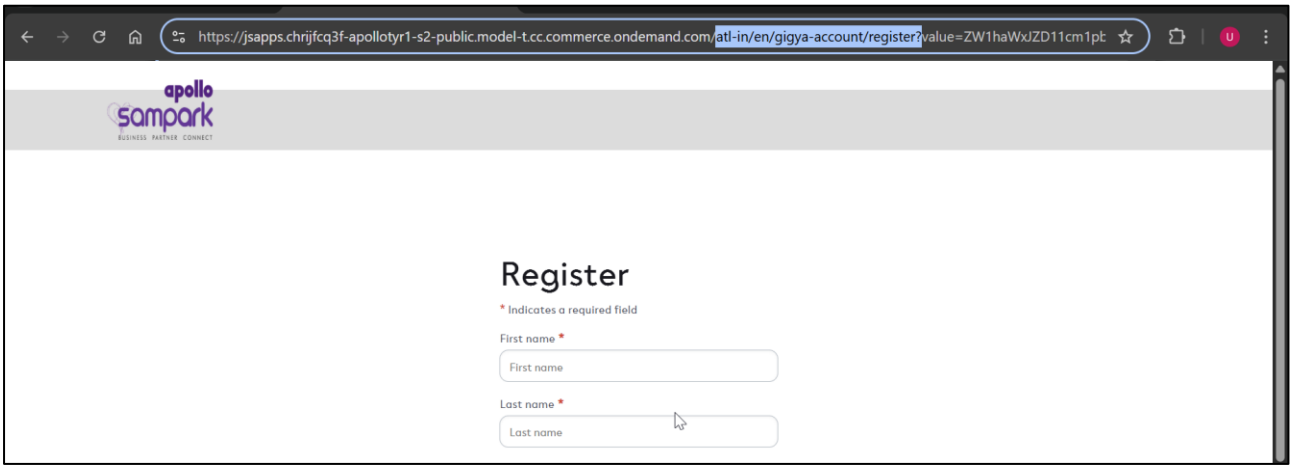
The auditor uses *Burp Suite Decoder* to decode the Base64-encoded registration URL, modifies the role parameter from a lower-privileged role (e.g., *peer group*) to a higher-privileged role (e.g., *dealer admin*), and re-encodes the value, as shown in the artifact below:





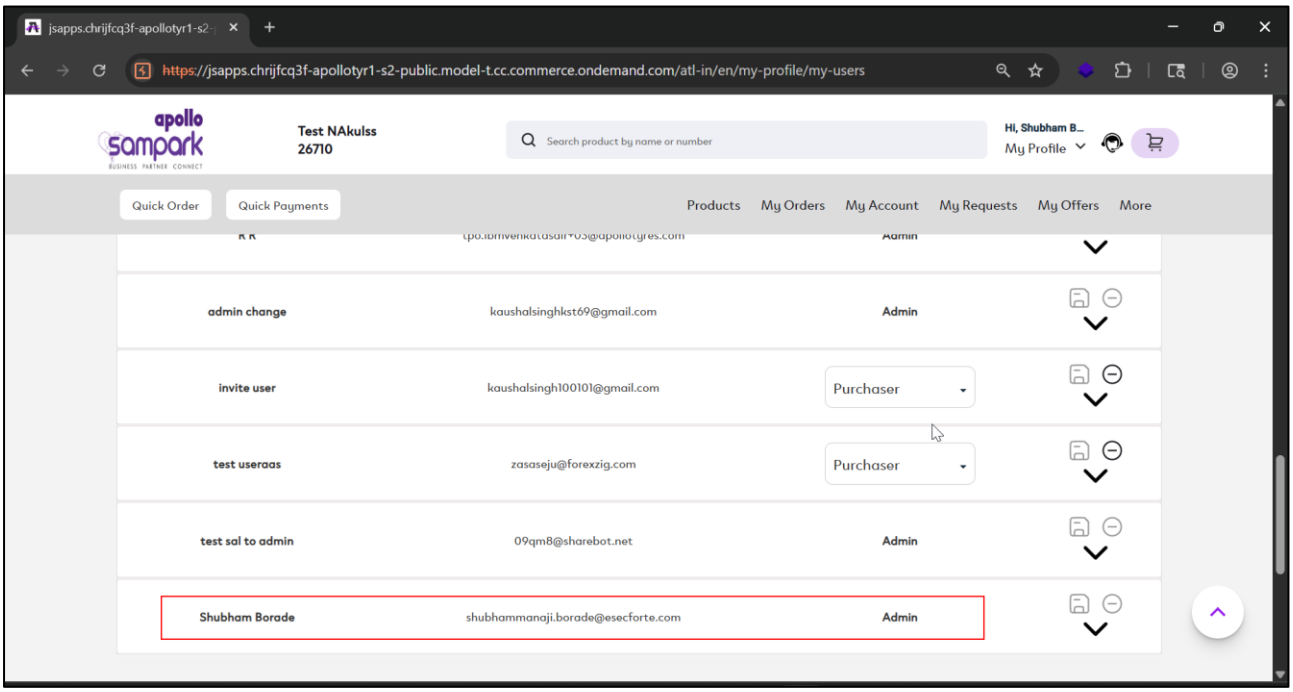
Step 4:

The auditor pastes the modified URL into the browser, completes the registration form, and successfully creates the account, as shown in the artifact below:



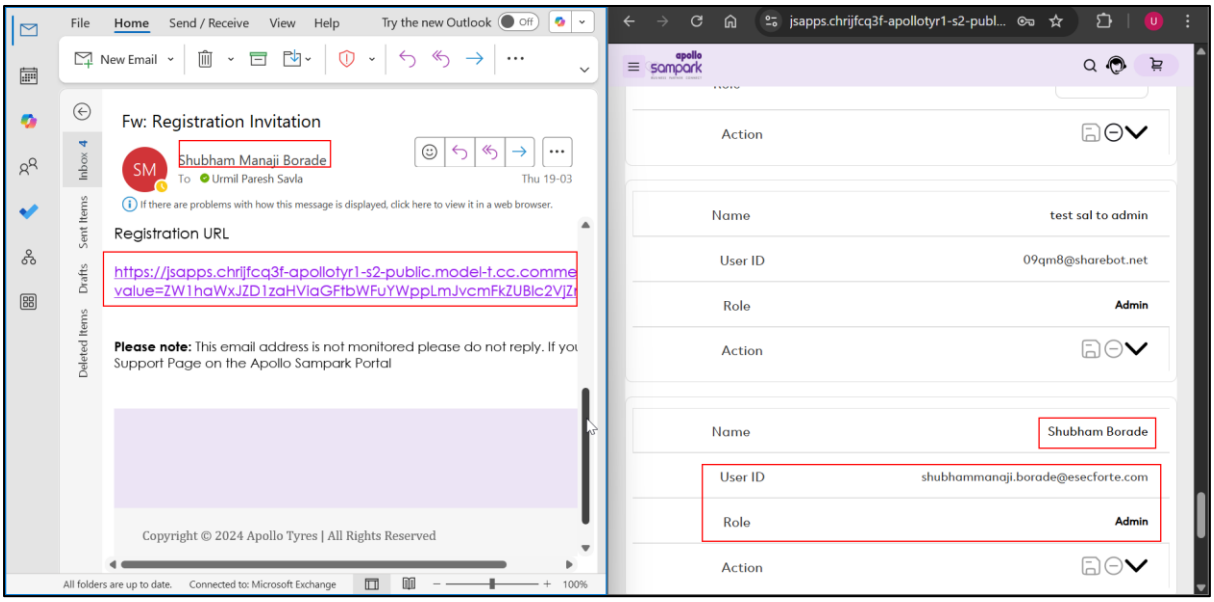
Step 5:

The auditor (admin) navigates too the *My Users* section and observes that the invited user now has elevated (admin) privileges instead of the originally assigned role, as shown in the artifact below:

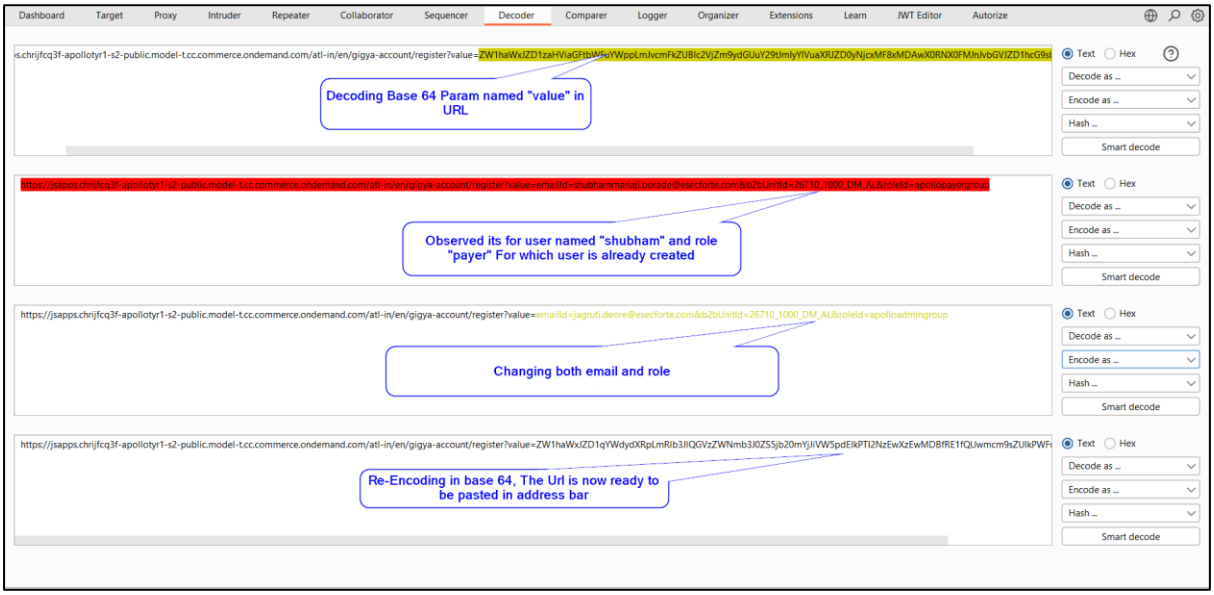


Case-2

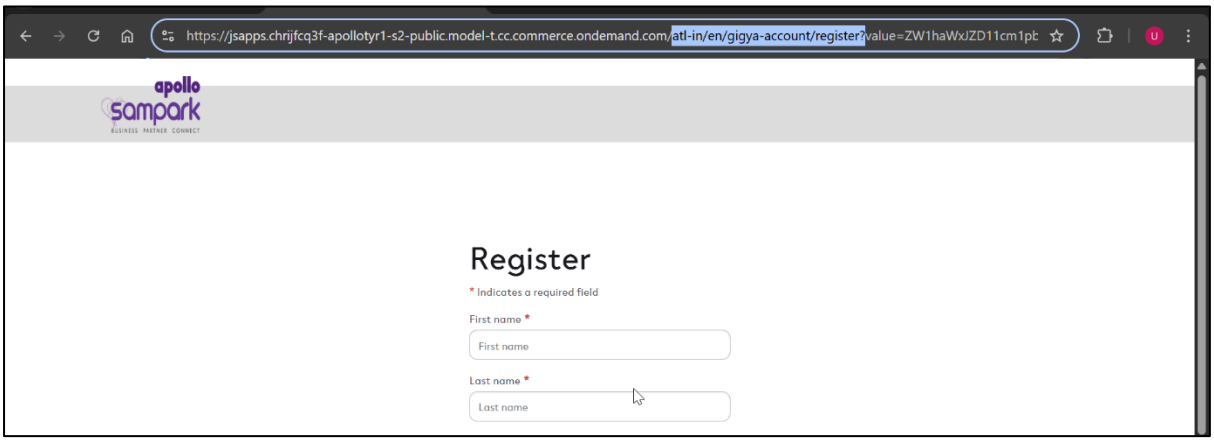
Step 1: The auditor reviewed the user invitation workflow and observed that an invite link was generated and shared for a user (e.g., ABC) with a predefined role and confirmed that an account was already created using this link.



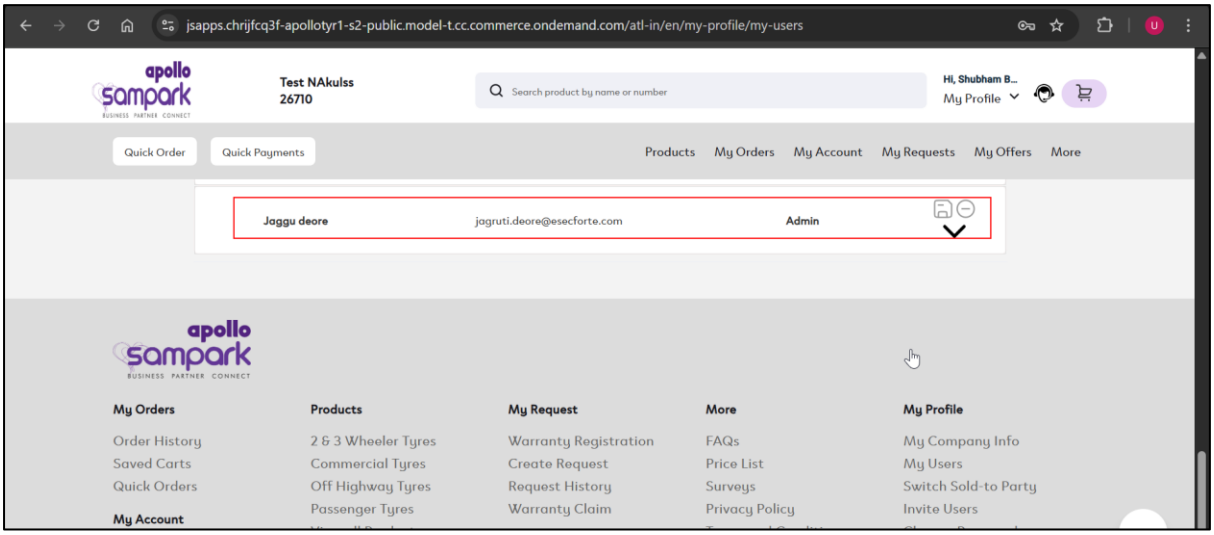
Step 2: The auditor copied the invitation link and decoded the Base64-encoded parameter using a decoder, where the decoded value revealed sensitive details such as the email ID and role ID; the auditor then modified both the email and role values and re-encoded them back into Base64 to craft a manipulated invitation link.



Step 3: The auditor pasted the modified invitation link into the browser and accessed the registration screen using the tampered URL. The registration screen reopenes



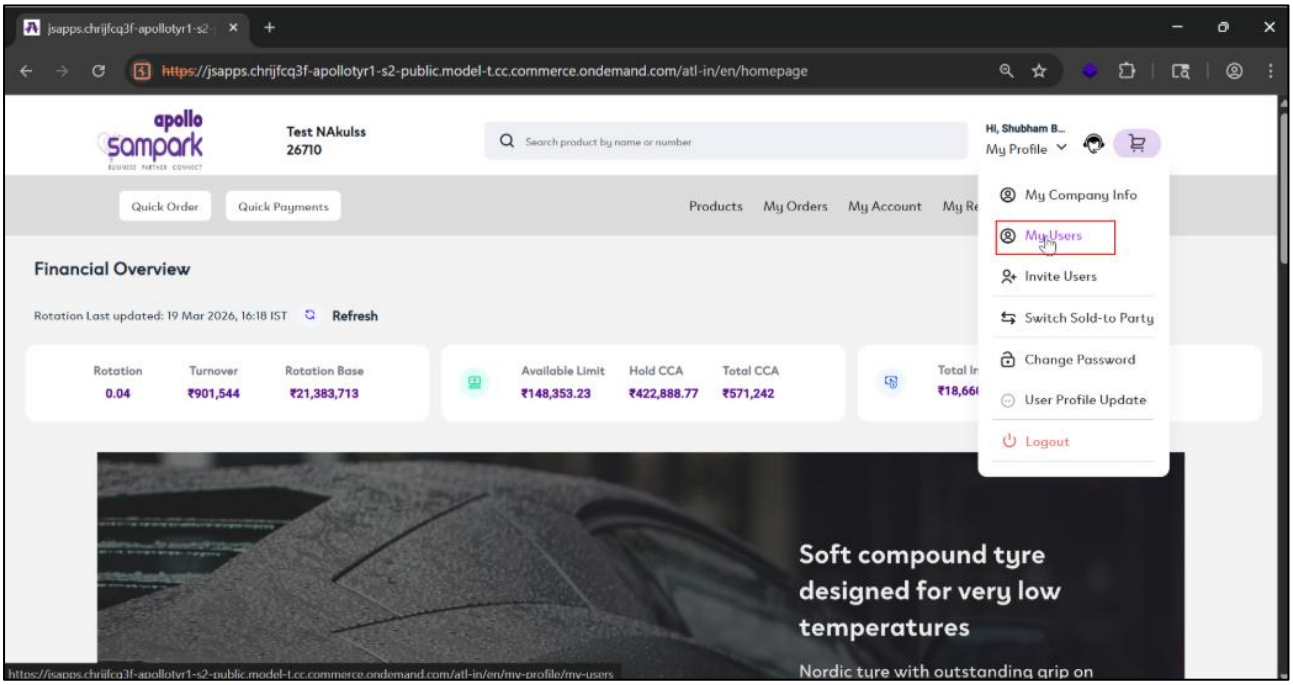
Step 4: The auditor observed that a new user account was successfully created with the modified details (e.g., elevated/admin role), confirming that the invitation link could be manipulated to create unauthorized users.



Case-3 Privilege Escalation via Response Manipulation

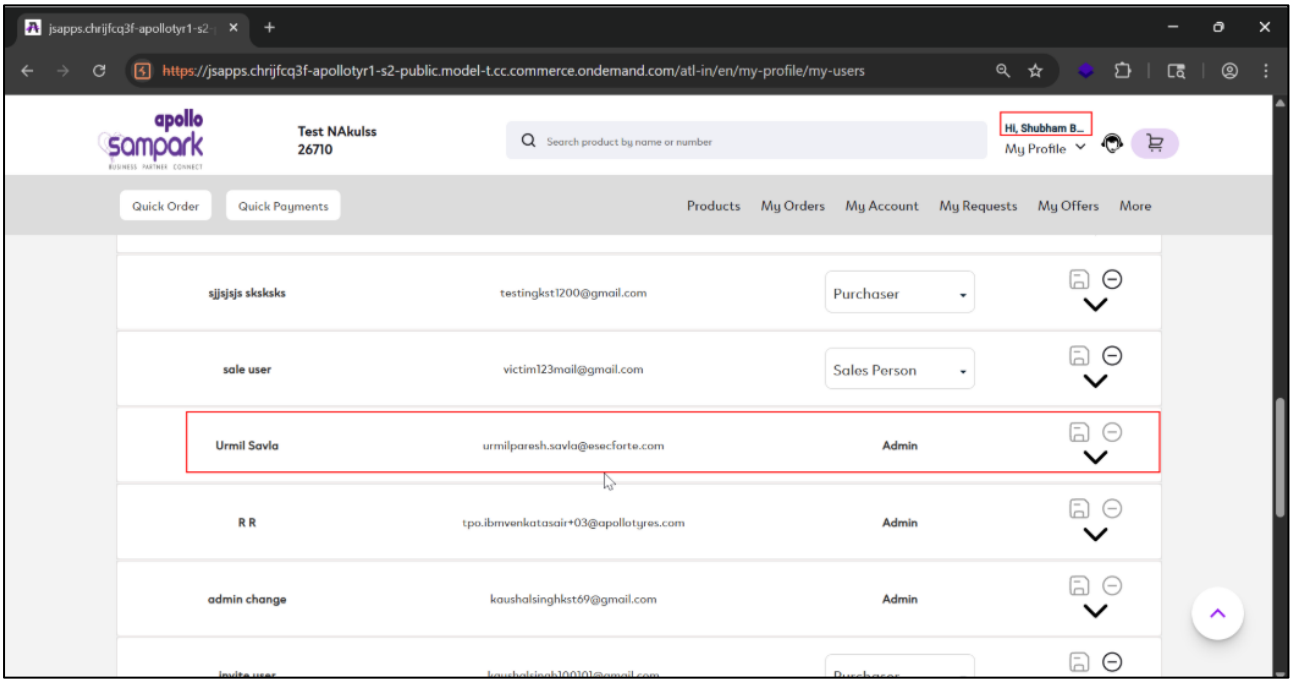
Step 1:

During the assessment, the auditor observed that they could log into the application as an admin and navigate to the *My Users* section as shown in the artifact below:



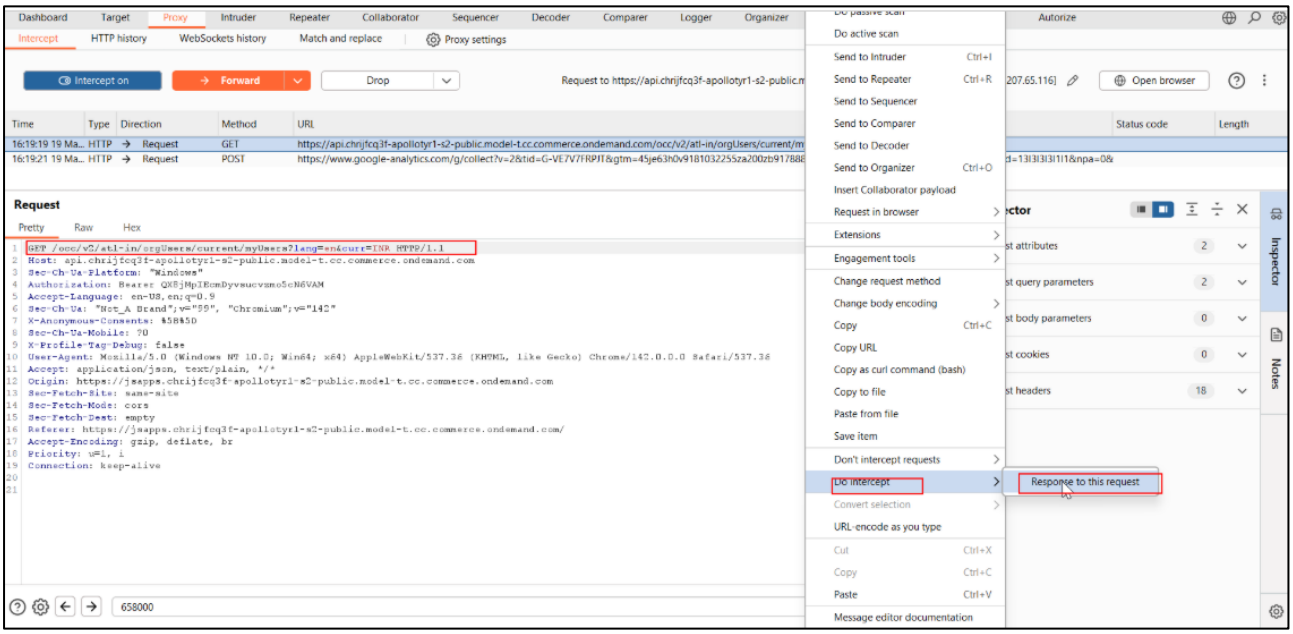
Step 2:

The auditor observes that there are multiple users with admin privileges and notes that admins are not intended to modify the privileges of other admin users as shown in the artifact below:



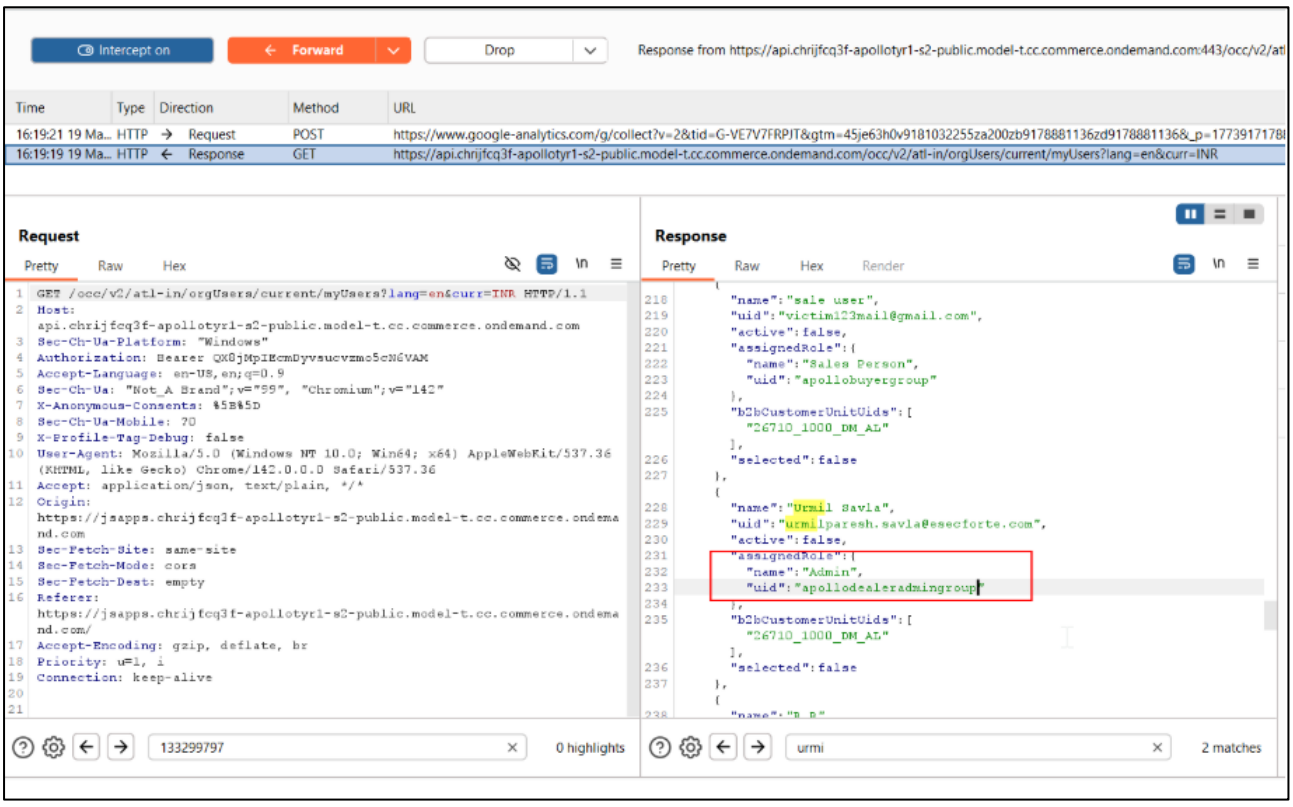
Step 3:

The auditor refreshes the page and intercepts the request/response for the /my-users endpoint using an intercepting proxy tool (e.g., Burp Suite) as shown in the artifacts below:



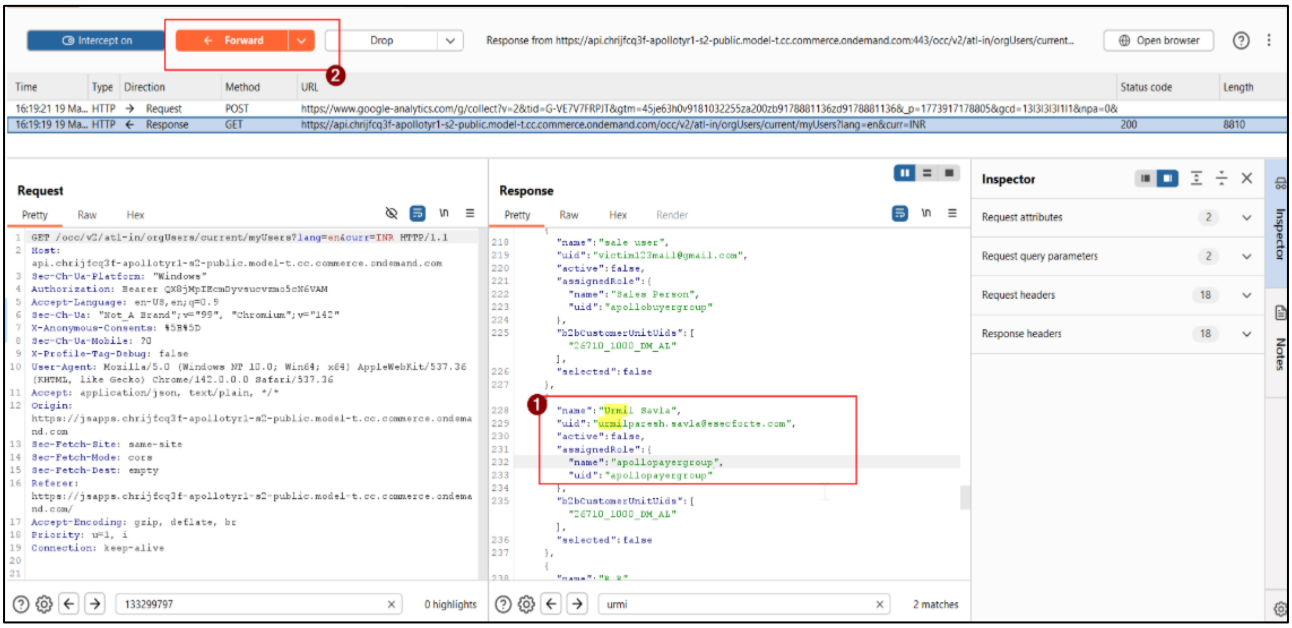
Step 4:

In the intercepted response, the auditor identifies the target admin user's details (e.g., name and role as admin dealer group) as shown in the artifact below:



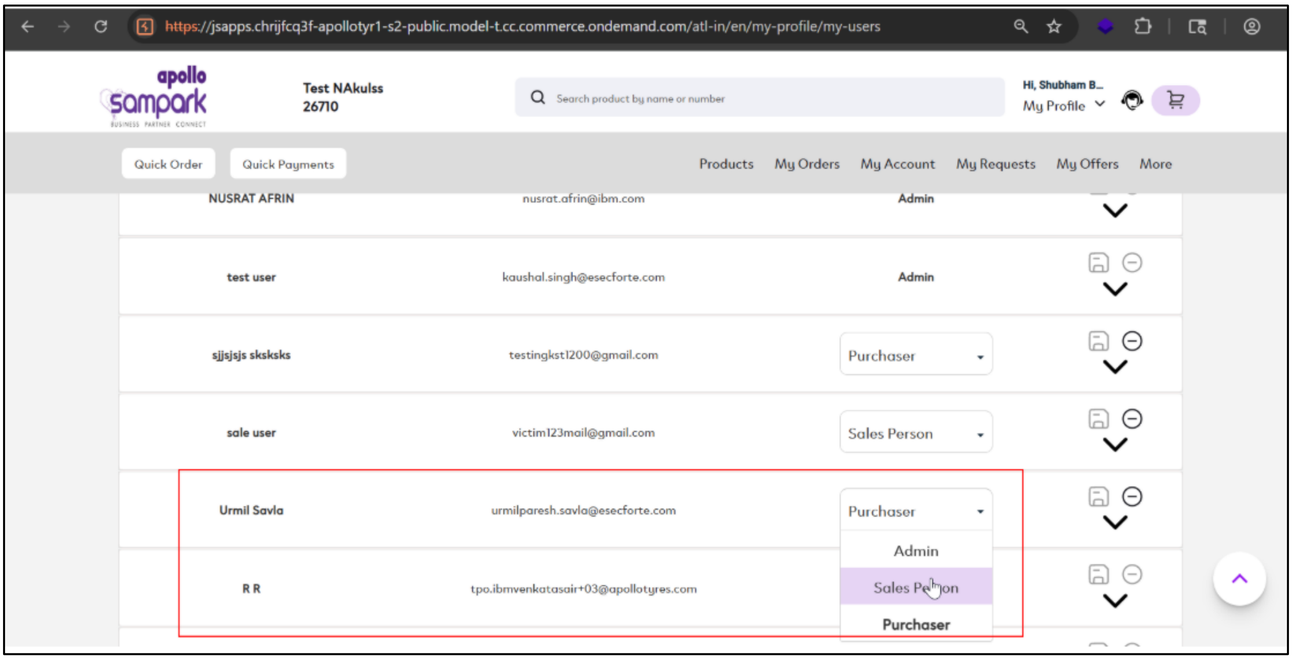
Step 5:

The auditor modifies the response by changing the role value from admin group to a lower-privileged group (e.g., peer group) and forwards the response to the browser, which now renders a dropdown instead of plain text for role selection as shown in the artifacts below:



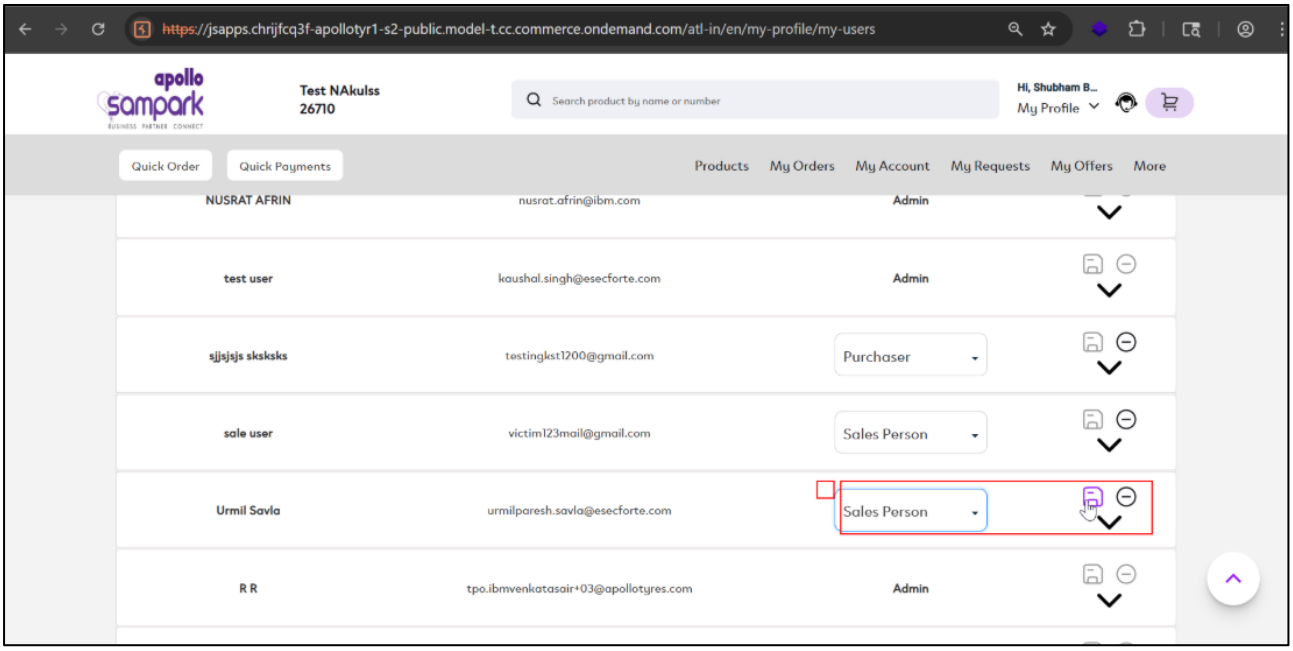
Step 6:

Using the newly exposed dropdown, the auditor selects a different role (e.g., salesperson) for the target admin user as shown in the artifact below:



Step 7:

The auditor clicks on save to update the role as shown in the artifact below:

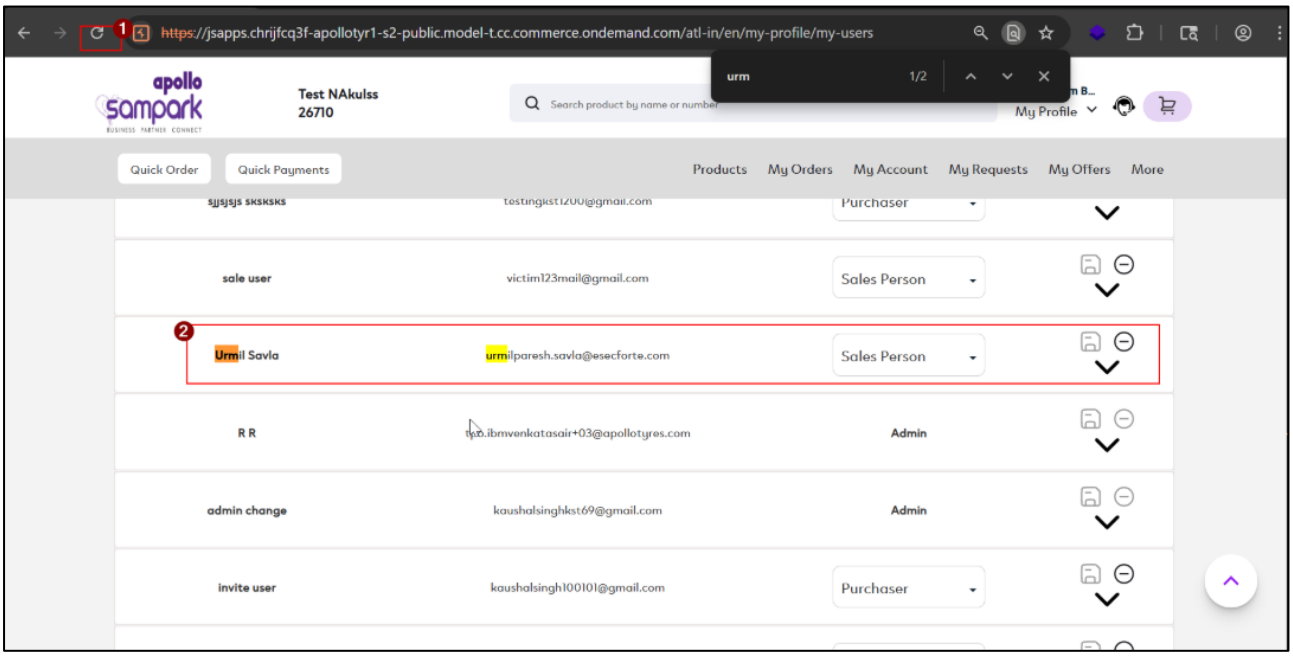


Step 8:

The corresponding request is intercepted and replayed via Repeater, receiving a 200 OK response indicating success as shown in the artifacts below:

Step 9:

The auditor refreshes the My Users page and confirms that the target admin user's role has been downgraded. Additionally, logging in as the affected user confirms reduced privileges as shown in the artifact below:



3.4 Broken Access Control

Description:

The application does not properly enforce access control on certain restricted functionalities. It was observed that a low-privileged user can directly access endpoints or features by force browsing, even though these actions should be restricted based on user roles. This indicates missing or insufficient server-side authorization checks.

Status:

Current Status: New

Overall CVSS Score:

6.3

Risk Rating:

High

Impact:

This vulnerability allows unauthorized users to access restricted functionalities beyond their intended privileges. Attackers may perform actions or view data that should only be accessible to higher-privileged roles. This compromises the integrity of role-based access control and may lead to further privilege escalation or data exposure.

Affected URL:

/atl-in/en/cart
/atl-in/en/credit-debit
/atl-in/en/my-account/account-statement
/atl-in/en/my-account/invoice-history
/atl-in/en/my-account/offtake-report
/atl-in/en/orders
/atl-in/en/my-account/saved-carts

Affected Parameters:

None

CWE/CVE Reference No:

CWE-284

Recommendation:

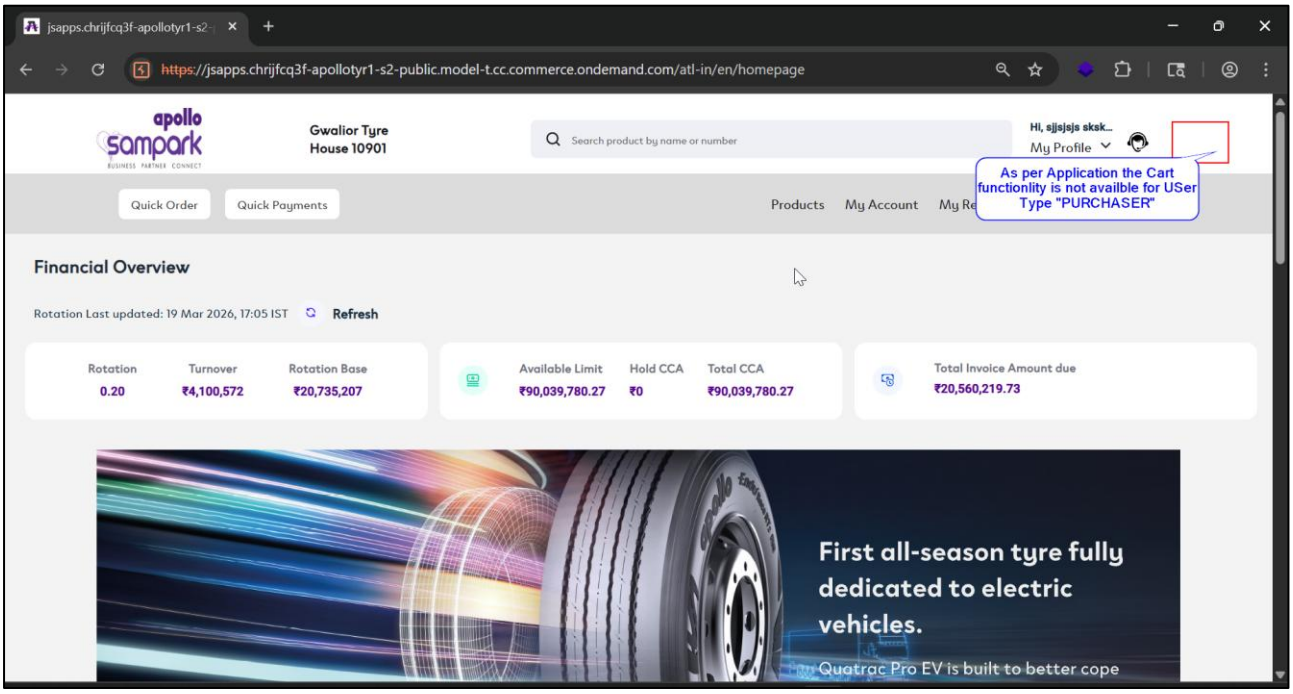
1. Enforce strict server-side authorization checks for all sensitive endpoints
2. Validate user roles and permissions before processing each request
3. Do not rely on client-side controls or hidden URLs for access restriction
4. Implement centralized role-based access control (RBAC) mechanisms
5. Return proper HTTP status codes (e.g., 403 Forbidden) for unauthorized access

POC:

Case-Cart Access via Forced Browsing & Feature Chaining

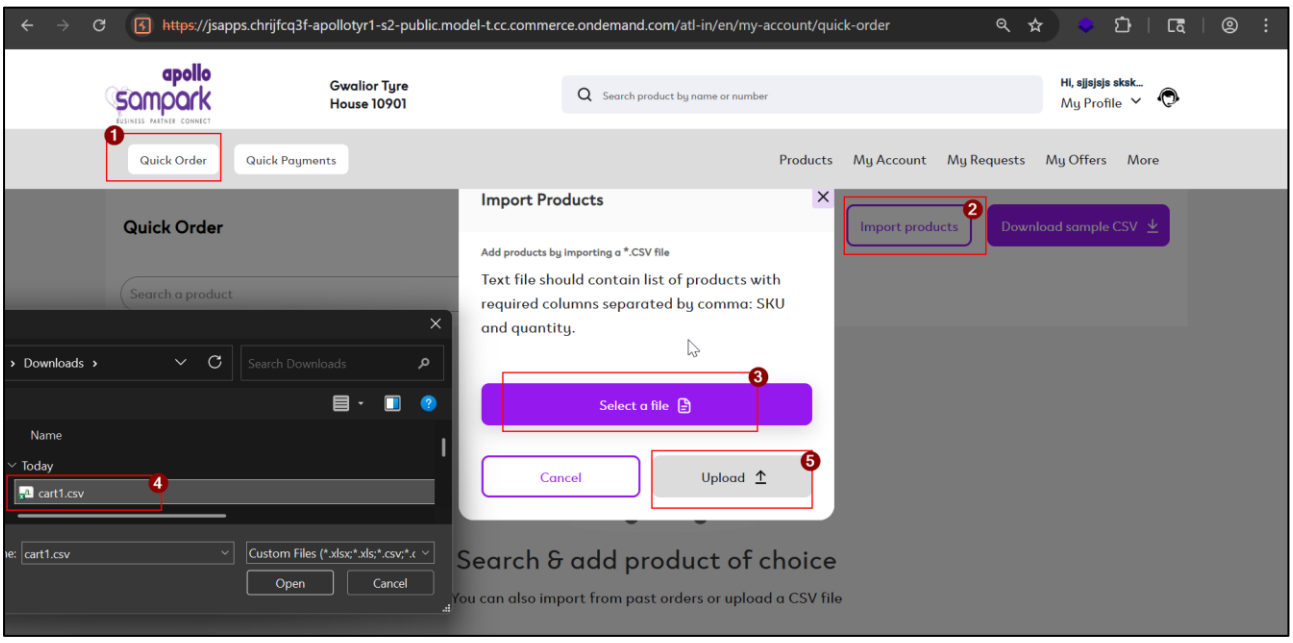
Step 1:

During the assessment, the auditor logs into the application using a low-privileged user (e.g., *peer* or *purchaser* role) and observes that the cart functionality is not available in the UI as shown in the artifact below:



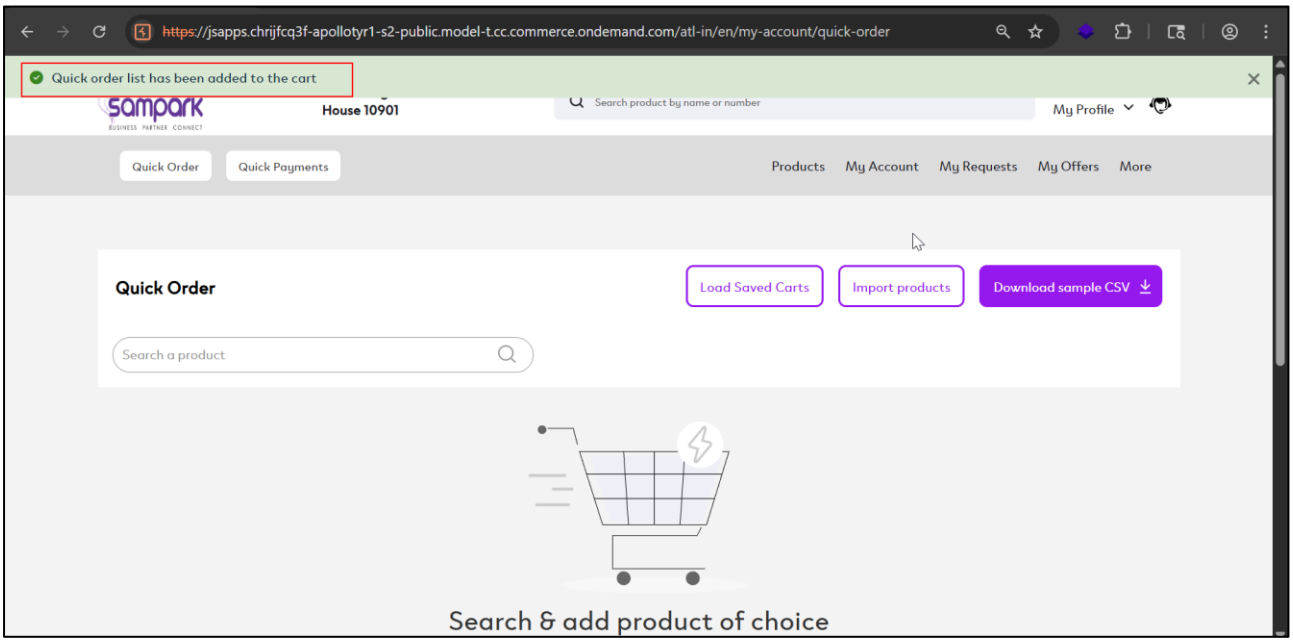
Step 2:

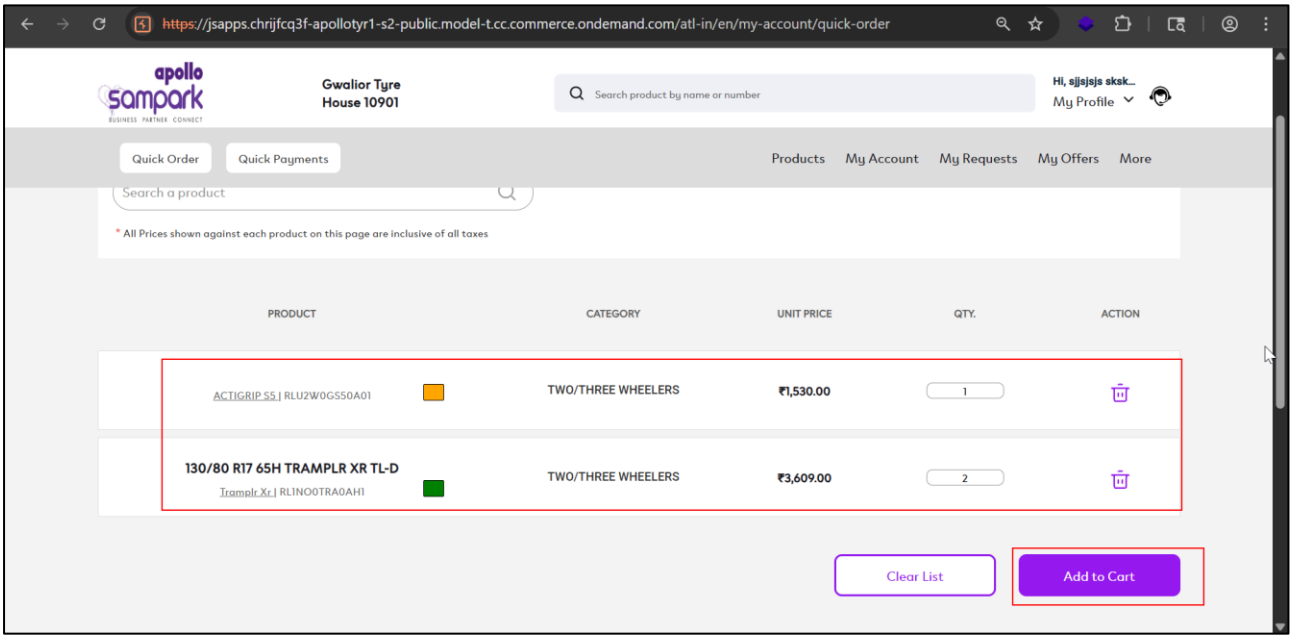
The auditor navigates to the Quick Order functionality and uses the Import Products feature to upload a CSV file containing product IDs and quantities as shown in the artifact below:



Step 3:

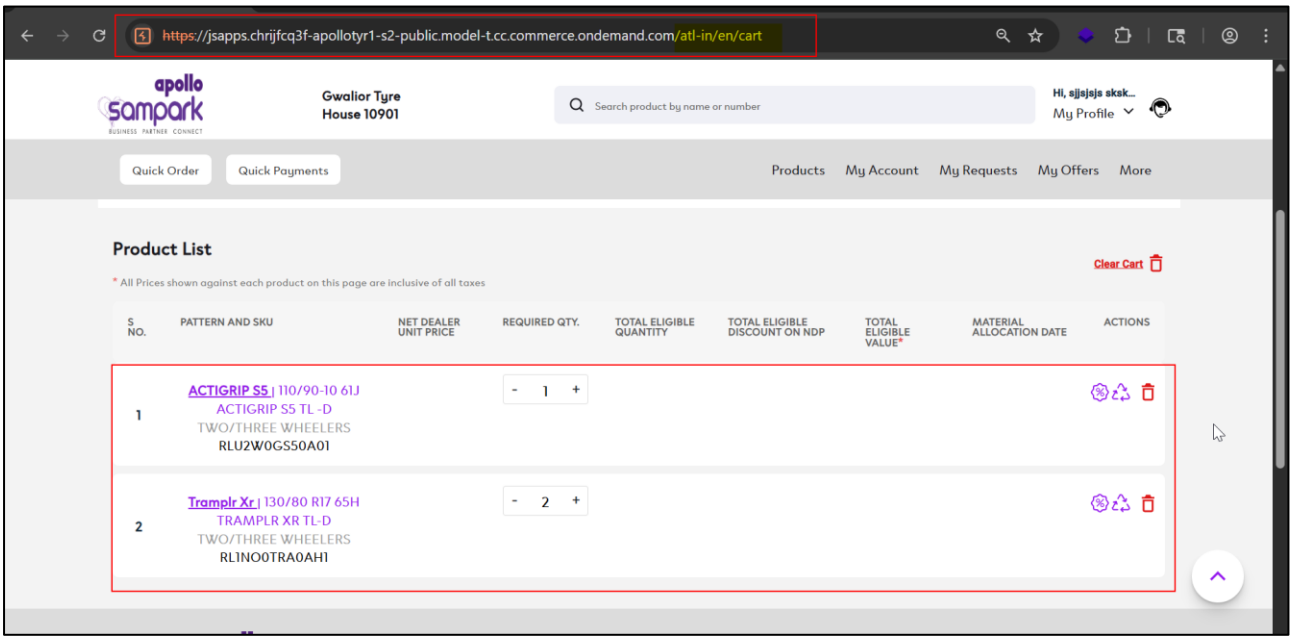
After processing the CSV file, the products are successfully mapped and displayed. The auditor clicks on *Add to Cart*, and the items are added to the cart using the quick order functionality as shown in the artifact below:





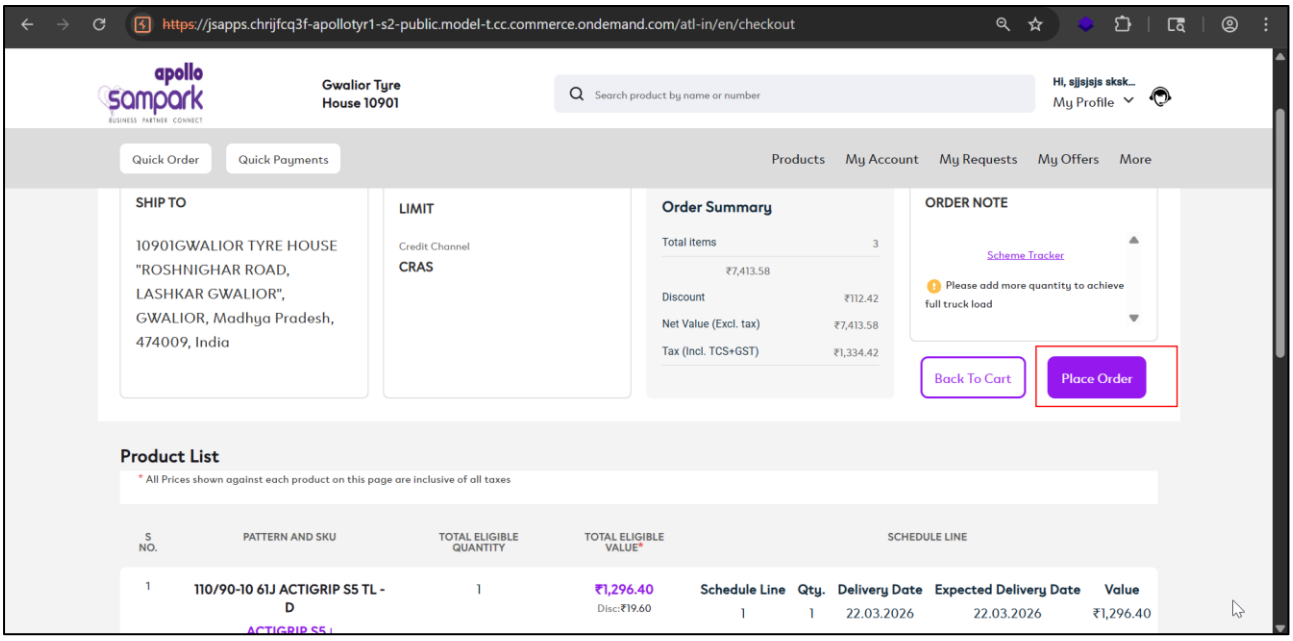
Step 4:

Since the cart is not accessible via UI, the auditor manually forces browsers to the /cart endpoint and observes that the previously added products are present in the cart as shown in the artifact below:

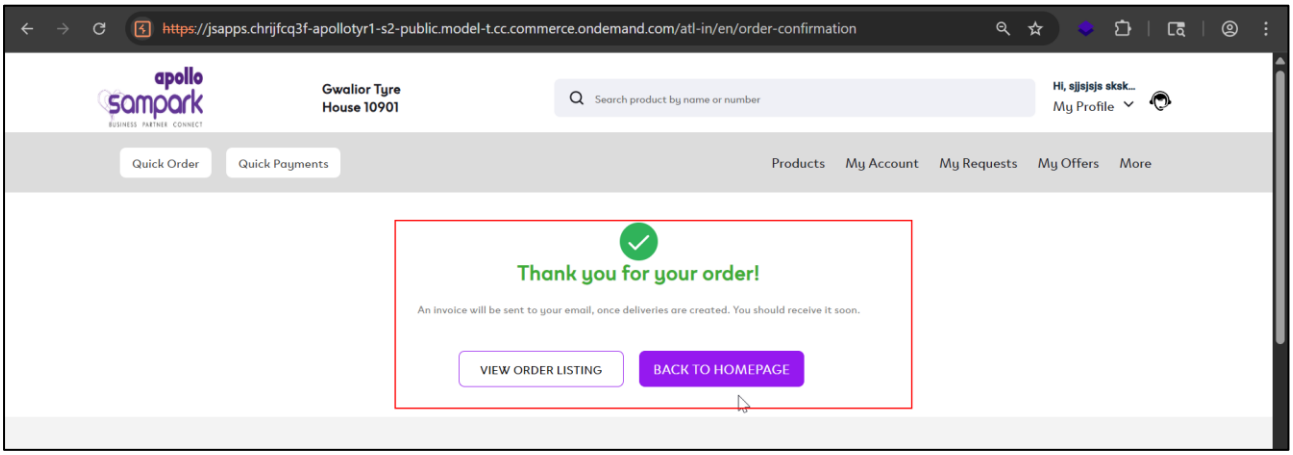


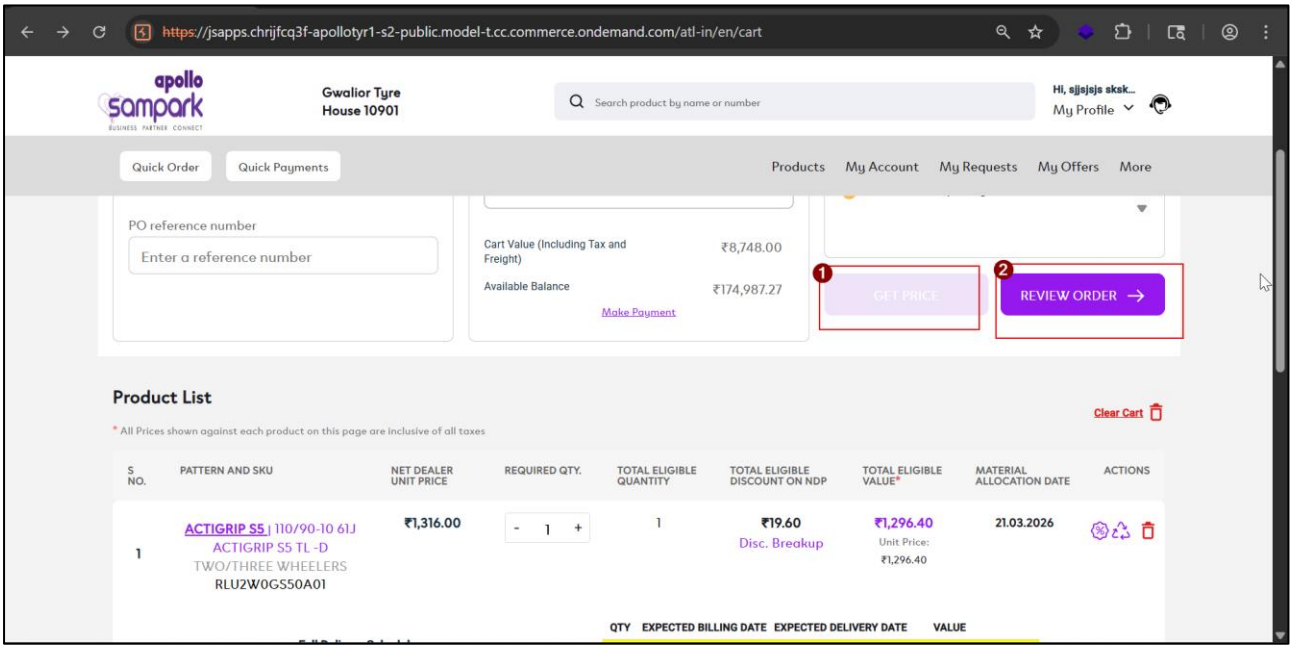
Step 5:

The auditor proceeds with the checkout flow (e.g., pricing, order review, and placing the order) and successfully places the order, confirming that restricted cart functionality is accessible to a low-privileged user as shown in the artifact below:



WEB APPLICATION
SECURITY ASSESSMENT

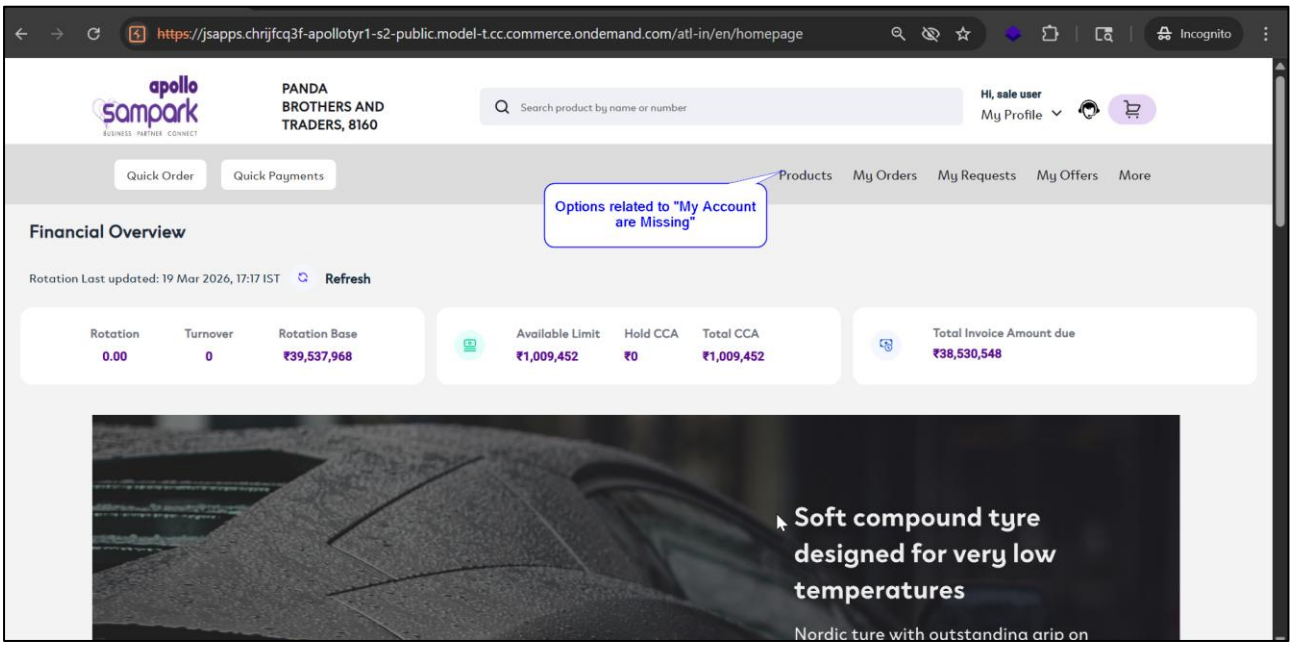




Case-Access to My Account Functionalities via Forced Browsing

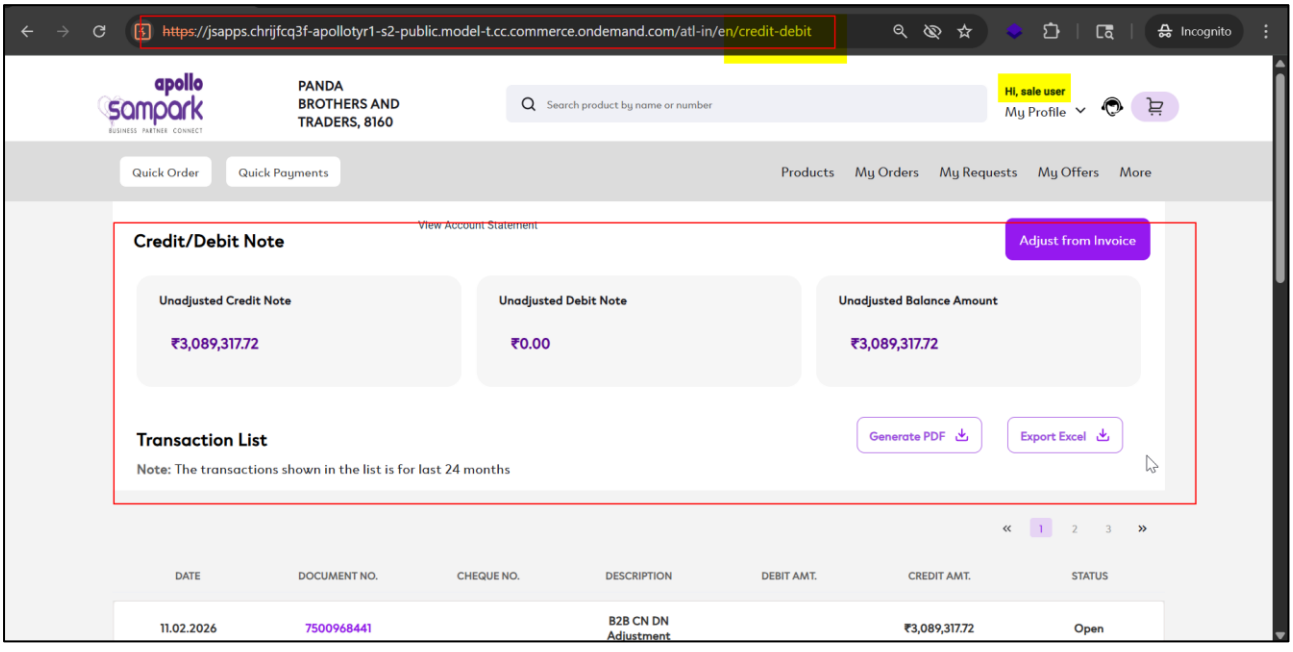
Step 1:

During the assessment, the auditor logs into the application using a low-privileged user (e.g., *buyer* role) and observes that *My Account*-related options are not available in the UI as shown in the artifact below:



Step 2:

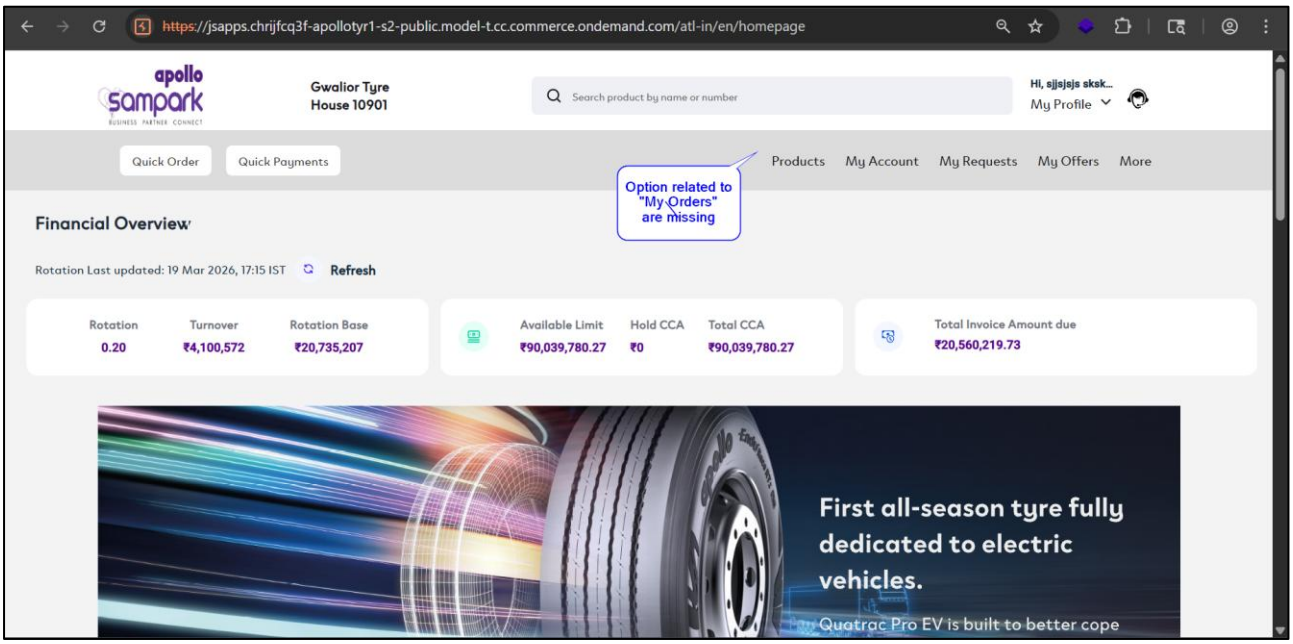
The auditor manually forces browses to known endpoints such as /credit-debit, /account-statement, /invoice-history, and /optik-report, and observes that the application allows access to these functionalities despite UI restrictions as shown in the artifact below:



Case-3 Access to Order Functionalities via Forced Browsing

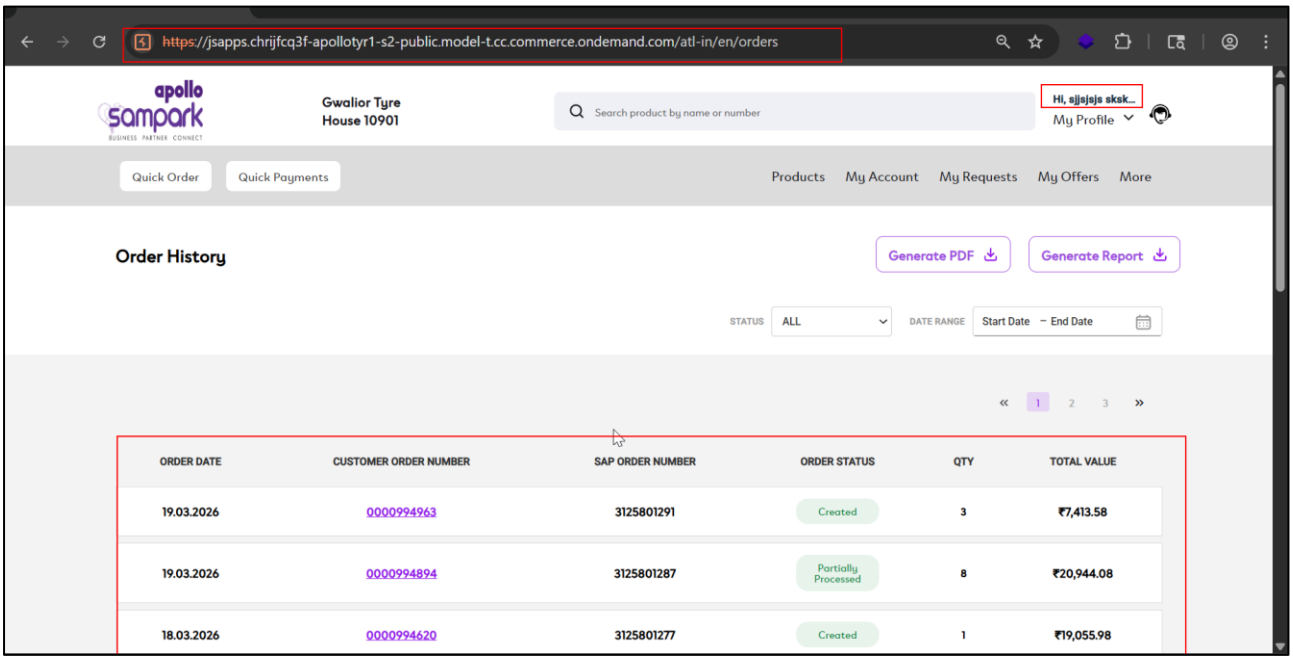
Step 1:

The auditor logs into the application using a low-privileged user (e.g., payer role) and observes that My Orders-related functionalities are not available in the UI as shown in the artifact below:



Step 2:

The auditor force browses to endpoints such as /orders, /saved-carts, and /carts, and observes that these functionalities are accessible despite being restricted for the user role as shown in the artifact below:



3.5 Lack of Rate Limiting

Description:

The application does not implement rate limiting on the user invitation functionality. An attacker can send many requests in a short period without any restriction or throttling. This allows abuse of the feature for unintended mass actions.

Status:

Current Status: New

Overall CVSS Score:

4.3

Risk Rating:

Medium

Impact:

Lack of rate limiting enables attackers to flood user inboxes with invitation emails, leading to denial of service or annoyance. It may also impact on the application's email infrastructure and reputation (e.g., email blacklisting). This can degrade user experience and system reliability.

Affected URL:

https://jsapps.chrijfcq3f-apollotyr1-s2-public.model-t.cc.commerce.ondemand.com/atl-in/en/my-profile/invite-users

Affected Parameters:

Application

OWASP Ref No:

https://owasp.org/www-project-api-security/

CWE/CVE Reference No:

CWE-770

Recommendation:

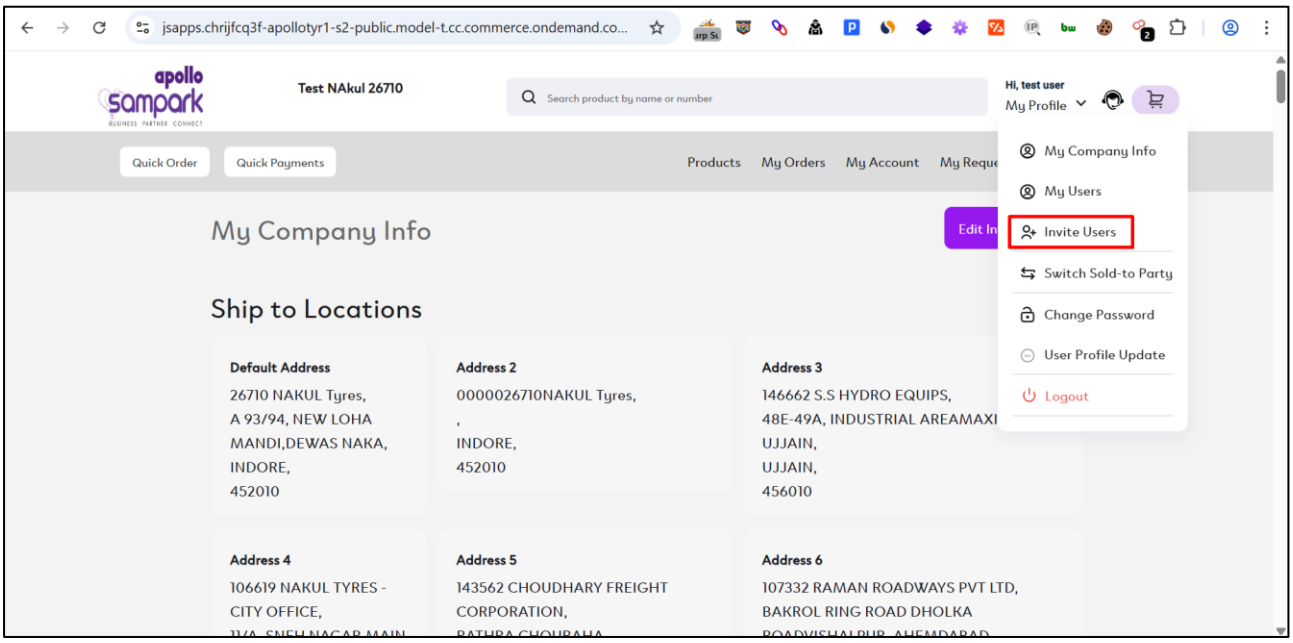
- Implement rate limiting on the invitation endpoint (e.g., limit requests per user/IP)
- Introduce CAPTCHA or challenge-response mechanisms for repeated requests
- Restrict the number of invitations per user within a defined time window
- Monitor and block abnormal or excessive request patterns

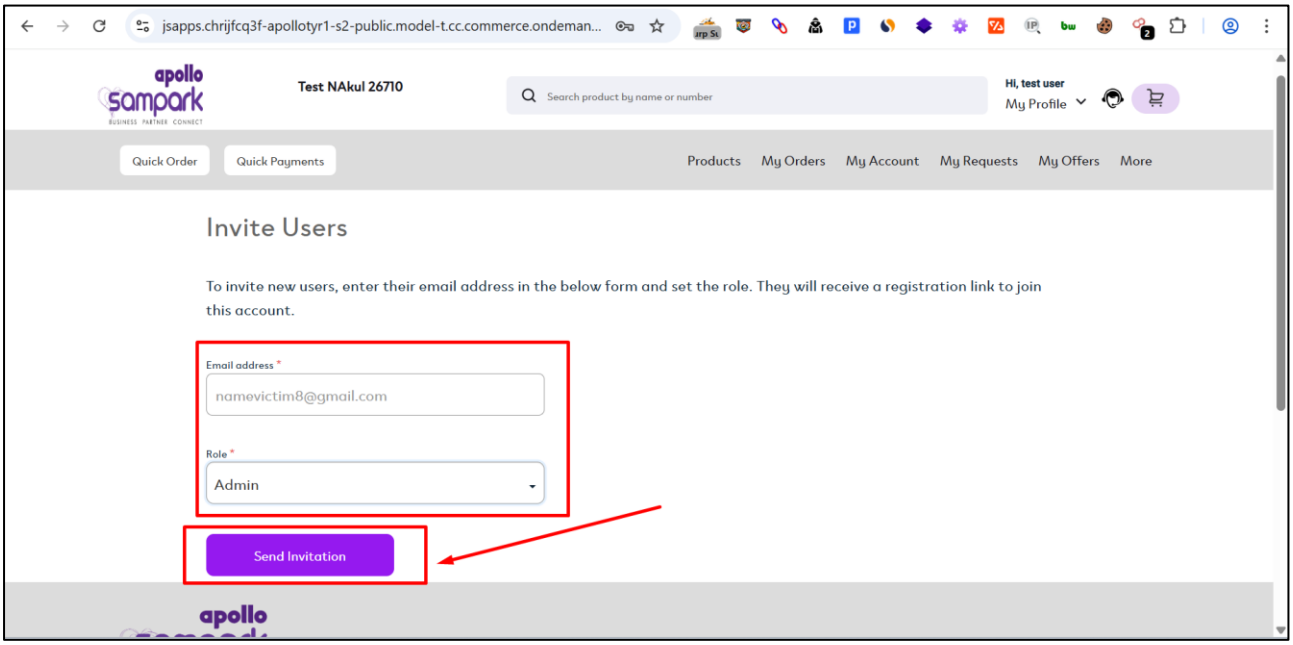
POC:

Steps to Reproduce

Step 1:

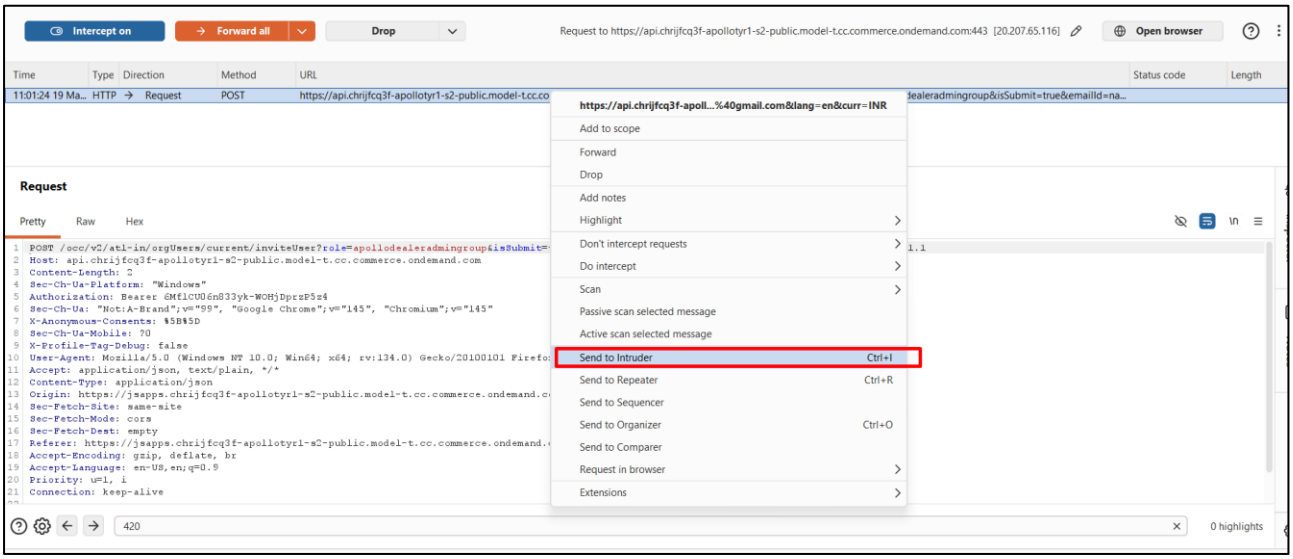
During the assessment, the auditor logs into the application with a privileged (admin) account, navigates to *My Profile* → *Invite Users*, and enters an email address along with the desired role as shown in the artifact below:





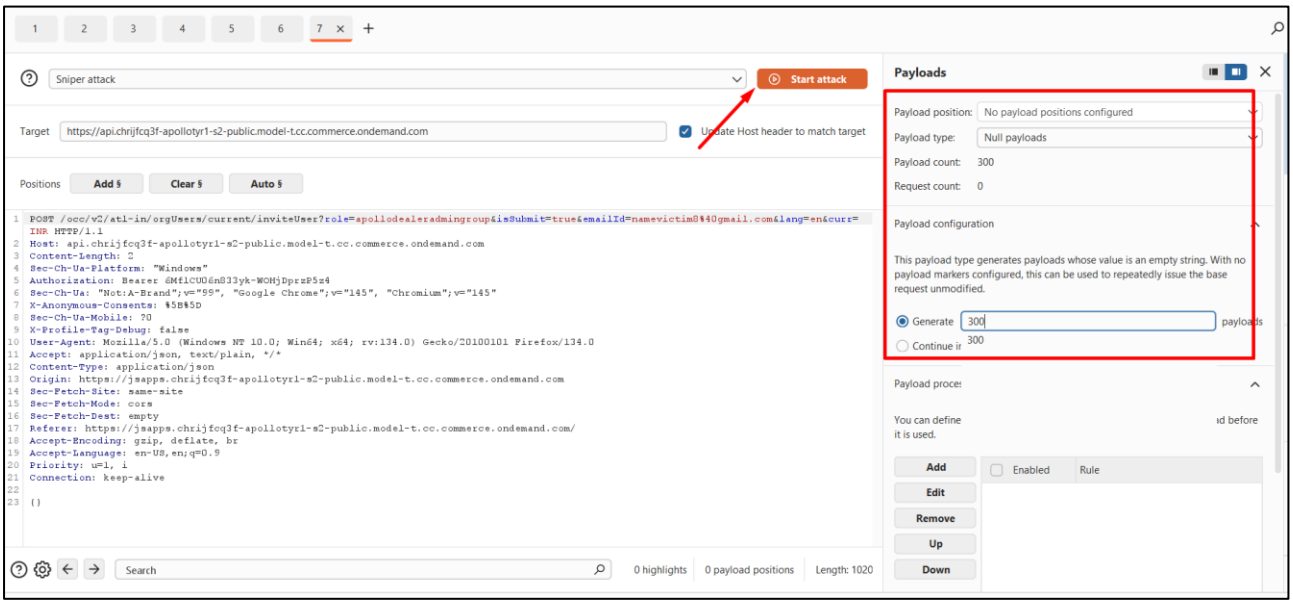
Step 2:

The auditor clicks the submit button, intercepts the request using *Burp Suite*, and sends it to *Intruder,* as shown in the artifact below:



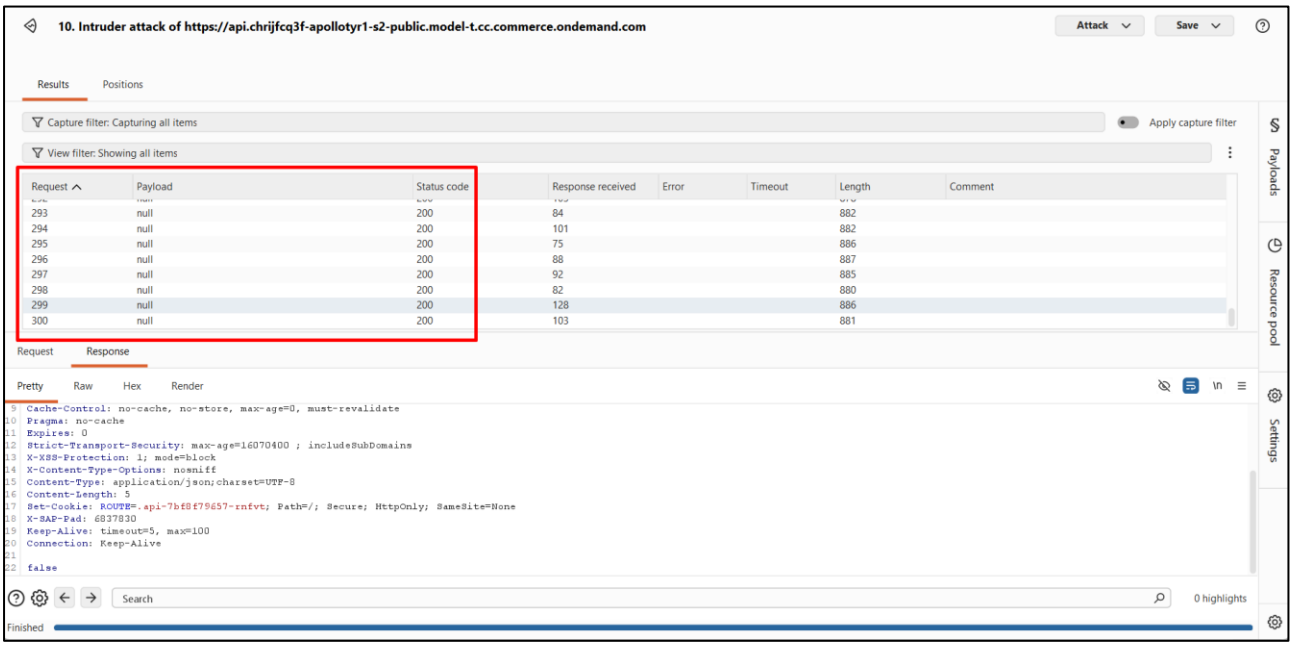
Step 3:

The auditor configures Intruder with null payloads and generates ~300 requests, then starts the attack as shown in the artifact below:



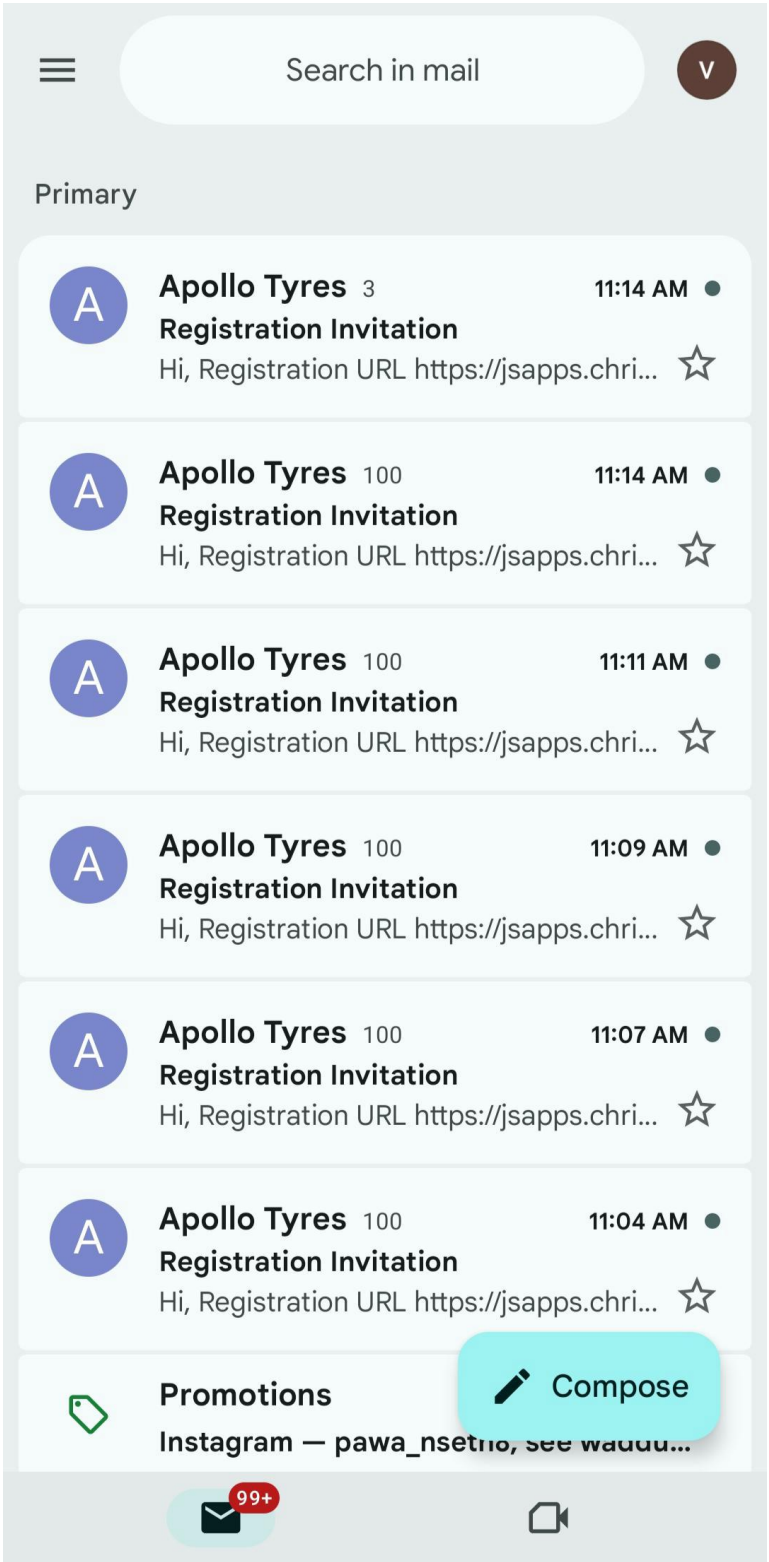
Step 4:

The responses are observed with HTTP status code 200, indicating all requests were processed successfully without restriction as shown in the artifact below:



Step 5:

The auditor verifies that the target email inbox receives many invitation emails, confirming email flooding as shown in the artifact below:



3.6 Session Token in URL

Description:

The application transmits session-related tokens (e.g., JWT) within URL query parameters during authentication flows. URLs may be stored in browser history, logs, and intermediary systems, making them an insecure channel for sensitive data. This indicates improper handling of authentication tokens and insecure transmission practices.

Status:

Current Status: New

Overall CVSS Score:

4.3

Risk Rating:

Medium

Impact:

Exposure of session tokens in URLs can lead to unintended leakage through browser history, logs, or referer headers. An attacker who obtains the token may reuse it to hijack user sessions. This increases the risk of unauthorized access and compromise of user accounts.

Affected URL:

https://cdns.eu1.gigya.com/accounts.auth.getMethods?

Affected Parameters:

Application

OWASP Ref No:

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

CWE/CVE Reference No:

CWE-598

Recommendation:

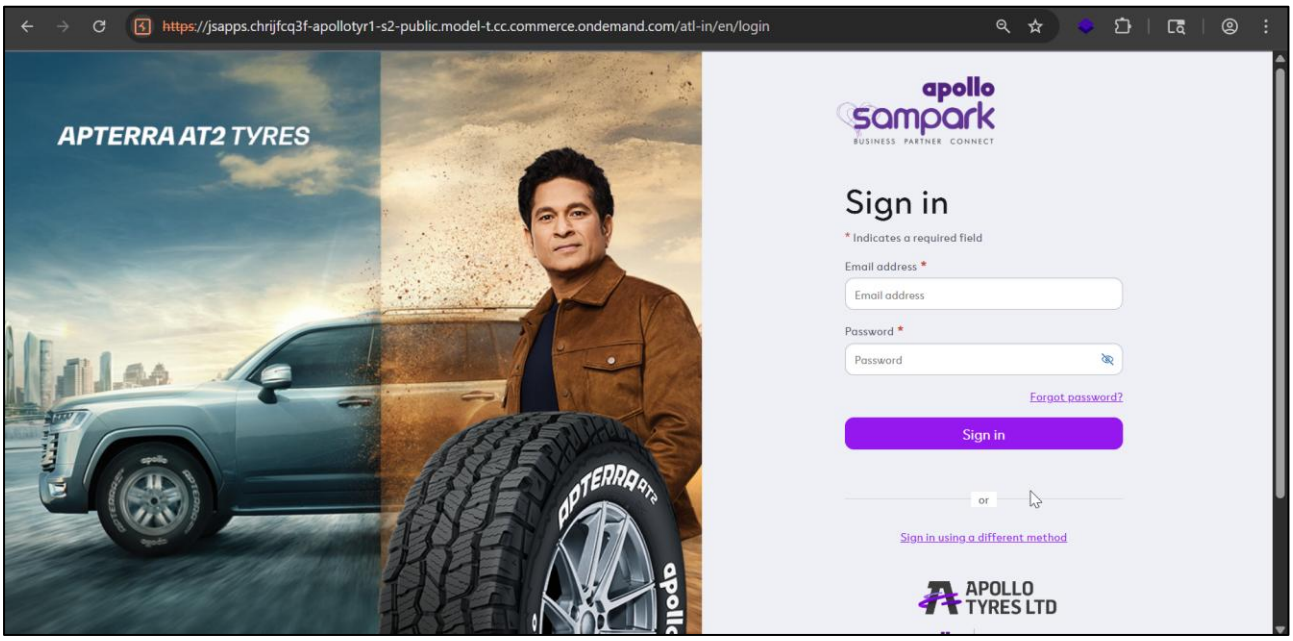
1. Avoid sending sensitive tokens in URL parameters; use HTTP headers (e.g., Authorization) or secure cookies instead
2. Ensure tokens are transmitted only via POST requests or headers, not in GET URLs
3. Implement short-lived tokens and proper session expiration controls
4. Prevent leakage via logs by sanitizing or masking sensitive data
5. Enforce HTTPS across all endpoints to protect data in transit

POC:

Steps to Reproduce

Step 1:

During the assessment, the auditor navigates to the login page, enters valid credentials, and clicks *Sign In* while capturing the traffic using an intercepting proxy tool, as shown in the artifact below:



Step 2:

During analysis of the captured requests, the auditor observes a request to the authentication service where a JWT/session token is passed as a GET parameter in the URL as shown in the artifact below:

3.7 Missing HTTP Security Headers

Description:

The application does not implement key HTTP security headers such as Content Security Policy (CSP), Referrer-Policy, and Permissions-Policy. These headers help enforce browser-side security controls by restricting resource loading, controlling referrer data, and limiting access to sensitive browser features. Their absence weakens protection against common client-side attacks.

Status:

Current Status: New

Overall CVSS Score:

3.1

Risk Rating:

Low

Impact:

The absence of these headers increases the risk of Cross-Site Scripting (XSS) and other client-side attacks. Sensitive information such as URLs or tokens may be exposed via the Referer header to external domains. Additionally, unrestricted browser feature access can increase the attack surface and lead to potential misuse.

Affected URL:

https://api.chrijfcq3f-apollotyr1-s2-public.model-t.cc.commerce.ondemand.com/occ/v2/atl-in/creditLimit/B2BUnit/10901_1000_DM_AL/ALL?lang=en&curr=INR

Affected Parameters:

Application

OWASP Ref No:

https://owasp.org/www-project-secure-headers/

CWE/CVE Reference No:

CWE-693

Recommendation:

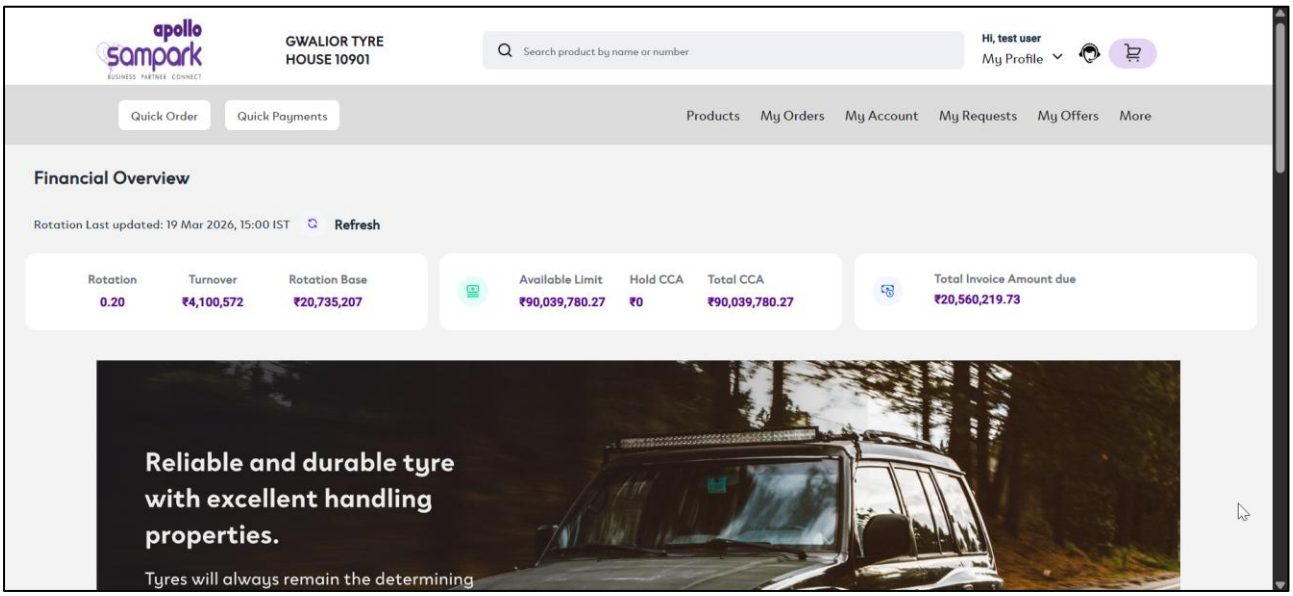
- Implement the following security headers with appropriate values:
- Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; frame-ancestors 'none';
- Avoid use of 'unsafe-inline' and 'unsafe-eval' in CSP and allow only trusted domains
- Referrer-Policy: strict-origin-when-cross-origin
- Permissions-Policy: geolocation=(), camera=(), microphone=()
- Ensure headers are configured at the web server, reverse proxy, or application level
- Apply these headers consistently across all application responses

POC:

Steps to Reproduce

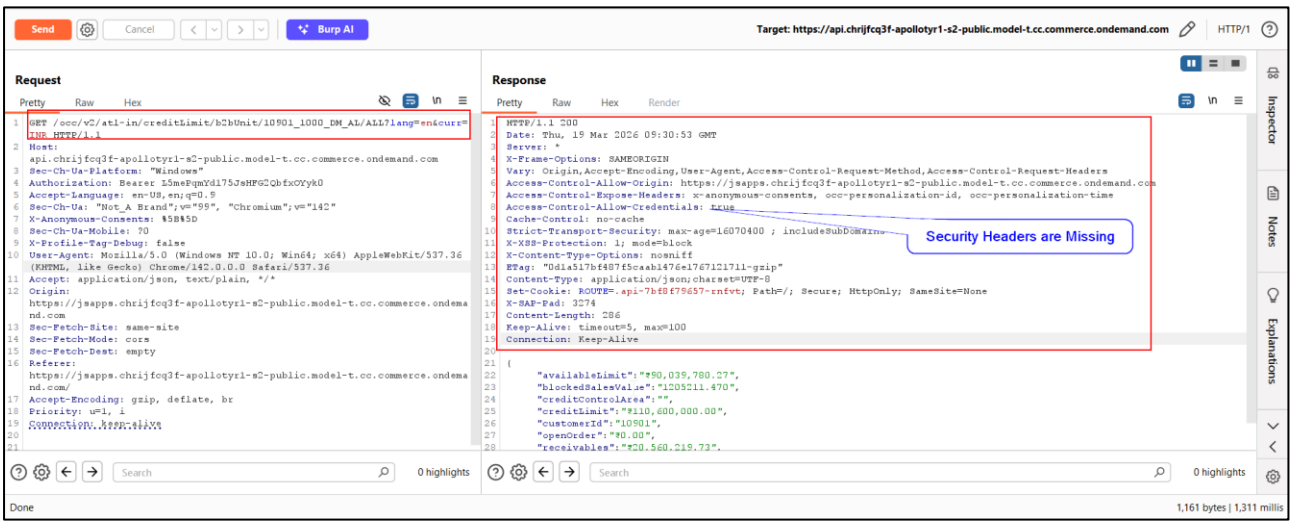
Step 1:

During the assessment, the auditor logs into the application with valid credentials and navigates to the Home Page as shown in the artifact below:



Step 2:

Using Burp Suite, the auditor intercepts the API request and analyzes the server response, observing that important security headers are missing as shown in the artifact below:



3.8 Misconfigured CORS

Description:

The application reflects the user-supplied Origin header in the Access-Control-Allow-Origin response header without proper validation. Additionally, Access-Control-Allow-Credentials is set to true, allowing authenticated cross-origin requests. This indicates improper CORS configuration where untrusted origins are implicitly trusted.

Status:

Current Status: New

Overall CVSS Score:

3.1

Risk Rating:

Low

Impact:

This misconfiguration allows an attacker-controlled domain to make authenticated requests to the application on behalf of a victim user. Sensitive data can be accessed and responses read by the attacker, leading to potential account data exposure. This effectively results in unauthorized actions and data exfiltration via the victim's browser session.

Affected URL:

https://api.chrijfcq3f-apollotyr1-s2-public.model-t.cc.commerce.ondemand.com/occ/v2/atl-in/creditLimit/B2BUnit/10901_1000_DM_AL/ALL?lang=en&curr=INR

Affected Parameters:

Application

OWASP Ref No:

https://owasp.org/www-community/attacks/CORS_OriginHeaderScrutiny

CWE/CVE Reference No:

CWE-16

Recommendation:

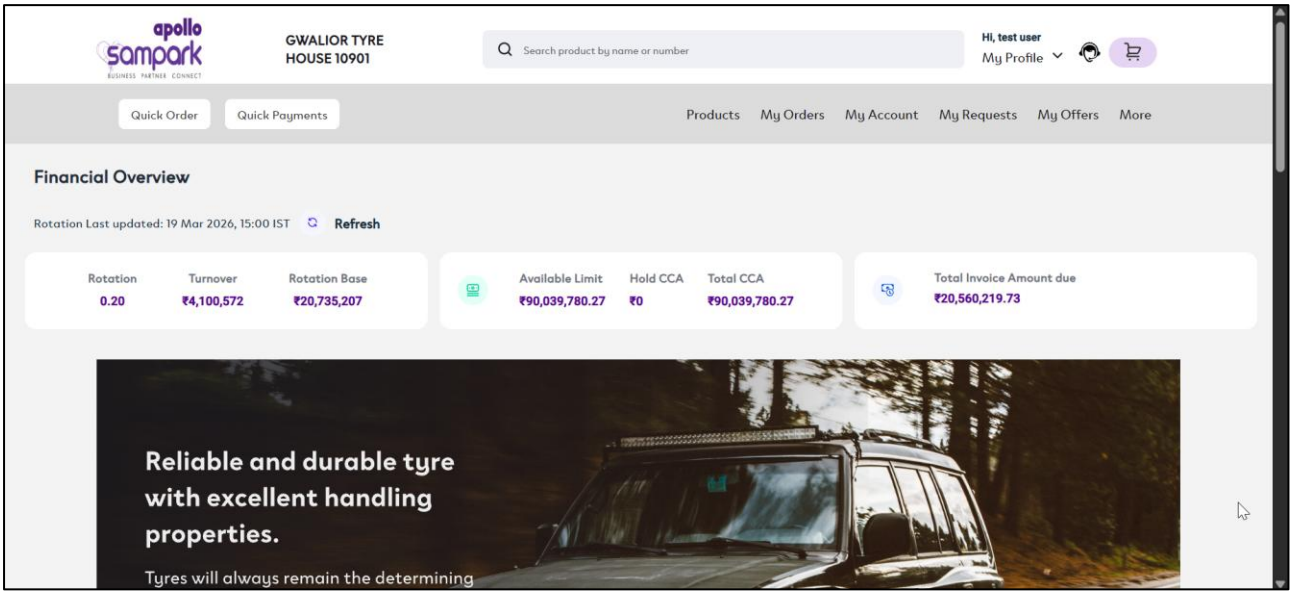
- Implement strict origin validation and secure CORS configuration:
- Access-Control-Allow-Origin: https://trusted-domain.com
- Access-Control-Allow-Credentials: true
- Do not reflect arbitrary origins; allow only whitelisted trusted domains
- Avoid using Access-Control-Allow-Credentials: true unless strictly required
- Validate the Origin header on the server side before including it in responses
- Apply consistent CORS policies across all endpoints and APIs

POC:

Steps to Reproduce

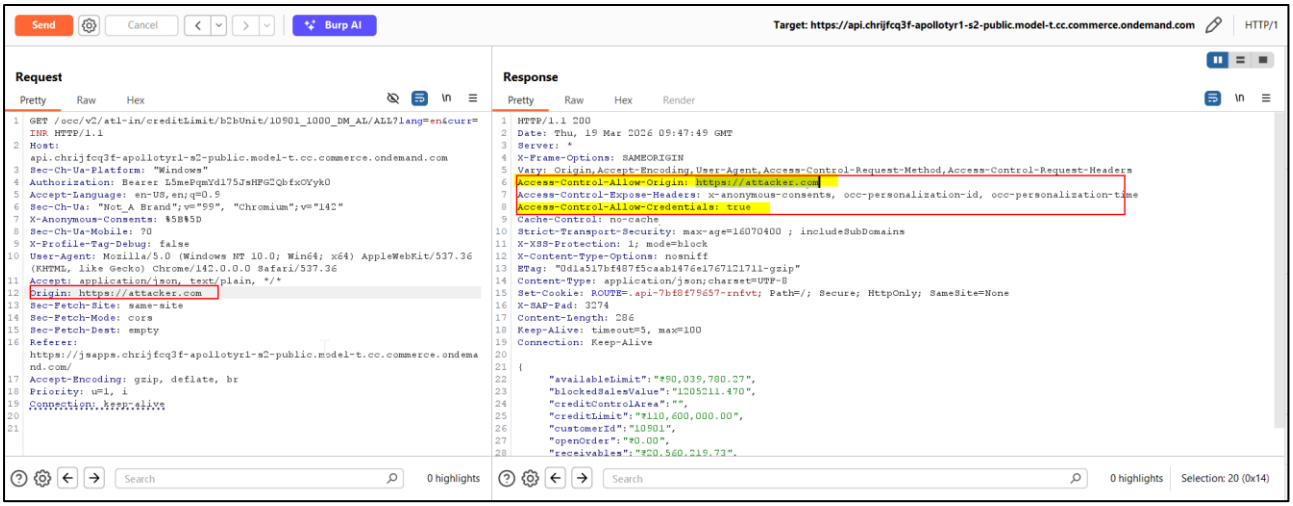
Step 1:

The auditor logs into the application with valid credentials and navigates to the Home Page.



Step 2:

Using *Burp Suite*, the auditor intercepts the API request and analyzes the server response, observing that CORS is misconfigured, as shown in the artifacts below:



3.9 Concurrent Login

Description:

The application allows multiple simultaneous active sessions for the same user account across different devices or browsers. There are no restrictions or controls in place to limit or manage concurrent logins. This may indicate lack of session management controls.

Status:

Current Status: New

Overall CVSS Score:

3.1

Risk Rating:

Low

Impact:

Allowing concurrent sessions can increase the risk of unauthorized access if user credentials are compromised. An attacker may maintain persistent access without alerting the legitimate user. It also limits the ability to detect and prevent suspicious session activity.

Affected URL:

https://jsapps.chrijfcq3f-apollotyr1-s2-public.model-t.cc.commerce.ondemand.com/atl-in/en/homepage

Affected Parameters:

Application

OWASP Ref No:

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

CWE/CVE Reference No:

CWE-1018

Recommendation:

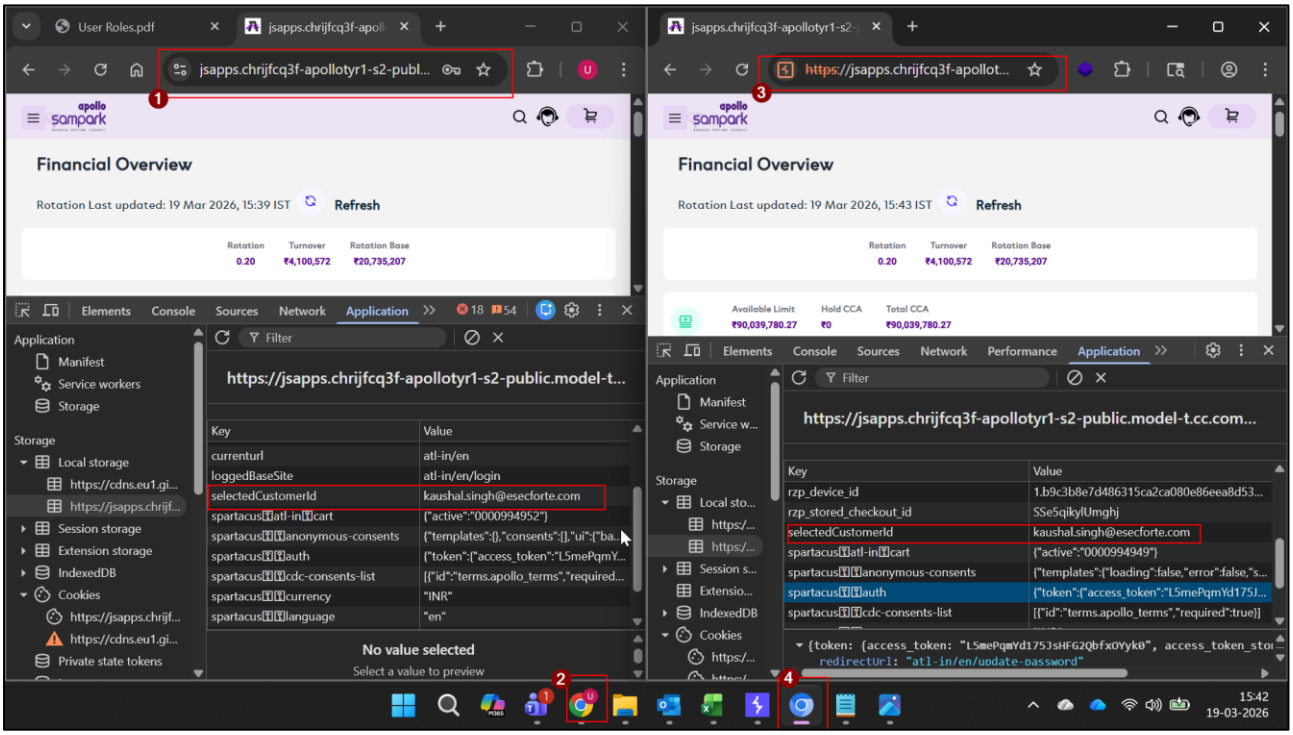
- Implement controls to restrict or manage concurrent user sessions
- Invalidate previous sessions upon new login (single-session enforcement)
- Provide visibility of active sessions and allow users to terminate them
- Trigger alerts or re-authentication for logins from new devices/locations

POC:

Case-Concurrent Login

Step 1:

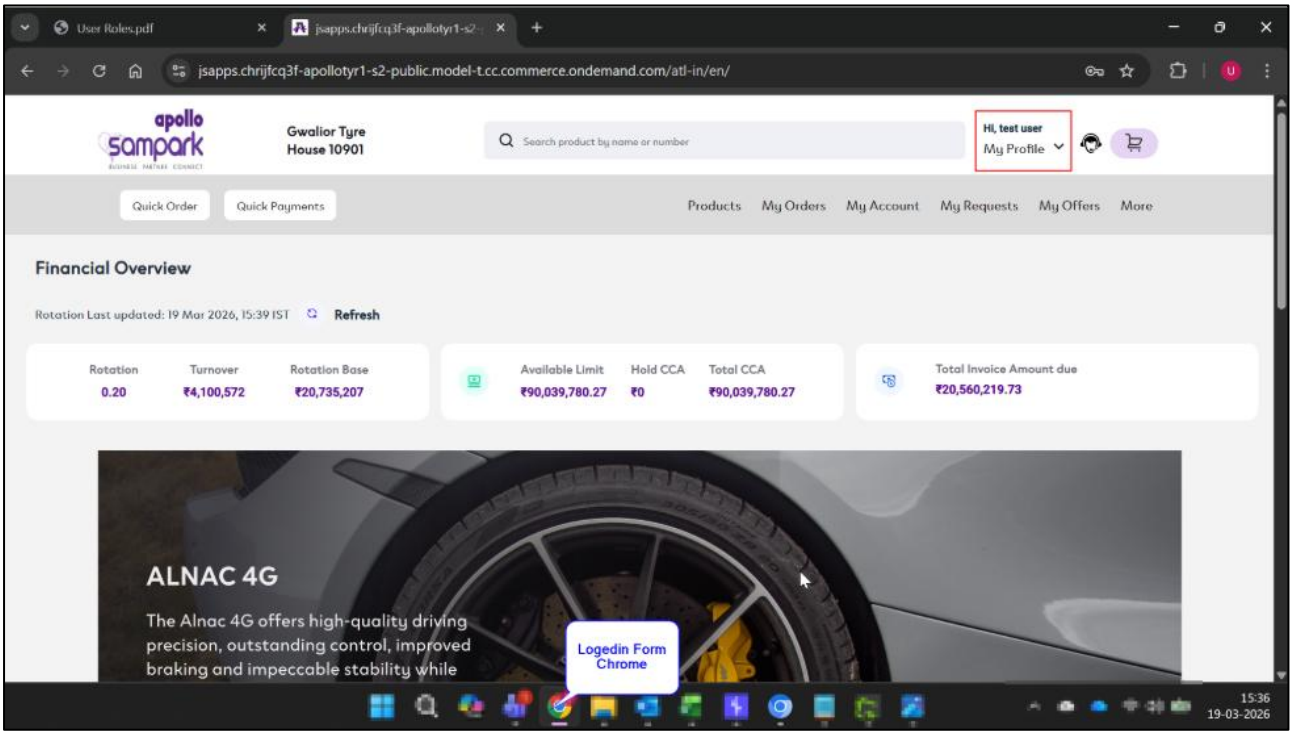
During the assessment, the auditor logs into the same user account simultaneously using two different browsers (e.g., Chrome and Chromium). In both sessions, the application remains active without invalidating the previous session. Upon inspecting browser storage (*Application → Local Storage*), the auditor observes that the same user/session context (e.g., customer ID) is active in both browsers, confirming that concurrent sessions are allowed as shown in the artifact below:



Case-Failure to Invalidate Session after Password Change

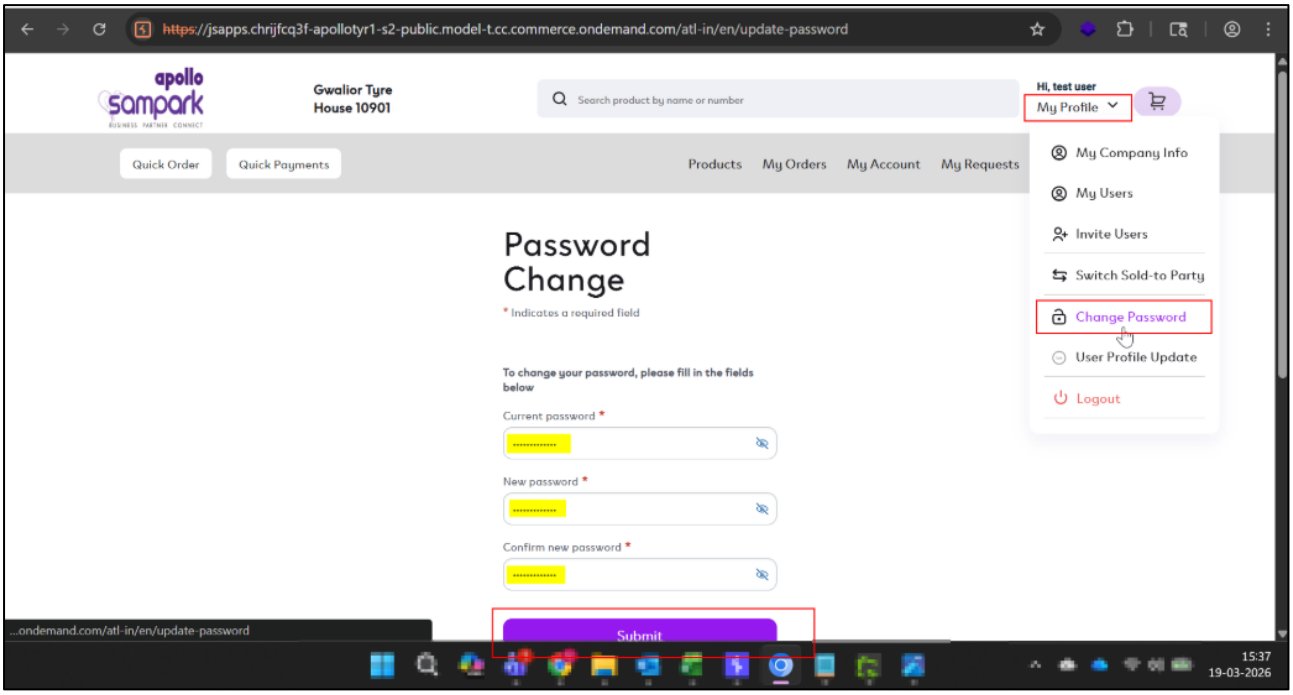
Step 1:

During the assessment, the auditor observed that the application allows login using a valid account via the Chrome browser, as shown in the artifact below:



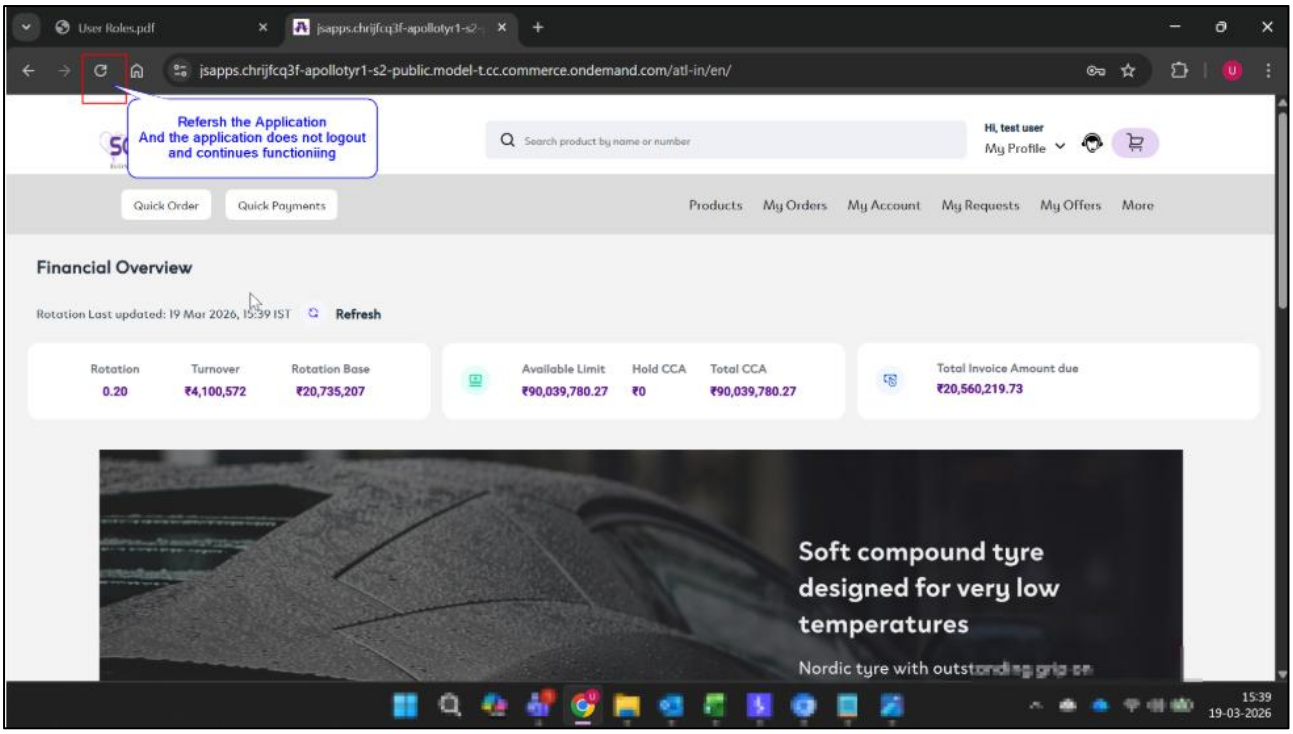
Step 2:

Now, the auditor logs into the same account using a different browser (Chromium), navigates to *My Profile → Change Password*, enters the current and new password, and successfully updates the password, as shown in the artifact below:



Step 3:

The auditor returns to the Chrome browser and refreshes the application, observing that the session remains active and the user is not logged out, as shown in the artifacts below:



3.10 EXIF Geolocation Data Not Stripped From Uploaded Images

Description:

The application does not remove EXIF metadata from uploaded images. It was observed that metadata such as location and custom comments added to the image remained intact even after uploading and downloading the file. This indicates lack of proper sanitization of user-uploaded files.

Status:

Current Status: New

Overall CVSS Score:

3.1

Risk Rating:

Low

Impact:

Exposure of EXIF metadata can lead to unintended disclosure of sensitive information such as geographic location and user-added data. This may result in privacy risks and potential information leakage. Attackers may leverage this data for reconnaissance or targeted attacks.

Affected URL:

https://jsapps.chrijfcq3f-apollotyr1-s2-public.model-t.cc.commerce.ondemand.com/atl-in/en/create-request

Affected Parameters:

Application

Recommendation:

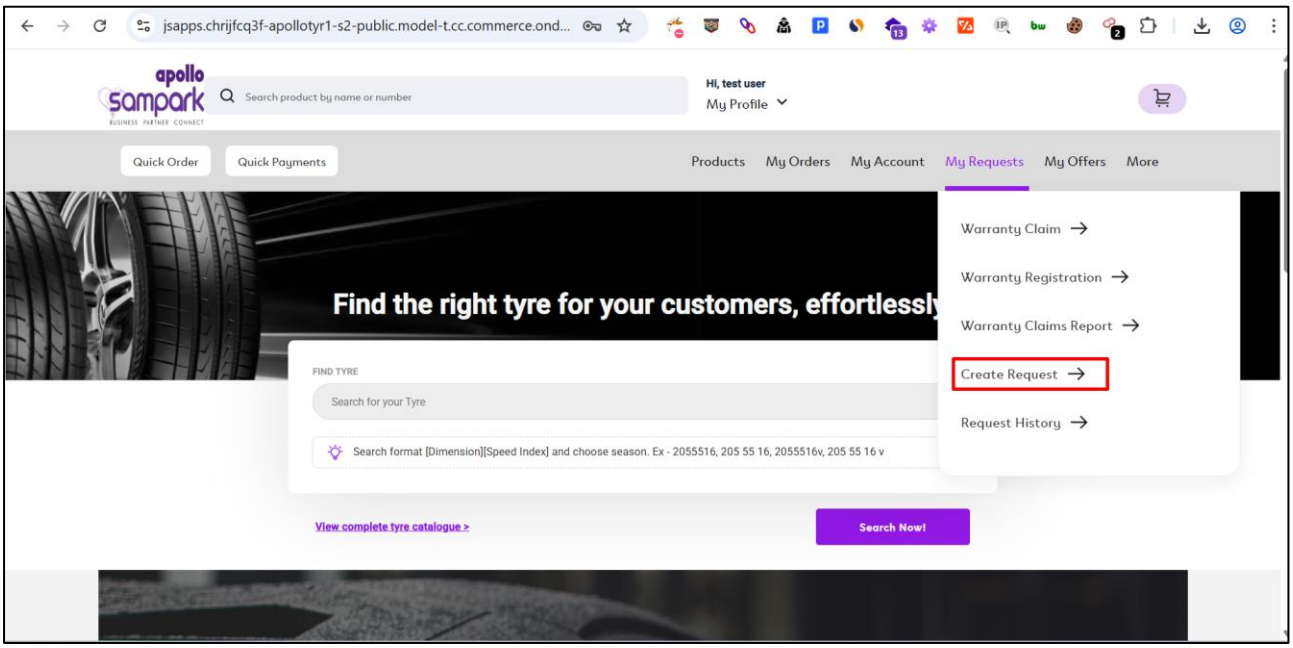
- Strip all EXIF metadata from images on upload at the server side
- Reprocess and re-encode images before storing or serving them
- Use image processing libraries to sanitize metadata (e.g., remove GPS, comments)
- Validate uploaded files and enforce secure file handling mechanisms

POC:

Steps to Reproduce

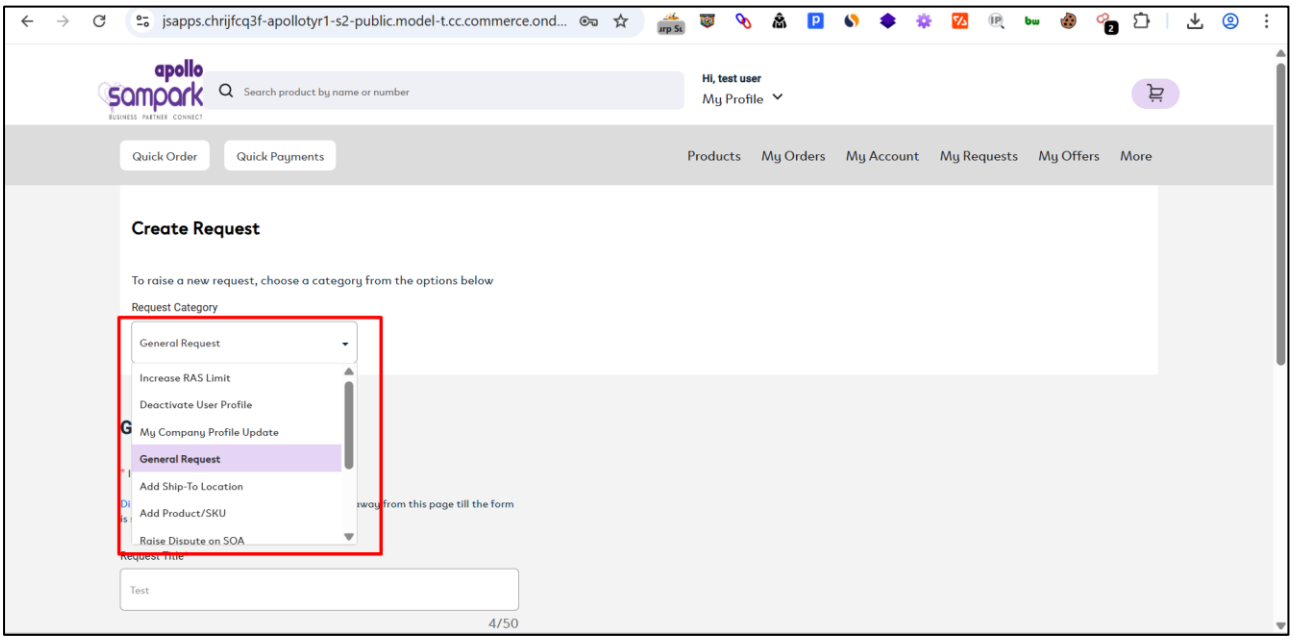
Step 1:

During the assessment, the auditor logs into the application, navigates to the *My Request* tab from the header, and clicks on *Create Request* from the dropdown as shown in the artifact below:



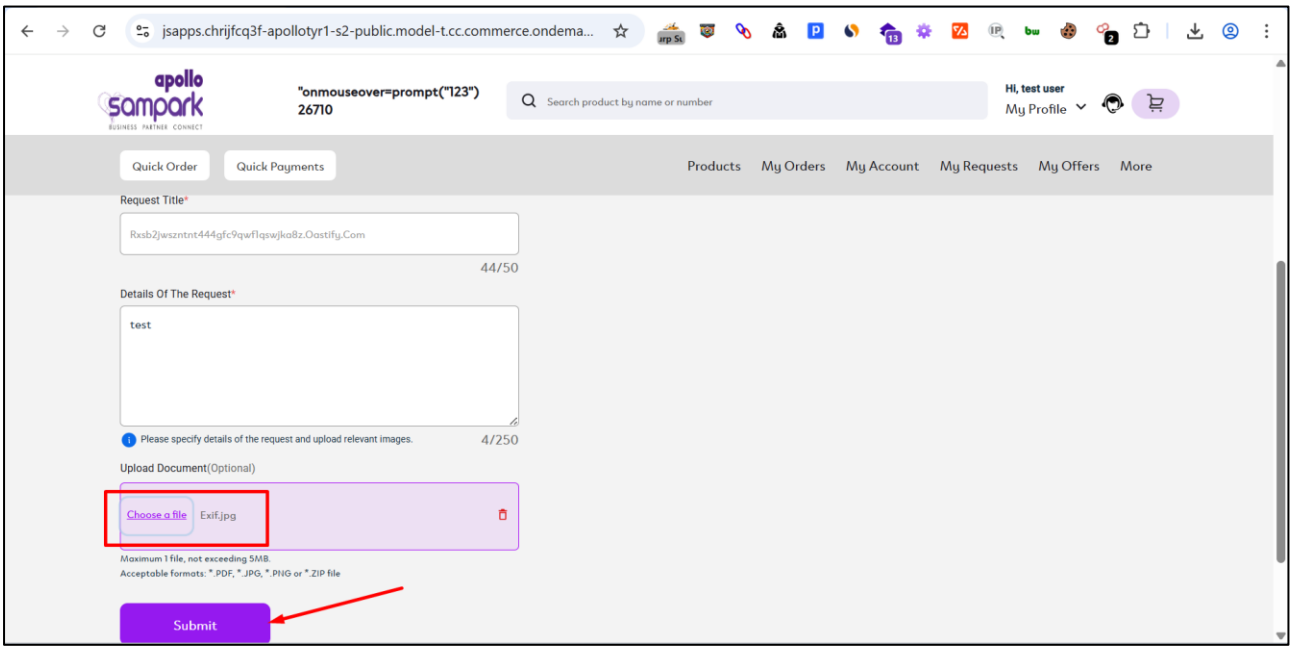
Step 2:

On the Create Request page, the auditor selects General Request and fills in the required details in the fields as shown in the artifacts below:



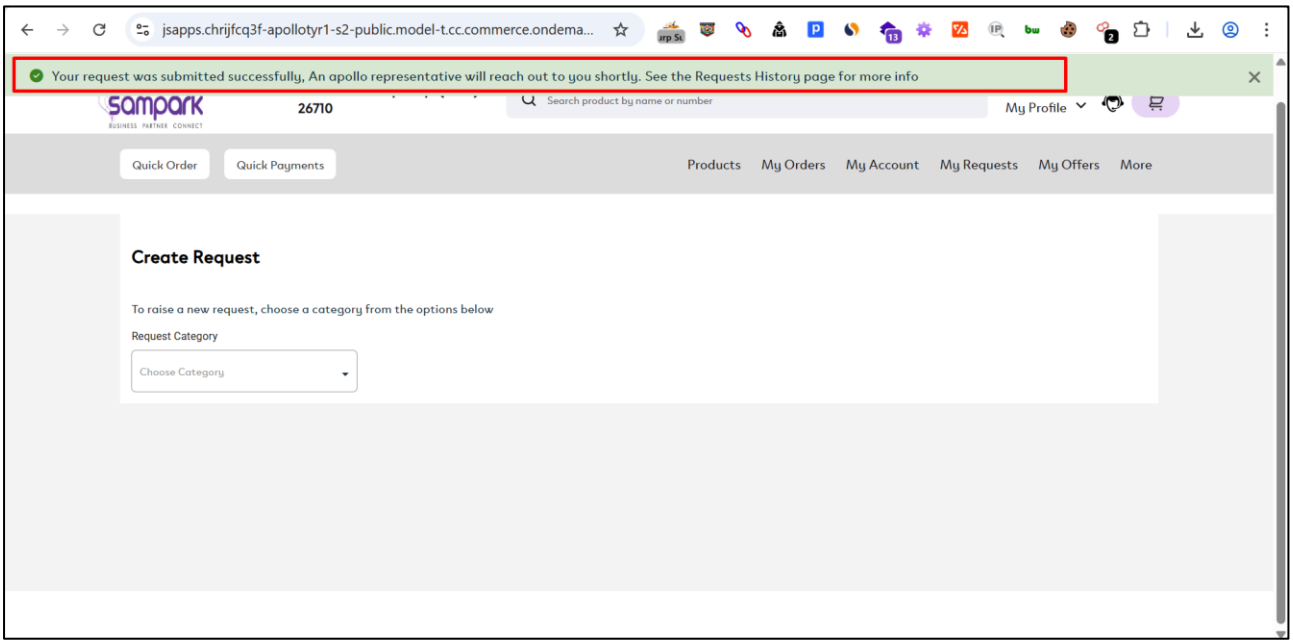
Step 3:

The auditor uploads an image file containing EXIF metadata (e.g., location/comments) and submits the request as shown in the artifact below:



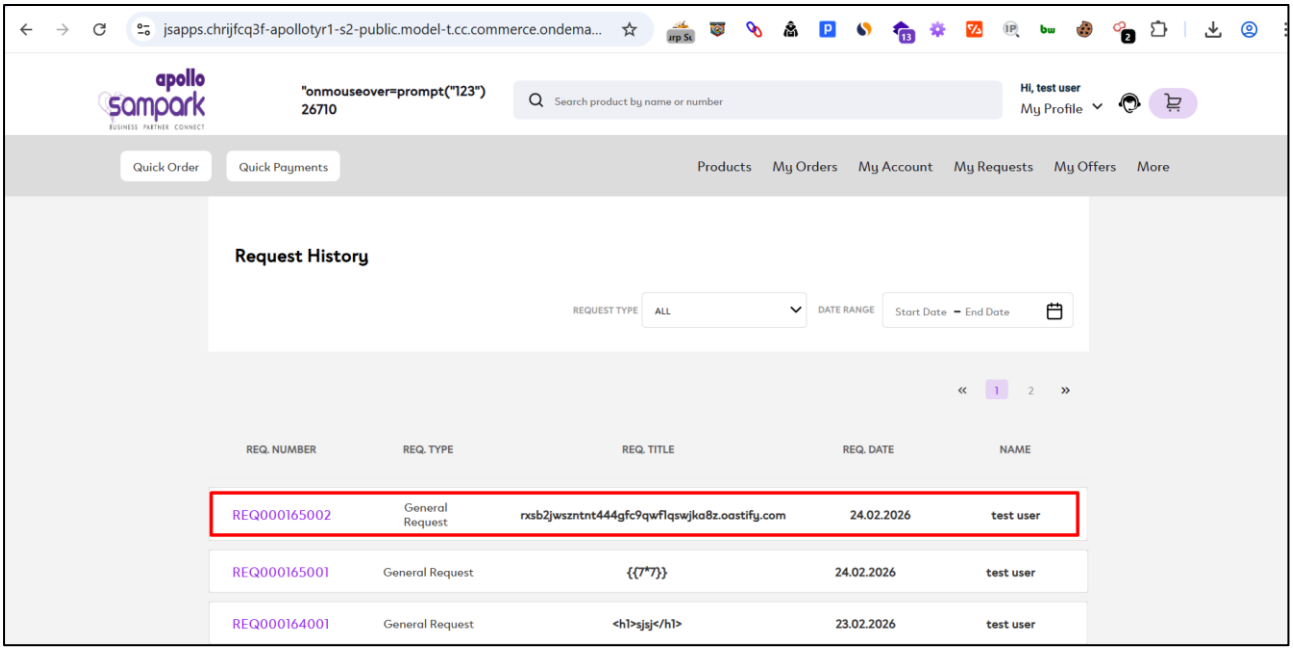
Step 4:

The auditor observes a confirmation message indicating that the request has been successfully submitted as shown in the artifact below:



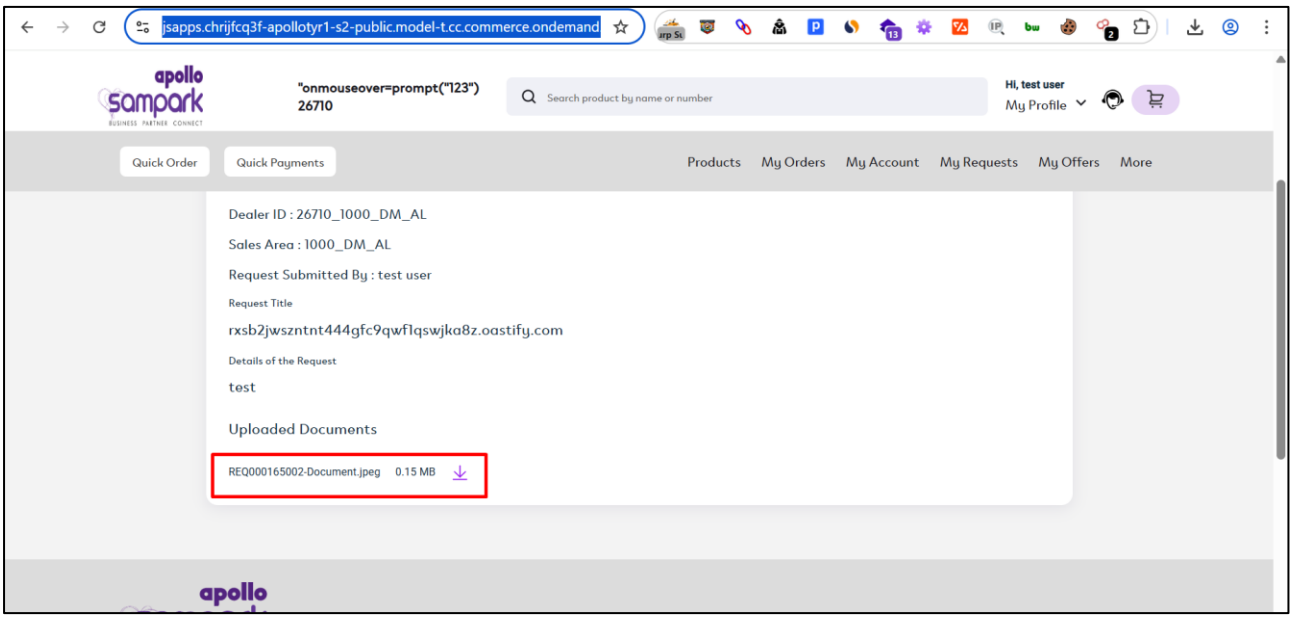
Step 5:

The auditor navigates the Request History page and locates the created request as shown in he artifact below:



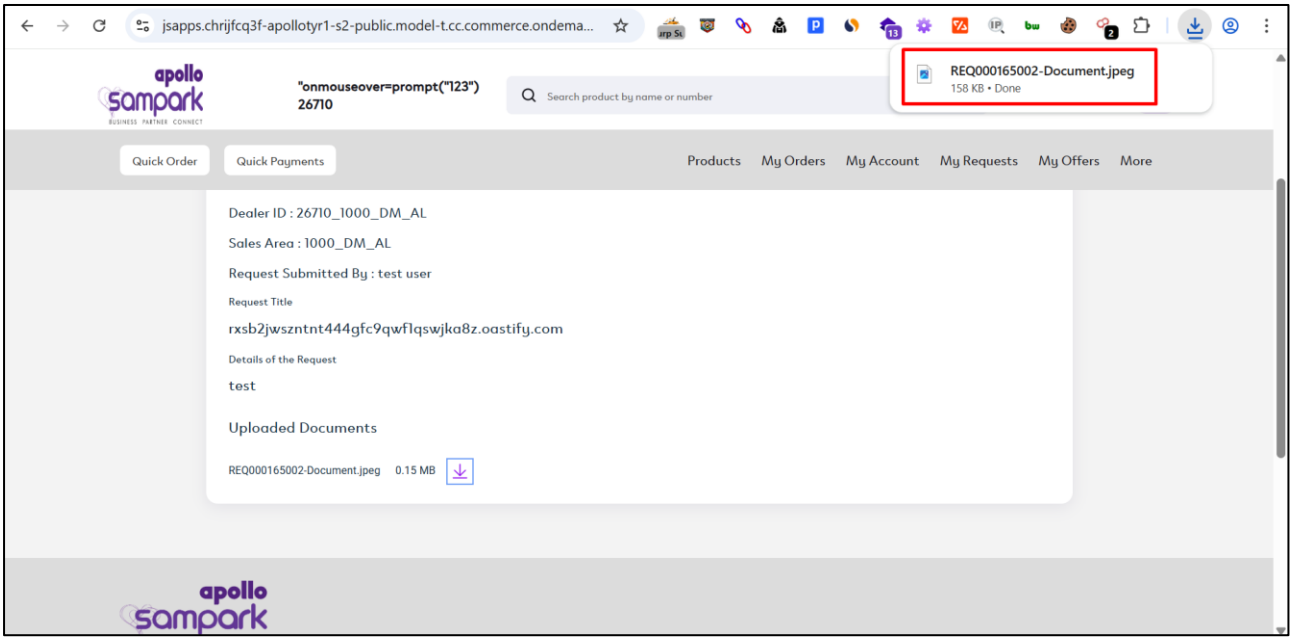
Step 6:

Upon opening the request, the auditor scrolls to the Uploaded Documents section and clicks on the available download option as shown in the artifact below:



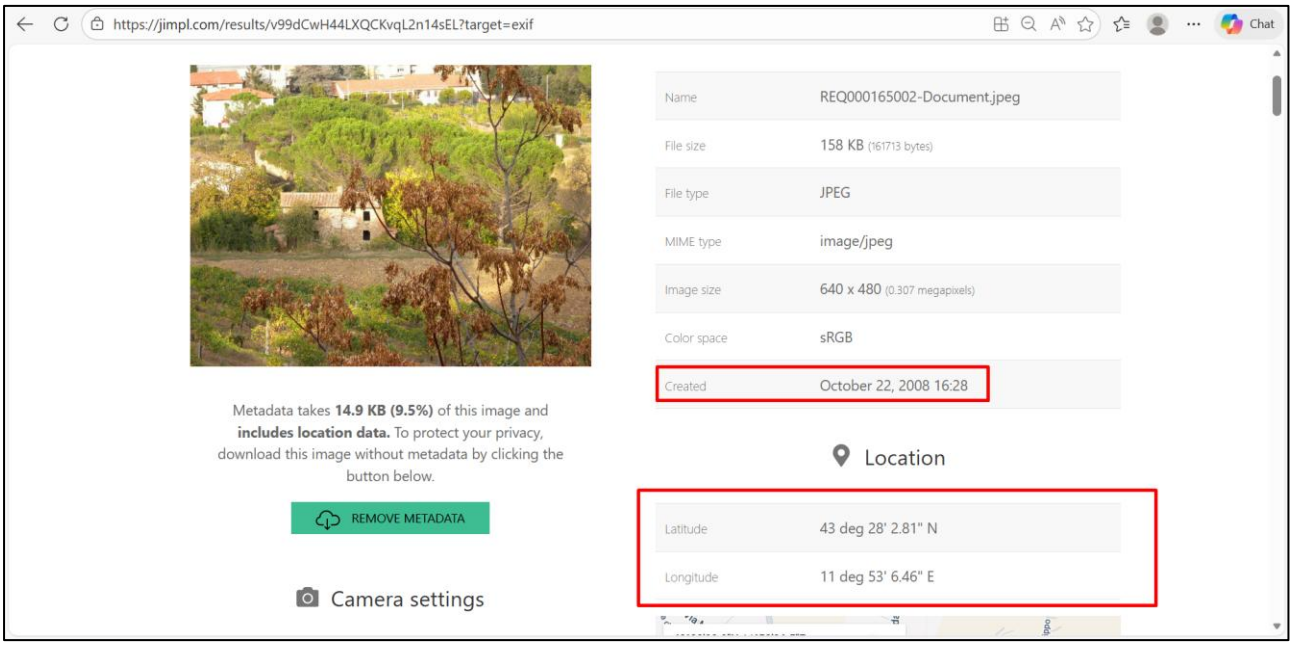
Step 7:

The uploaded image is downloaded from the application as shown in the artifact below:



Step 8:

The auditor re-uploads the downloaded image to a metadata viewer tool and observes that the EXIF metadata (including location/comments) is still present as shown in the artifact below:



3.11 Improper Input Validation

Description:

The application does not properly validate user input in multiple fields in application. It allows submission of malicious payloads such as script tags, which are reflected in the application (even if encoded). This indicates insufficient input validation and improper handling of user-supplied data.

Status:

Current Status: New

Overall CVSS Score:

3.1

Risk Rating:

Low

Impact:

Improper input validation may allow attackers to inject malicious payloads into the application. Although currently encoded, such inputs may lead to stored or reflected XSS if encoding is bypassed or inconsistently applied. This increases the risk of client-side attacks and impacts on overall application security.

Affected URL:

https://jsapps.chrijfcq3f-apollotyr1-s2-public.model-t.cc.commerce.ondemand.com/atl-in/en/MyRequests/warranty-listing

Affected Parameters:

Application

OWASP Ref No:

https://www.owasp.org/index.php/Improper_Data_Validation

CWE/CVE Reference No:

CWE-20

Recommendation:

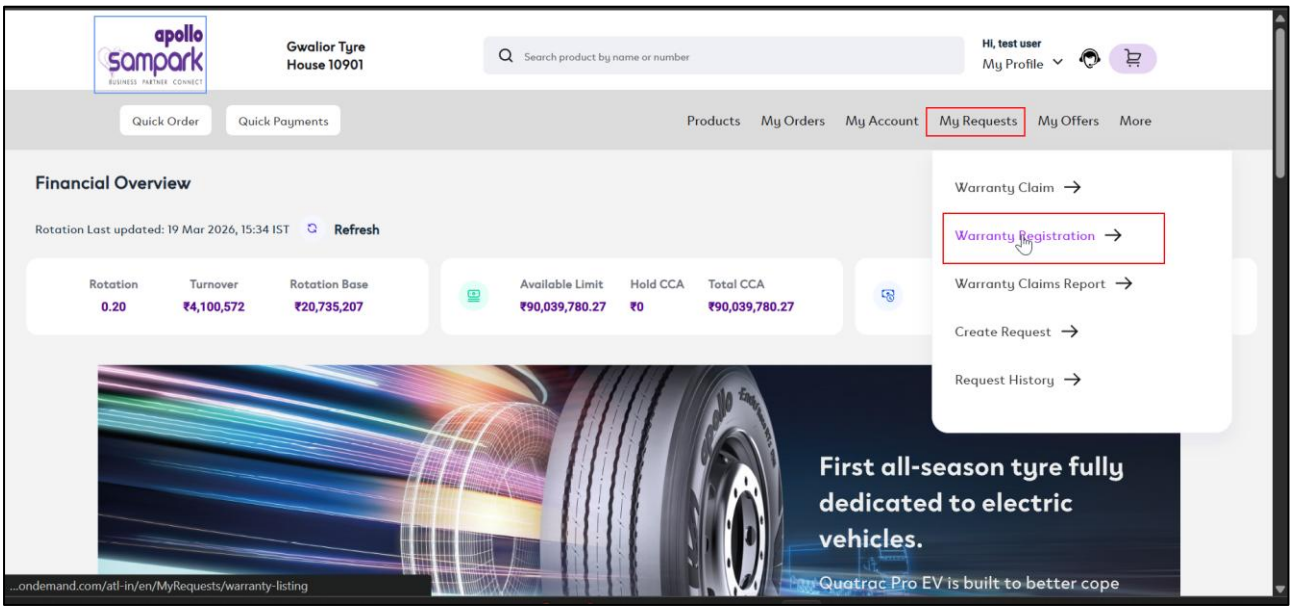
1. Implement strict server-side input validation to allow only expected characters (e.g., alphabets and limited safe symbols)
2. Reject or sanitize inputs containing HTML tags or special characters such as <, >, ", '
3. Apply input validation consistently across all user input fields
4. Use a whitelist-based validation approach rather than blacklist filtering

POC:

Steps to Reproduce

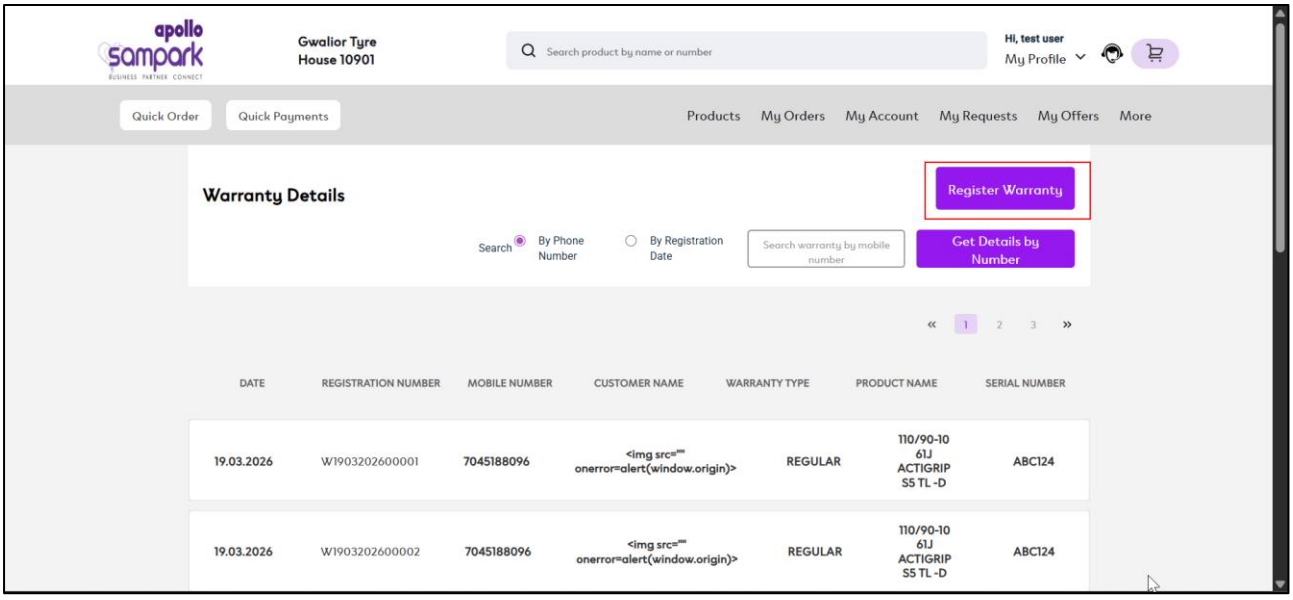
Step 1:

During the assessment, the auditor logs into the application, navigates to the *My Request* section from the header, and clicks on *Warranty Registration*, as shown in the artifact below:



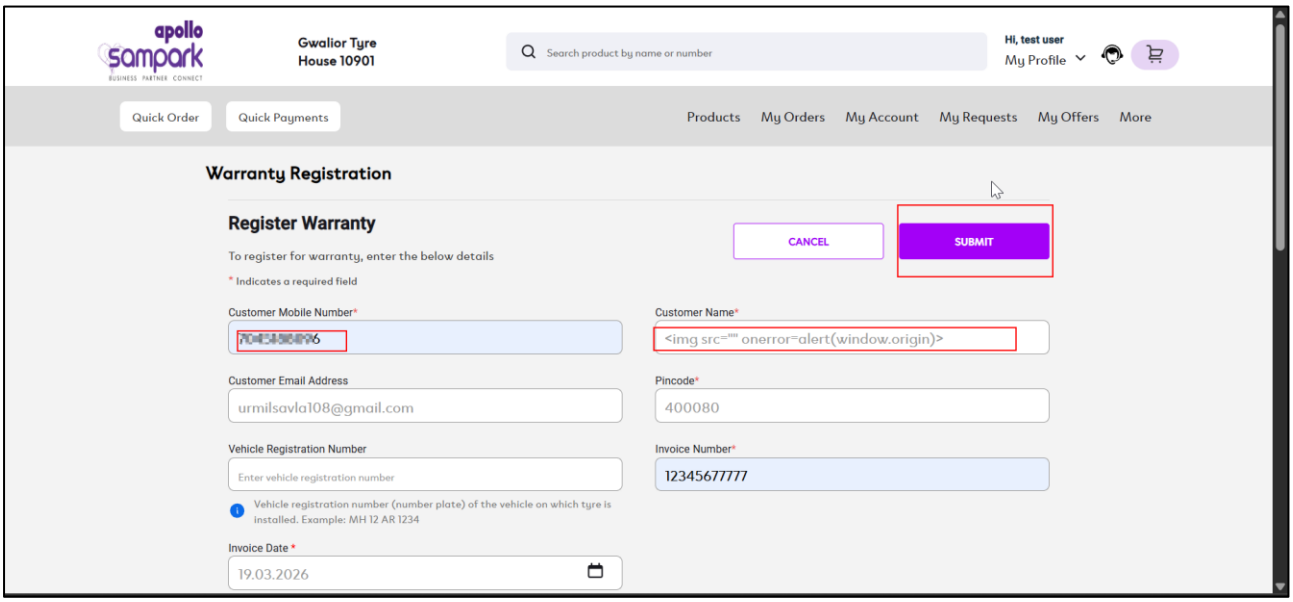
Step 2:

The auditor clicks on the Register Warranty button as shown in the artifact below:



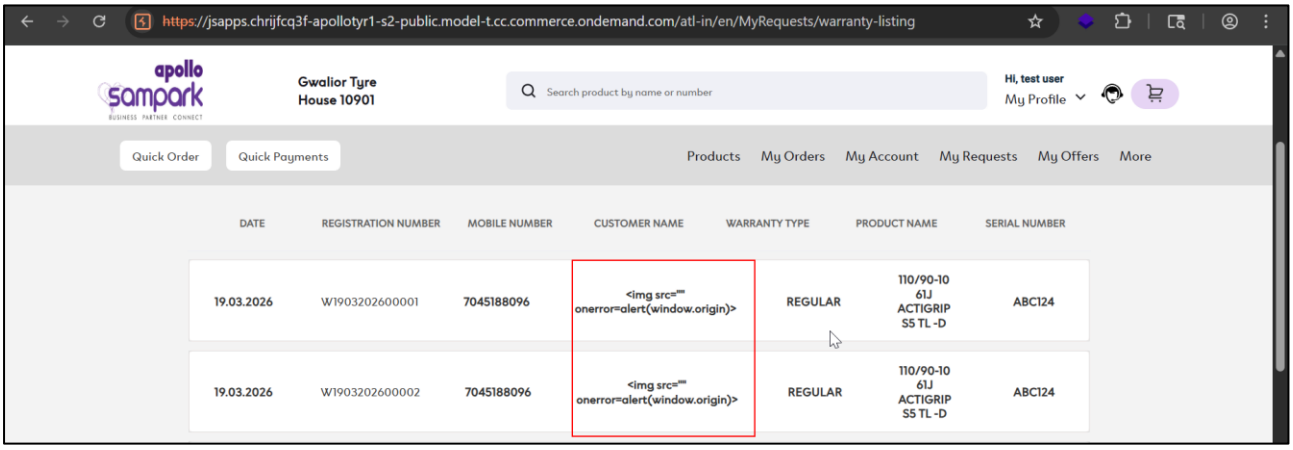
Step 3:

The auditor fills in the required details in the form and injects an XSS payload (e.g., <script>alert(1)</script>) in the Customer Name field, then submits the request as shown in the artifact below:



Step 4:

The auditor navigates to the warranty listing page and observes that the payload is reflected in the response (encoded but not sanitized), indicating lack of proper input validation as shown in the artifact below:



3.12 Application accepting same password

Description:

The application allows users to reuse previously used passwords during password changes. There are no controls in place to enforce password history or prevent reuse of old credentials. This indicates weak password policy enforcement.

Status:

Current Status: New

Overall CVSS Score:

3.1

Risk Rating:

Low

Impact:

Allowing password reuse reduces the effectiveness of password rotation policies. Users may continue using the same passwords, increasing the risk of compromise if credentials are exposed. It weakens overall account security posture.

Affected URL:

https://jsapps.chrijfcq3f-apollotyr1-s2-public.model-t.cc.commerce.ondemand.com/atl-in/en/update-password

Affected Parameters:

Application

OWASP Ref No:

https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/04-Authentication_Testing/07-Testing_for_Weak_Password_Policy

CWE/CVE Reference No:

CWE-521

Recommendation:

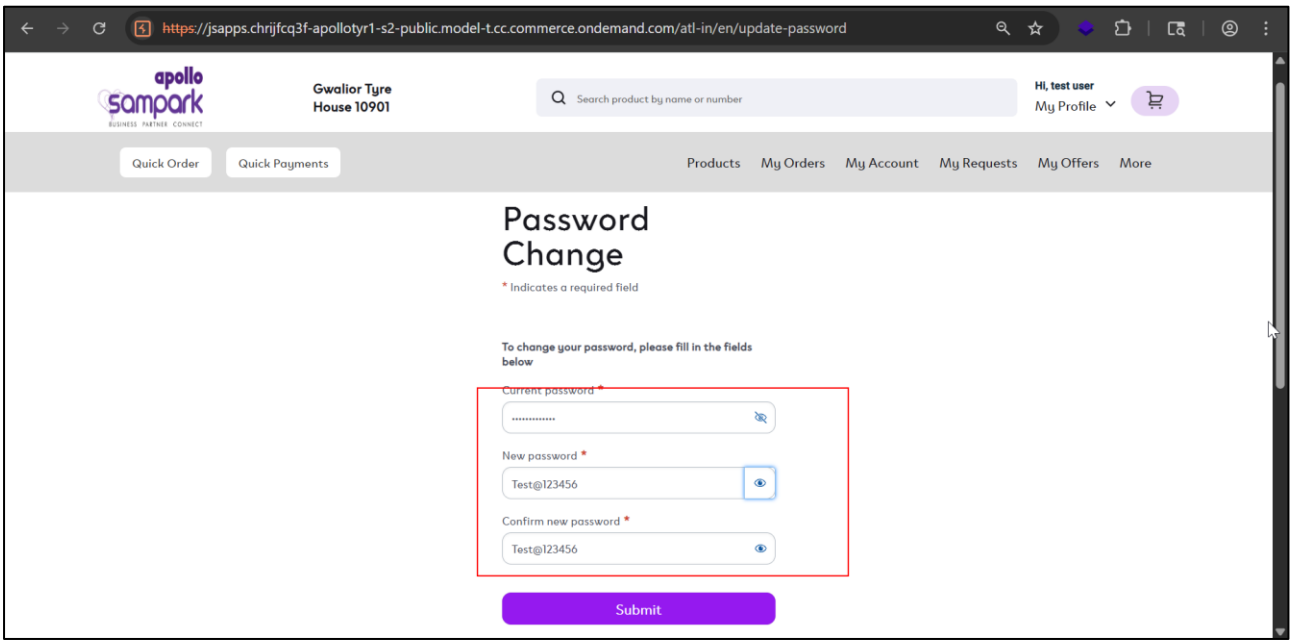
1. Enforce password history to prevent reuse of recent passwords (e.g., last 5 passwords)
2. Implement strong password policy controls during password change
3. Validate new passwords against previously used credentials on the server side
4. Provide user guidance to create unique and strong passwords

POC:

Steps to Reproduce

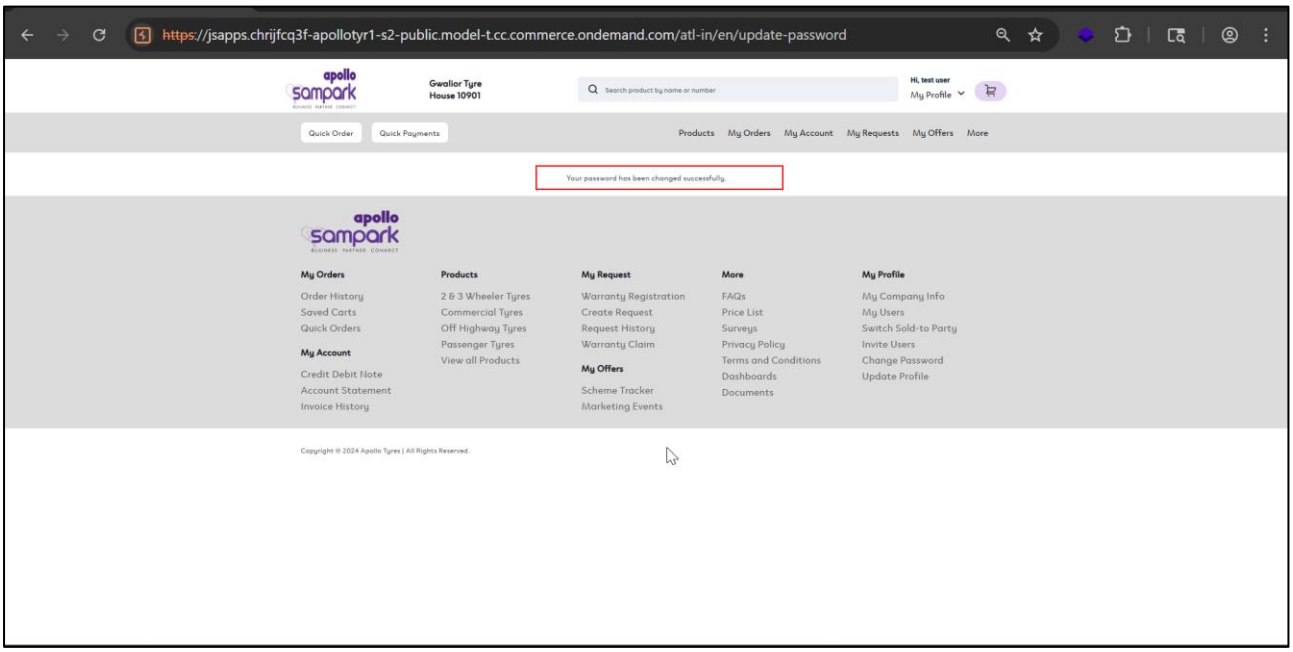
Step 1:

During the assessment, the auditor logs into the application and changes the account password from **test@123456** to **123XYZ@123456** as shown in the artifact below:



Step 2:

The auditor again attempts to change the password and sets it back to the previously used password (test@123456), observing that the password change is accepted successfully as shown in the artifact below:



4. CONCLUSION

Web Application Security Assessment of **Apollo Sampark (B2B Application)** carried out from 16th March 2026 – 23rd March 2026. The goal of the test was to identify all vulnerabilities within the application provided by Apollo Tyres.

Critical, High, Medium & Low vulnerabilities were found during the Assessment and recommendations were made to counter the effects of the found vulnerabilities.

Regardless of the frequency of the Security Audits, no system can be considered 100% secure as new exploits and attacks are discovered daily. Conclusion me i have to add different severity and different severity details